claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_7049e1ed-b93\agent-a80fcf97c631fce5a.jsonl`
- SHA-256: `11fa38b7112d314485823ceb0436e9f43b042a2fa6a98e072b663f5c417004a3`
- Source modified: `2026-07-02T04:57:13+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_7049e1ed-b93`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T04:45:39.414Z)

You are a senior engineer producing ONE isolated PR. Today is 2026-07-02.
REPO: C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector (a shared checkout ? follow the safety rules exactly).
AUDIT FINDINGS YOU ARE FIXING: R2-6 (golden-corpus registry rot) + critic #1 ? Grep C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md for these IDs and read the full evidence + verifier notes before coding.
TASK BRIEF:
In C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector: (1) add a 'curate verify' subcommand (or extend the existing curate CLI) that resolves every registered videos.local_path and reports dangling rows (non-zero exit with --strict); (2) re-point the 6 owned curated rows (adarsh_avi_singles, kushagra_singles, largetest_doubles, gbaaddy, testlarge_short, mahadevpura_singles) to where the files ACTUALLY exist now ? verify on disk first: check 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup/' (Collect/Golden Labelled) and if the files are not on C: at all check the F: drive paths documented in C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/docs/DATA_IN_GCS.md §5/§6 ? point at the real existing location, and only for files you personally confirmed exist (Test-Path/ls); if some files exist nowhere reachable, leave those rows and flag them in the PR body; (3) the DB is a git-tracked binary sqlite ? commit the updated DB and note the binary-diff in the PR body; also purge or clearly mark the two 'watch?v=example' placeholder junk rows (critic finding) if trivially safe. Add tests for the verify logic (use a temp DB).

BRANCHING SAFETY (mandatory ? shared checkout, sibling sessions exist):
1. In the MAIN repo checkout run: git fetch origin
2. Create an ISOLATED WORKTREE: git worktree add "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-<shortname>" -b <branch-name> origin/master   (branch explicitly FROM origin/master ? never from the current local branch; use origin/main for repos whose default is main)
3. Do ALL work inside the worktree. Before committing: git branch --show-current + git log --oneline -1 must show YOUR branch on the origin-master base.
4. Stage EXPLICIT paths only (git add <file> <file>) ? NEVER git add -A / . / -u
5. Before opening the PR: git log --oneline origin/master..HEAD must list ONLY your commits.
COMMIT/PR CONVENTIONS:
- Commit message: conventional style matching the repo's history; end the message with a blank line then: Co-Authored-By: Claude Fable 5 <noreply@anthropic.com>
- PR body: what/why, a "## Verification" section stating exactly what you ran locally and what CI covers, and end with: ? Generated with [Claude Code](https://claude.com/claude-code)
- Do NOT merge the PR. Do NOT remove the worktree (a fixer may need it). Return the PR number, branch, worktree path.
QUALITY BAR:
- Read the repo's CLAUDE.md first; for the engine also docs/CODE_MAP.md §0 (file placement) before adding any new file.
- Add/extend tests for every behavior change. Run the targeted tests you touched (python -m

pytest tests/<relevant> -q) + ruff check on changed files; if mypy config exists, run mypy on touched modules. Full suite is CI's job ? do not run it.
- Default-safe: new knobs default OFF / behavior-preserving unless the task explicitly says otherwise.
- The full audit evidence is in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md ? Grep it for your finding IDs and READ the verifier notes; they contain corrections (exact file:line, scoping traps) you MUST honor.

If the brief's premise turns out false against current code (something already fixed, file moved), adapt honestly and say so in notes ? never force a pointless diff. If you genuinely cannot complete (env broken, auth missing), open NO PR and explain in notes with pr_number=0.

2. user (2026-07-02T04:45:50.668Z)

```
README.md
__pycache__
cleanup.py
config
data
docs
pyproject.toml
src
tests
```

3. user (2026-07-02T04:45:50.778Z)

```
486:### R2-6 [P1/M] ? sports-data-collector
```

4. user (2026-07-02T04:45:57.070Z)

```
440     ## Confirmed findings ? round 2 (8, previously unverified)
441
442     ### R2-1 [P2/S] ? rally-annotator
443     **Content script runs a 3 Hz full-DOM shadow-piercing scan on every page the user visits, even with the panel hidden**
444     - verdict: CONFIRMED
445     - reasoning: Every element of the claim reproduces in the current working tree (rally-annotator @ main 3d604bc, clean):
446
447     (1) Runs on every page: web/entrypoints/content.ts:13 `matches: ["<all_urls>"]` (top frame only per line 16), and web/wxt.config.ts:13 `host_permissions: ["<all_urls>"]`. content.ts:32-44 unconditionally calls `mountPanel(...)` then `panel.toggle()` ? the panel starts hidden (display:none) but stays mounted.
448
449     (2) 3 Hz loop regardless of visibility: web/src/ui/panel.ts:376 `const clock = window.setInterval(render, 333);` starts at mount and is cleared only in `destroy()` (panel.ts:395), which is never called from production code ? only from web/test/panel.smoke.test.ts. `toggle()` (panel.ts:391-393) merely flips display:none; it does not pause the interval.
450
451     (3) Each tick triggers a full-DOM shadow-piercing scan on video-less pages: render() calls `video.now()` (panel.ts:264) ? `active()` (web/src/video/directVideo.ts:55-58) ? `findActive()` ? `collectVideos(document, ...)` (directVideo.ts:37-39), which does `root.querySelectorAll("video")` plus `root.querySelectorAll("*")` with a per-element `chrome.dom.openOrClosedShadowRoot` call, recursing into every shadow root to depth 8 (directVideo.ts:25-32, 13-23). Crucially, when no video exists, `this.current` is set to null (directVideo.ts:40-43), so the cache NEVER fills and every 333 ms tick repeats the full scan. Each tick also rewrites the hidden panel's DOM (status.innerHTML, list.replaceChildren, panel.ts:265-281).
452
453     (4) No fix landed: git log for content.ts/panel.ts/directVideo.ts shows only ad6ac9c (initial), 5ec1831 and 0a05ecf (i18n) ? no gating change. No
```

visibilitychange/IntersectionObserver/MutationObserver/activeTab anywhere in web/ (grep: only hit is the setInterval itself).
454
455   (5) Intent: `<all_urls>` is deliberate (DESIGN.md LD-7: panel hidden by default, icon is the explicit "annotate this page" gesture) ? but LD-7 gates only the UI, not the scan loop, so the perf cost contradicts the design's own "not intrusive" rationale.
456   - notes: Sharper failure scenario: user installs the extension and opens ordinary video-less pages (Gmail/Docs/GitHub ? tens of thousands of DOM nodes, many shadow roots). In every such tab, every 333 ms, forever, the content script walks the entire DOM (`querySelectorAll("*")`) and makes one chrome.dom.openOrClosedShadowRoot call per element ? continuous per-tab CPU burn and laptop battery drain, plus jank on heavy pages, with the panel never once opened. Cost scales with number of open tabs. On pages WITH a video the cache (directVideo.ts:55-57) makes ticks cheap, so the pathological case is precisely the "every other page" case the finder described.
457
458   Severity P2 not P1: no correctness/data-loss/security impact, and the extension is version 0.1.0/private (not store-published), so blast radius today is dev machines; would be P1 if shipped broadly.
459
460   Correctly-scoped fix (S): (a) don't start the 333 ms clock until the panel is first shown and pause/clear it when hidden ? gate it inside `toggle()` in panel.ts; (b) make render's clock read non-scanning (e.g. a `peekNow()` on DirectVideoHandler that uses only the cached element), keeping the full scan for explicit triggers (Refresh button, panel open). Optional follow-up (M, separate decision): replace the `<all_urls>` content script with activeTab + scripting.executeScript on icon click, which eliminates injection on every page entirely and drops the broad host permission ? aligns with LD-7's own rationale, worth raising but not required to fix the perf debt. Not OWNER_GATED ? pure code change, no spend or product pivot.
461
462   ### R2-2 [P2/S] ? rally-annotator
463   **Locked decision LD-i8 (web language persisted + seeded from browser language) is documented as shipped but not implemented ? panel always boots in English and forgets the choice on reload**
464   - verdict: CONFIRMED
465   - reasoning: Doc claims shipped: rally-annotator/docs/LOCALIZATION.md:55-57 (LD-i8: "Web: persisted in chrome.storage.local (seeded from the browser language; default en)") and :99-100 mark the web i18n phases done (PR #18 "selector", PR #19); web/README.md:22-23 says "persisted in-panel language selector". Code says otherwise: (1) web/src/i18n/index.ts:56 boots the module-level locale to DEFAULT_LOCALE ("en") ? a fresh content-script injection always starts English; (2) negotiateLocale (index.ts:72-80), which implements the stored?navigator.languages?en order, has ZERO runtime callers ? grep across web/ shows its only callers are unit tests (web/test/i18n.test.ts:47-52); (3) panel.ts:18 declares onLocaleChange with the comment "optional: persist the choice (content script wires this)", but the actual mountPanel call in web/entrypoints/content.ts:32-42 passes only annotator/video/identity/download ? onLocaleChange is never wired, so the change handler at panel.ts:306-311 updates only the in-memory `current`; (4) no storage read/write of any language key exists anywhere: persist/store.ts handles only "rally:"-prefixed rows, background.ts (all 27 lines) handles only downloads and icon clicks. Git: repo is at origin/main (3d604bc); no commit after 0a05ecf (#19) touches web locale persistence ? 3d604bc ("docs: sync READMEs, multi-agent audit fixes") actually re-affirmed the false "persisted" README claim. The E2E (web/e2e/extension.spec.ts:80-101) switches the locale within one page session and never reloads, so it can't catch this. Failure scenario: a Hindi rater (browser hi-IN) opens a video ? panel boots in English despite the "seeded from browser language" claim; they select ??????; on the next page load/navigation the panel is English again and the choice must be re-made on every video.
466   - notes: Claim verified exactly as stated; not OWNER_GATED (LD-i8 is already the locked decision ? this is pure implementation). Sharper framing: both halves of LD-i8-web are missing (no boot-time seeding AND no persistence), yet all plumbing already exists and is tested, which is why the fix is S: in content.ts main(), read e.g. `lang` from browser.storage.local, call setLocale(negotiateLocale(stored, navigator.languages)) before mountPanel, and pass onLocaleChange: (l) => void browser.storage.local.set({ lang: l }); add a unit test for the wiring and extend the Playwright E2E with a reload-persists-locale assertion (the current per-locale E2E never reloads, which is the coverage gap that let the docs drift). Scope note: the VLC half of LD-i8 (config-file persistence, PRs #20/#21) was not part of this claim and was not verified either way. Severity P2: no data loss/crash, but a locked decision is documented and README-advertised as shipped while absent ? it defeats the localization value prop for the

target hi/kn/te raters (English boot every page load) and the doc-sync commit 3d604bc propagated the false claim.

    ### R2-3 [P2/S] ? sports-data-collector
    Collector pins obsreport as a plain PyPI requirement while the name is unclaimed on PyPI ? dependency-confusion window and install broken today
    - verdict: CONFIRMED
    - reasoning: (1) Bare PyPI-style pin exists in current tree: C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector/pyproject.toml:38-40 ? `report = ["obsreport>=0.1.3"]`, with the comment at lines 32-37 itself admitting the intended form is a git+ssh direct reference. (2) Name is unclaimed on PyPI today (2026-07-02): https://pypi.org/pypi/obsreport/json returns 404, and `pip index versions obsreport` returns "ERROR: No matching distribution found for obsreport" (personally observed). (3) The private package really is named `obsreport` (sports-obsreport/pyproject.toml:6, version 0.1.3, tag v0.1.3 exists) and lives at the private repo Khelsutra/sports-obsreport (gh api confirms private=true; avidullu/sports-obsreport redirects there). (4) Contrast confirmed: the engine already pins it safely ? badminton-highlight-indexer/pyproject.toml:68 `obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3`. (5) No fix landed: the bare pin was introduced in commit 038424d and no later commit touched it (git log -S on pyproject.toml); HEAD adcd206, working tree clean. Failure scenario A (today): `pip install .[report]` in any clean env fails resolution ? the extra is uninstallable unless the sibling checkout was pre-installed editable. Failure scenario B (window): anyone can register `obsreport` on PyPI and publish >=0.1.3; a later `pip install .[report]` then silently executes attacker code (sdist build hooks/import) instead of failing loudly.
    - notes: Severity calibrated down from an implied P1: the exposure is real but narrow. `report` is an optional extra; the base install is unaffected; the collector treats obsreport as a soft dependency that no-ops when absent (sports-data-collector/src/reporting/__init__.py:15-22); and CI installs only `.[dev]` (.github/workflows CI line 19), so no automated pipeline ever resolves the bare name ? the confusion window only fires when a human/agent runs `pip install .[report]` in a fresh env (in the documented sibling-editable workflow, pip sees obsreport 0.1.3 already satisfied and never queries PyPI). It is still a standing security window on a generic, squattable name, plus a broken-today extra, in a repo whose sibling project already solved the identical problem. Fix (S, one line): replace pyproject.toml:39 with `obsreport @ git+ssh://git@github.com/Khelsutra/sports-obsreport.git@v0.1.3` (prefer the canonical Khelsutra org path ? the avidullu path in the engine's pin only works via GitHub's transfer redirect and is itself mildly stale; consider updating badminton-highlight-indexer/pyproject.toml:68 too). setuptools accepts PEP 508 direct references without extra config (no hatchling-style allow-direct-references needed), and since sports-data-collector is not published to PyPI, the PyPI ban on direct references does not apply. Optional hardening (owner-gated, not required for the fix): claim the `obsreport` name on PyPI with a stub to close the squatting window permanently.

    ### R2-4 [P2/S] ? badminton-highlight-indexer
    No timeout on any serving-path ffmpeg subprocess ? one hung ffmpeg permanently wedges the single-worker job queue
    - verdict: CONFIRMED
    - reasoning: Single-worker executor: backend/api/job_worker.py:47-49 (max_workers=1) with the in-code "O2 RISK" admission at :32-39 that one hung job blocks all subsequent jobs and "the executor itself cannot enforce timeouts on the Thread". Serving-path ffmpeg/ffprobe calls with NO timeout=: (compile job, POST /api/compile main.py:1594 -> _execute_compile -> compiler.compile_highlights at main.py:1686, no wrapper) backend/pipeline/stitcher/compiler.py:130 (ffprobe), :321 (per-clip libx264 encode), :494 (concat); (process job, _execute_processing main.py:1132) validators backend/pipeline/validators/format.py:43, pts_continuity.py:35, resolution_fps.py:15; backend/utils/video.py:18,:75 (chunking); backend/pipeline/segmenters/gemini.py:166; backend/pipeline/audio/hit_detector.py:53. The mitigations listed in job_worker.py:34-37 do NOT cover ffmpeg: run_bounded wraps only the Gemini generate call (backend/providers/gemini.py:233, op_timeout_sec=180); detector timeout_sec covers only GPU runner subprocesses (detectors/base.py:116, native_wasb_runner.py:134). Recovery is restart-only: fail_stale_jobs is called once at startup (main.py:2030; database.py:1122-1125) and marks ALL queued/running jobs failed. No fix landed: git head e410136; commit 24178a7 (#374) documents the O2 risk but adds no timeouts. Secondary confirmed defect: /api/source proxy warm (main.py:657 daemon thread) runs annotation_proxy.py:81 ffmpeg with no timeout; a hang leaks the _proxy_inflight dedup entry forever (main.py:607-620), permanently disabling proxy generation for that video (does not wedge the job queue).

- notes: Claim is literally true, but "permanently wedges" should be "wedges until server restart, and the restart sweep (fail_stale_jobs) then fails every queued job". Sharper failure scenario: POST /api/compile for a video whose stored local filepath sits on a network-backed mount (Google Drive streaming / SMB) ? storage.acquire_local_input is identity for local paths (main.py:1683) ? the mount stalls mid-read (or the MP4 is truncated/corrupt and ffmpeg's demuxer loops); subprocess.run at compiler.py:321 blocks forever; the compile job stays 'running'; every subsequent /api/process and /api/compile returns 202 but stays 'queued' indefinitely with zero error surfaced (drains submitted at main.py:1635 queue behind the blocked task on the max_workers=1 executor); only a full restart recovers, at the cost of failing the whole queue. Severity P2 not P0/P1: the trigger class is rare (mitigations already bound the historically hang-prone ops ? Gemini generate 180s, GPU/WSL detector timeout_sec 1800s, Cloud Run poll max_wait_sec), the risk is explicitly documented in-code, and restart recovers; but impact when triggered is a silent total outage of the job pipeline. Fix (S, mechanical): add timeout= to the ~10 serving-path ffmpeg/ffprobe subprocess.run/check_output sites (config-driven, e.g. 30-60s for ffprobe probes, stitching-level 1800s default for encode/concat/chunking) and map subprocess.TimeoutExpired to the existing clean-job-failure paths (compiler already converts CalledProcessError to a failed-job result at main.py:1687-1695; annotation_proxy already catches broadly at annotation_proxy.py:82). The broader per-job wall-clock timeout job_worker.py:38 calls a "future slice" is a separate M item and not required to close this finding. Not OWNER_GATED ? timeout defaults are an engineering choice with no spend/product implications.

R2-5 [P1/S] ? badminton-highlight-indexer

Engine CLAUDE.md 'Current state' section is ~3 weeks false ? claims platform/owned-model work 'has not started'

- verdict: CONFIRMED
- reasoning: Personally reproduced against the working tree and git history. (1) C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\CLAUDE.md:30-31 states "**No platform/owned-model implementation has started.**"; lines 34-37 say the "committed next PLATFORM slice = async job model" and the owned-model track "starts at a greenlit Phase-0 milestone... Do not start implementation without explicit owner approval"; lines 41-42 call `StorageBackend` + `ComputeBackend` registries "*Future:*". git blame shows lines 30-37 were last written 2026-06-10 (commit 62fb2fb) and never touched since, despite CLAUDE.md itself being edited 6+ times after (e735fd4 06-13 ... af1ccbe 06-30 ? all other sections). (2) Current-tree reality contradicts every one of those claims: backend/storage/ contains a StorageBackend ABC + local/gcs/gdrive/gdrive_api/s3/capabilities implementations; backend/compute/base.py + cloud_run.py exist (ComputeBackend); backend/api/job_worker.py + tests/test_async_jobs.py exist ? the "committed next slice" async job model already shipped (8bcf6ce #339 "make /api/compile async (202 + poll)", 2026-06-22); backend/flywheel.py + backend/worker + backend/eval/served_gate_a.py exist. Owned-model track: a555887 (2026-06-20) "M2 FINALIZED ? Gen-0 weights/0.1.0-l4 pushed"; 4134721 archives DECOUPLED_COMPUTE as "DELIVERED (M0-M2 + M3.5 on GCP)"; 334ad0f (#336) M11 data-flywheel; b72e6de (#319) training worker fix. 314 commits landed since 2026-06-13, through 2026-07-01 (e410136), including a recorded launch NO-GO decision 2026-06-30 (4a7bfbe #404) ? i.e., the project reached launch-consideration, the opposite of "not started". (3) No fix landed: git log -- CLAUDE.md shows the last commit touching it (af1ccbe, 2026-06-30) added only the bit-flip-PR convention + Python 3.10 line; the Current state section is untouched since 06-10. Bonus staleness in the same file: lines 18-19 claim rally_gate.py Gate A is "not wired into production" while backend/eval/served_gate_a.py and c8a493d "served Gate-A litmus" (#401) indicate a served Gate-A path now exists.
- notes: Sharper failure scenario: CLAUDE.md is auto-loaded project memory for every fresh agent session in this heavily agent-driven repo. A fresh session reads "No platform/owned-model implementation has started" + "Do not start implementation without explicit owner approval" and will (a) tell the owner the async job model is an un-started, gated future slice and propose designing/implementing what already exists (backend/api/job_worker.py), (b) treat StorageBackend/ComputeBackend as "Future" and re-design duplicates of backend/storage/ and backend/compute/, (c) misreport project status upward (e.g., in handoffs/audits) as pre-implementation when Gen-0 weights shipped and a launch NO-GO was formally recorded 06-30. This also directly violates the workspace's own "keep it honest ? reflects real, shipped state" convention. Correctly-scoped fix (S, one doc-only PR): rewrite CLAUDE.md "Current state" (lines 29-37) to reflect reality as of 2026-07-01 ? platform slice (async jobs, storage/compute backends, capabilities/assets endpoints) SHIPPED; owned-model M0-M2+M3.5 DELIVERED on GCP, Gen-0 weights pushed; launch NO-GO 2026-06-30 with pointer to #404/docs; fix line 41-42 ("Future" ? "implemented, see backend/storage/, backend/compute/"); fix lines 18-19 Gate-A "not wired" claim against backend/eval/served_gate_a.py; re-point "committed next slice" at whatever docs/NEXT_STEPS.md currently prioritizes. Not OWNER_GATED ? this is a factual truth-sync of docs

to shipped code, the same class as the repo's prior "audit truth-sync" commits (62fb2fb, 9736a3b); no spend or strategy decision required. Round-1 claim of "~3 weeks false" is accurate: section frozen 2026-06-10, contradicting implementation landed from ~2026-06-20 onward.
485
486   ### R2-6 [P1/M] ? sports-data-collector
487   **Golden-corpus registry rot: all 6 owned 'curated' videos point at a deleted absolute local_path and the 4.9GB source videos are single-copy with no documented backup**
488   - verdict: CONFIRMED
489   - reasoning: Registry rot half CONFIRMED by direct observation. sports-data-collector/data/sports_metadata.db (git-tracked; rows landed in commit 9e6f204 "#19 ingest the n=6 owned golden corpus", DB last touched 9f1e792 with no path fix since): videos table has 8 status='curated' rows; the 6 owned ones (adarsh_avi_singles, kushagra_singles, largetest_doubles, gbaaddy, testlarge_short, mahadevpura_singles; license_type=Proprietary, is_commercial=1) all carry local_path under 'C:\Users\avidu\Projects\Annotation Setup\Collect\Golden Labelled*.MP4' ? that directory does not exist. The folder moved to 'C:\Users\avidu\Projects\khelsutra-guru\Annotation Setup\' whose 'Golden Labelled\' now contains ONLY the six .rallies.csv label files; zero copies of the MP4s exist anywhere in the workspace (searched). The committed export artifact data/eval_export/manifest.json repeats the same 6 dead absolute paths. Code consequence at sports-data-collector/src/cli/curate.py:679-684: export of a curated video with missing local_path emits md5=None plus a warning, and src/core/manifests.py:23-26 documents md5 as "the indexer's join key" ? so any fresh export of the entire owned golden corpus loses the deterministic MD5 join to badminton-highlight-indexer. Data-loss half CONFIRMED as single-copy but the "no documented backup" wording is wrong: (a) verified live via read-only 'gcloud storage ls' (2026-07-02): gs://khelsutra-rally-corpus/videos/ has 10 objects and NONE of the six originals, while labels/ DOES hold all six (2026-06-08_adarsh_avi_singles.rallies.csv etc.) and trajectories are in the bucket ? annotation work-product is safe, source footage is not; (b) Google Drive golden folder (G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15) holds only the 7 proxy clips, not the original six; (c) the sole remaining copy is documented at F:\[Khelsutra] GoPro Hero Black 12\KhelSutra\Training\Golden Labelled\ per badminton-highlight-indexer/scratch/copy_videos_to_gcs.ps1:31-36 and docs/DATA_IN_GCS.md:153 (§6 Phase-2) ? and the F: drive is NOT currently mounted (only C: and G: exist), so even that copy's integrity is unverifiable today, and no MD5s of the six were ever persisted anywhere (eval_export/manifest.json predates the md5 field from PR #51; no proxy-registry rows).
490   - notes: Corrections to the round-1 claim: (1) Size is ~37.9 GB, not 4.9 GB ? per-clip sizes annotated in scratch/copy_videos_to_gcs.ps1:31-36 (10.9+7.7+11+6.7+1.4+0.19 GB); the ~4.9-5.2 GB figure matches the three unrelated Roora pilot videos under 'Annotation Setup/Archive/.../Annotation Videos/'. (2) "No documented backup" is false ? badminton-highlight-indexer/docs/DATA_IN_GCS.md §5/§6/§8 explicitly documents the F: archive location and a Phase-2 migration plan (tracker at doc line 246: 14/15 videos still unmigrated); but "single-copy" stands: exactly one copy of the six source videos exists, on an intermittently-mounted external HDD, unverifiable right now, with no persisted hashes. (3) Downgrade from potential P0 to P1 because labels + trajectories for all six ARE in gs://khelsutra-rally-corpus (verified), so GPU-free eval/litmus and synth-train survive a drive failure; what would be permanently lost is the proprietary source footage (no re-extraction via --trajectory-source detector, no re-annotation, no re-rendering). Sharper failure scenarios: (a) F: HDD failure = permanent loss of ~38 GB of irreplaceable owned golden footage; (b) today, 'curate export' on the owned corpus emits md5=None for all six (curate.py:679-684), silently breaking the indexer MD5 join contract added in PR #51. Correctly-scoped fix (M, cross-repo): mount F:, verify + hash the six MP4s (persist MD5s into the DB/manifest immediately ? S and decision-free), generate 1080p proxies, upload via the existing scratch/copy_videos_to_gcs.ps1 or the rclone cloud path (Phase-2, already planned), then repoint/annotate local_path (or add a gs:// video_uri column) in sports_metadata.db and refresh data/eval_export/manifest.json. Partially OWNER_GATED sub-decision only: DATA_IN_GCS.md §7.4 leaves proxies-only vs full-4K migration scope OPEN and the upload is real (though tiny, cents/month) GCS spend; the hash-and-record step and the DB path fix need no owner decision.
491
492   ### R2-7 [P2/S] ? non-git
493   **Roora vendor pilot record (paid deliverables + corrected golden labels + 3 videos) is a single unversioned local copy at risk of loss**
494   - verdict: CONFIRMED
495   - reasoning: CONFIRMED for the paid deliverables + corrected golden labels; the "3 videos" component of the claim is REFUTED (Drive copies exist). Evidence: (1) Single-copy artifacts verified on disk: raw paid CVAT deliverables at 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup/Archive/Roora Pilot Review - 6 June 2026/' ? badminton/{badminton,

badminton (1), badminton (2)}/Annotations/*.xml (~2.3 MB per-frame boxes), pickleball/*.zip (3 zips, 260 KB), tabletennis/*.zip (3 zips, 213 KB) ? plus the human-corrected golden labels 'Annotation Setup/golden_labels_badminton.csv' (2.5 KB, mtime 2026-06-06) and the rescued multisport goldens 'Annotation Setup/Trajectories/roora/{pickleball,tabletennis}_rallies.csv'. (2) Not in git: sports-data-collector tracks only the AS-DELIVERED parse (data/roora_badminton_canonical.csv, verified via git ls-files + diff ? it still has rally 2 = 0:02.503?0:32.533, rally 7 end 1:51.612, no rallies 7.5/16.5), while golden_labels_badminton.csv holds the corrections (rally 2 start?0:29, rally 7 end?1:38, missed rallies 7.5@1:46 and 16.5@3:34 added, winner?out fixes) ? those corrections exist NOWHERE else (find over C:/Users/avidu/Projects + Downloads returned only this one file). (3) Not in GCS: recursive 'gcloud storage ls -r gs://khelsutra-rally-corpus/**' contains zero objects matching roora|pilot|pickle|tabletennis|sample_long|golden_labels; labels/ holds exactly the 15 canonical corpus clips (pilot video not among them); archive/ holds only a droplet archive. (4) Not on Drive: 'G:/My Drive/Sharing' has guidelines/briefs and 'Golden Annotations' (MahadevpuraSingles only) ? no CVAT exports. (5) Outside every backup convention: badminton-highlight-indexer/docs/DATA_IN_GCS.md §5 gap table (lines 117?131) covers only the 15-clip corpus and never mentions the pilot artifacts (and its line 121/142 path 'C:\?\Annotation Setup\Collect\Trajectories\' is stale ? the folder moved, no Collect dir exists); the F: backup (memory checkpoint 2026-06-22) covered only the 7 new-clip labels; docs/archives/roora-vendor/README.md archives only guidelines+ADR, not pilot results; F: is currently unmounted. REFUTED part: the 3 pilot videos have byte-size-identical cloud copies at 'G:/My Drive/Sharing/Annotation Videos/Pilot Videos/' (Sample Long Video - Baadminton.MP4 = 2,882,861,900 B; Video 2 - Pickleball.mp4 = 1,435,276,306 B; Video 3 - Tabletennis.mp4 = 871,944,715 B ? all match local), and the local 'Annotation Videos/desktop.ini' references GoogleDriveFS.exe proving Drive File Stream provenance. Owner's own memory audit had flagged the precursor ("Roora multisport CSVs live ONLY in %TEMP%? one cleanup from loss"); the rescue moved them to Annotation Setup but they remain a single unversioned C:-disk copy.
496   - notes: Correction to the finding: the at-risk payload is NOT the 4.9 GB folder ? the videos are safe on the owner's Google Drive. What is genuinely one C:-disk failure (or one folder-cleanup) from loss is ~3 MB of paid, partly irreplaceable work product: the raw vendor CVAT exports for all 3 sports (box geometry the tracked canonical CSVs discard), the human frame-by-frame correction record (golden_labels_badminton.csv + rally_review.csv ? hours of review labor and the calibration evidence behind the vendor feedback), the pickleball/tabletennis golden CSVs backing the recorded TT-0.909/PB-0.308 multisport result, and the pilot review MD/PDF record. Vendor re-export from CVAT job 4075945 is not under owner control. Failure scenario: C: SSD failure or a cleanup of the non-git 'Annotation Setup' folder silently destroys the only record of the paid pilot and its corrected labels; nothing in DATA_IN_GCS.md's migration plan would ever pick it up because the pilot artifacts are absent from its §5 gap table. Fix (S, not owner-gated for the metadata): one 'gcloud storage cp -r' of 'Annotation Setup/Archive/Roora Pilot Review - 6 June 2026/' (excluding Annotation Videos/, already on Drive) + 'Annotation Setup/{golden_labels_badminton.csv,rally_review.csv,Trajectories/roora/}' to gs://khelsutra-rally-corpus/archive/roora-pilot-2026-06/ (archive/ prefix precedent already exists; ~3?10 MB, negligible cost), then add a Roora-pilot row to DATA_IN_GCS.md §5 and fix its stale 'Annotation Setup\Collect\Trajectories' path (lines 121/142). Including the 3 pilot videos in the bucket is the only owner-gated piece (DATA_IN_GCS §7 open decision 4, ~5 GB ? $0.10/mo) and is optional given the Drive copies.
497
498   ### R2-8 [P1/S] ? badminton-highlight-indexer
499   **Upload retention sweep tool exists but nothing schedules it ? unbounded upload growth + unenforced 7-day privacy retention on any hosted deployment (#301)**
500   - verdict: CONFIRMED
501   - reasoning: Tool exists: badminton-highlight-indexer/tools/cleanup_caches.py (T11 retention sweep, --uploads-retention-days default 7.0 at ~line 407; classify() purges uploaded videos + .partial-* in the uploads zone; landed cb7394b/#299). Nothing schedules it: workspace-wide grep for cleanup_caches hits only the tool/tests/docs and an error string (backend/pipeline/detectors/base.py:162); zero references in .github/workflows/nightly.yml, deploy/cloudrun/, deploy/digitalocean/, scripts/, and zero cron/timer/sweep references in khelsutra/deploy/droplet/ (khelsutra-engine.service has no companion timer). Backend startup only sweeps Gemini remote orphans (backend/main.py:1808 _maybe_sweep_gemini_orphans), not local uploads. Issue #301 is OPEN (viewed via gh today); no scheduling commit in git log between cb7394b and master e410136. Deployment is LIVE: khelsutra/deploy/DEPLOY_ALPHA.md line 3 "Status: LIVE (deploy live-verified on the droplet 2026-06-22)"; uploads land via backend/api/uploads.py into security.upload_dir (/srv/khelsutra/incoming/uploads per runbook §2.2).
502   - notes: Sharper failure scenario: the alpha droplet is live and accepting uploads

behind Cloudflare Access; without any scheduled `python -m tools.cleanup_caches --apply`, a tester's footage uploaded day 0 is still on disk day 30+ (unenforced 7-day privacy retention, an explicit owner policy per the tool docstring, 2026-06-21), and output/uploads grows unboundedly until require_free_bytes (backend/api/uploads.py:93) starts rejecting new uploads (service degradation on the small CPU droplet). Mitigations that keep this P1 not P0: runbook notes testers had not yet exercised the upload CUJ (OTP login + Gemini key rotation pending as of 2026-06-22), and the owner can run the sweep manually. Fix scope (S): do NOT re-implement the sweep ? ship a systemd timer + oneshot unit (or cron line) alongside khelsutra/deploy/droplet/khelsutra-engine.service that runs `python -m tools.cleanup_caches --apply --uploads-retention-days 7 --uploads-dir <security.upload_dir>` daily with --json output captured to journald; default-OFF/opt-in flag so local single-user installs are unaffected; document enable+verify steps in DEPLOY_ALPHA.md; one smoke test asserting the wrapper passes the right flags (tool logic already unit-tested in tests/test_cleanup_caches.py). Issue #301's acceptance criteria already spec this exactly. Partial owner gate: shipping the timer unit + docs is not gated, but *enabling* it on the live droplet is an ops action requiring the owner's SSH access, and #301 flags one open design call (whether apply should also skip uploads referenced by non-terminal jobs rows) ? recommend defaulting to the time-window-only behavior and letting the owner ratify.
503
504
505   ## Owner-gated (flagged, no action)
506
507   - [P2] badminton-highlight-indexer: **P0-rename remainder: person/venue-attributable video names still committed in code and baselines** ? Defect verified in current tree: backend/eval/rally_seg_eval.py:52-54 commits MULTICOURT_CLIPS = frozenset({"mahadevpura_2", "GX010128", "Badminton_BXH_2", "Boxhill_Doubles"}) (venue-attributable), and eval_baselines/heuristic_lovo_n6.json per_video keys are adarsh_avi_singles, kushagra_singles, largetest_doubles, gbaaddy, testlarge_short, mahadevpura_singles (person first names + venues). git log on rally_seg_eval.py (latest 0c9c693 #395) and on the baseline (#77) shows no rename ever landed. docs/GOLDEN_REGRESSION_FIXTURES.md:151 and docs/BACKLOG.md:61 track this exact remainder as DEFERRED per the owner's safe-scope decision (2026-06-16) with trigger "before any open-sourcing / public commit" ? a product/strategy owner gate ? and the coordinated fix requires renaming the owner-held gitignored output/human_lovo_manifest.json in lockstep or the gen0 LOVO training floor breaks. Repo is currently PRIVATE (gh repo view: Khelsutra/badminton-highlight-indexer, isPrivate=true), so no live public exposure today. Real, unfixed debt, but remediation is explicitly owner-gated on the open-sourcing decision and owner-side data coordination.
508
509   - [P2] khelsutra (avidullu/khelsutra): **Issue #114 stale-open: CF Access guest-login debugging against a deployment that no longer exists** ? Debt is real but the fix is an owner tracker call on the owner-gated alpha/serving track. Verified: (1) gh issue view 114 ? OPEN, createdAt == updatedAt == 2026-06-23T10:25:52Z, 0 comments; body is CF-dashboard-only triage of the Access allowlist for the droplet alpha and explicitly says "Deferred by owner 2026-06-23". (2) Cited anchor exists: deploy/DEPLOY_ALPHA.md:106 is the Allow-policy/Emails-allowlist step (line 105 is the app-domain step ? off by one). (3) Deployment gone: memory session-handoff.md:19 ? droplet claude-do-rally-test/168.144.126.176 deleted 2026-06-25, "the prior 'LIVE alpha on the droplet' ... is GONE ? alpha is paused"; api.khelsutra.guru CNAME/CF-Access flagged for cleanup. (4) Not already fixed: git log -- deploy/DEPLOY_ALPHA.md shows nothing relevant after 2026-06-23 (latest 6e4dbaf PR #113 is the unrelated GPU-VM doc); no in-repo reference to #114. (5) Nothing in-repo can fix or verify it ? the Access policy lives in the Cloudflare dashboard and the origin behind app./api. no longer exists. Fix requires owner disposition (close vs retitle), squarely under the alpha/serving go-live gate.
510
511
512   ## Critic additions (round 1 completeness pass)
513
514   - [P1/M] critical-fix: **Golden-corpus registry rot: all 6 owned 'curated' videos point at a deleted absolute path, and the 4.9GB source videos are single-copy with no documented backup** (`sports-data-collector/data/sports_metadata.db:0`)
515   - evidence: Query of the git-tracked registry: videos rows adarsh_avi_singles, kushagra_singles, largetest_doubles, gbaaddy, testlarge_short, mahadevpura_singles (all is_commercial=1, status=curated) have local_path='C:\Users\avidu\Projects\Annotation Setup\Collect\Golden Labelled\...' and video_url=NULL. `ls /c/Users/avidu/Projects/Annotation Setup` -> 'No such file or directory'; the data now lives at 'khelsutra-guru/Annotation Setup/Golden Labelled' (folder moved, registry never updated). src/cli/curate.py:224 registers

owned videos with `video_url=None, local_path=video_path` ? raw operator-supplied absolute path, no normalization or later integrity check. ShuttleSet rows are also CWD-relative with mixed separators ('./data/videos\shuttleset_1.mp4'). 'Annotation Setup/Archive' is 4.9G of source match videos; grep for gs://|bucket|backup across sports-data-collector docs/README returns nothing, while docs/MANUAL_INGESTION_POSITIONING.md:13 declares the collector the 'single canonical system-of-record' for golden rally labels.
516   - fix: Add a `curate verify` (or extend cleanup.py) that resolves every registered local_path and flags dangling rows; re-point the 6 golden rows to the current 'khelsutra-guru/Annotation Setup/Golden Labelled' location; store paths relative to a configurable corpus root instead of absolute machine paths; and back the 4.9GB Archive up to the existing GCS bucket, recording the gs:// URI in video_url so the registry has a durable pointer.
517
518   - [P2/S] ci-infra: **sports-obsreport has no CI at all ? combined with the engine's skipped reporting tests, zero reporting-stack code is gated anywhere** (`sports-obsreport/pyproject.toml:7`)
519   - evidence: `ls sports-obsreport/.github/workflows` -> no such directory (every other repo in the umbrella has workflows). The repo has 10 test suites (tests/test_customer_message.py ... test_spec.py) and a dev extra with pytest+pytest-cov, but nothing runs them on push/PR to github.com/Khelsutra/sports-obsreport (HEAD = v0.1.3, main). The engine consumes it as a tagged dependency (badminton-highlight-indexer/pyproject.toml:68) and the engine's own 17 obsreport-gated tests never run in CI (already-confirmed finding) ? so no test anywhere gates the reporting stack.
520   - fix: Add a minimal .github/workflows/ci.yml to sports-obsreport: checkout, pip install -e .[dev], pytest --cov on push/PR. This closes the library half of the reporting-stack gap independently of the engine's obsreport-lane work.
521
522   - [P3/S] hygiene: **Engine .git bloated to 281MB by ~229MB of dangling raw match-video blobs; 6.5MB yolov8n.pt is permanently in pushed history and .gitignore has no guard against re-staging raw videos** (`badminton-highlight-indexer/.gitignore:49`)
523   - evidence: `git count-objects -vH`: 4716 loose objects, 266.33 MiB loose vs only 2.61 MiB in packs; du .git = 281M. Four loose blobs are raw MP4s (ftypisom headers): 22.4MB, 31.7MB, 84.5MB, 90.2MB ? all unreachable even via `git rev-list --all --reflog` (dangling from an aborted `git add`), never gc-pruned. Separately, yolov8n.pt (6,549,796 bytes) is reachable history on origin/master (added 5257fd5, removed d19557e 'replace AGPL ultralytics YOLO'), so every clone carries the AGPL-era weights forever. .gitignore covers *.pt (line 5) but only two narrow mp4 patterns (lines 49-50: backend/static/highlights_*.mp4, *_compiled.mp4) ? nothing stops the raw-match-video staging accident from recurring.
524   - fix: Run `git gc --prune=now` in the local checkout to reclaim ~260MB, and add a broad guard (e.g. `*.mp4` with explicit `!` exceptions for any tracked fixtures, or ignore the video scratch dirs) to .gitignore. Optionally note yolov8n.pt in docs as accepted history weight; rewriting pushed history is not worth it at 6.5MB.
525
526   - [P3/S] tech-debt: **obsreport dependency URL still points at the pre-transfer avidullu org ? installs survive only via GitHub's transfer redirect** (`badminton-highlight-indexer/pyproject.toml:68`)
527   - evidence: pyproject.toml:68: "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3" (echoed in requirements.txt:35). The repo actually lives at Khelsutra/sports-obsreport ? `gh repo view avidullu/sports-obsreport` resolves to nameWithOwner Khelsutra/sports-obsreport, i.e. a transfer redirect. The redirect breaks the moment any repo is re-created under the old name, and like the already-flagged trackers pin this is a mutable tag, not a commit SHA. The local clone's own remote already uses the Khelsutra URL, so the pyproject reference is the only stale spot.
528   - fix: Update the dependency URL in pyproject.toml (and the requirements.txt comment) to github.com/Khelsutra/sports-obsreport.git and pin the v0.1.3 commit SHA instead of the tag, matching whatever convention the trackers-pin fix adopts.
529
530   - [P3/S] hygiene: **1.7GB of stale yt-dlp partial-download debris in data/videos and placeholder junk rows in the committed corpus registry** (`sports-data-collector/src/crawlers/badminton_crawler.py:231`)
531   - evidence: data/videos contains _diag.mp4.part (354MB), _t.mp4.part (516MB), _v.mp4.part (849MB) plus Frag*.part and .ytdl markers, all dated 2026-05-26 ? 1.7G total of dead partial downloads (du -ch *.part *.ytdl). The crawler's failure cleanup (badminton_crawler.py:231-246) removes only `local_path`, never yt-dlp's .part/.part-Frag*/.ytdl side files. The tracked sports_metadata.db also carries two placeholder rows with video_url='https://youtube.com/watch?v=example' (staging_match_non_comm, shuttleset_12) ? test

fixtures committed into the canonical registry.
532   - fix: Delete the stale *.part/*.ytdl files; extend the crawler's failure cleanup (or cleanup.py) to glob and remove yt-dlp fragment side files for the failed target; purge or clearly mark the two 'watch?v=example' placeholder rows out of the committed registry.
533
534
535   ## Unverified backlog (dropped at the round-1 cap; lower-ranked or duplicate)
536
537   - [P2] sports-data-collector: Stale doc pointers after archive moves (cross-repo DECISIONS.md path + 6 in-code CODE_HEALTH_PLAN references)
538   - [P2] sports-data-collector: curate.py is a 1531-line monolith (CLI + interactive review + import/export/proxy/licensing)
539   - [P2] sports-data-collector: Golden-label store is a binary SQLite file tracked in git ? unmergeable across the owner's concurrent sessions
540   - [P1] rally-annotator (Khelsutra/rally-annotator; local checkout C:/Users/avidu/Projects/khelsutra-guru/rally-annotator): Content script runs a 3 Hz full-DOM shadow-piercing scan on every page the user visits, even with the panel hidden
541   - [P1] rally-annotator (Khelsutra/rally-annotator; local checkout C:/Users/avidu/Projects/khelsutra-guru/rally-annotator): Locked decision LD-i8 (web language persisted + seeded from browser language) is documented as shipped but not implemented ? panel always boots in English and forgets the choice on reload
542   - [P2] rally-annotator (Khelsutra/rally-annotator; local checkout C:/Users/avidu/Projects/khelsutra-guru/rally-annotator): Issue #26 open and unaddressed: CSV carries zero provenance ? labels are unattributable to their source video except by filename
543   - [P2] rally-annotator (Khelsutra/rally-annotator; local checkout C:/Users/avidu/Projects/khelsutra-guru/rally-annotator): E2E never reloads a page: the two headline persistence behaviors (resume numbering from storage, locale persistence) have no end-to-end proof, and entrypoints/ are outside unit coverage
544   - [P2] rally-annotator (Khelsutra/rally-annotator; local checkout C:/Users/avidu/Projects/khelsutra-guru/rally-annotator): Issue #22 still fully pending: all six translation catalogs remain machine-draft ? the declared quality gate before any public launch
545   - [P2] rally-annotator (Khelsutra/rally-annotator; local checkout C:/Users/avidu/Projects/khelsutra-guru/rally-annotator): 'Annotation Setup' is the un-versioned annotation-data workspace (not a code duplicate) and its Roora pilot-review artifacts appear to exist nowhere else
546   - [P1] sports-obsreport: Collector pins obsreport as plain PyPI requirement while the name is unclaimed on PyPI (dependency-confusion window; install broken today)
547   - [P2] sports-obsreport: Engine (and both repos' comments) reference obsreport via the stale pre-transfer org URL avidullu/ instead of Khelsutra/
548   - [P2] sports-obsreport: sports-obsreport has zero CI ? tags/pushes are cut with no automated test or lint gate
549   - [P2] sports-obsreport: Engine-obsreport integration is never exercised in any CI ? the ~12 reporting tests only ever run on dev machines
550   - [P1] non-git dirs: "Annotation Setup" + "khelsutra-screencast" (workspace root C:/Users/avidu/Projects/khelsutra-guru): Roora vendor pilot record (paid deliverables + corrected golden labels + 3 videos) is a single unversioned local copy
551   - [P2] non-git dirs: "Annotation Setup" + "khelsutra-screencast" (workspace root C:/Users/avidu/Projects/khelsutra-guru): khelsutra-screencast: 8 ad-hoc QA scripts and doc-cited DoD evidence are unversioned; the 5 cited checkpoint screencasts are absent from the canonical dir
552   - [P2] non-git dirs: "Annotation Setup" + "khelsutra-screencast" (workspace root C:/Users/avidu/Projects/khelsutra-guru): DATA_IN_GCS.md §5 gap table contradicts its own §7: trajectories/labels marked 'not in bucket' though migrated 2026-06-22
553   - [P2] non-git dirs: "Annotation Setup" + "khelsutra-screencast" (workspace root C:/Users/avidu/Projects/khelsutra-guru): Stale-open #175: OneDrive golden-data sharing plan never executed and largely superseded by the GCS bucket
554   - [P1] khelsutra-guru workspace (cross-repo): badminton-highlight-indexer, khelsutra, sports-data-collector, rally-annotator, sports-obsreport: Vendored SPA bundle in engine is 10 web/ commits stale ? engine-served UI lacks the fps display its own API just shipped
555   - [P1] khelsutra-guru workspace (cross-repo): badminton-highlight-indexer, khelsutra, sports-data-collector, rally-annotator, sports-obsreport: Engine PR #390 (D1 main.py split) is now CONFLICTING against master and going staler
556   - [P1] khelsutra-guru workspace (cross-repo): badminton-highlight-indexer, khelsutra, sports-data-collector, rally-annotator, sports-obsreport: Engine CLAUDE.md 'Current state' is ~3 weeks false ? claims platform/owned-model work 'has not started'

```
557     - [P1] khelsutra-guru workspace (cross-repo): badminton-highlight-indexer, khelsutra,
sports-data-collector, rally-annotator, sports-obsreport: DO droplet retired 2026-06-25 but
runbooks in BOTH repos still say the deploy is LIVE on it
558     - [P2] khelsutra-guru workspace (cross-repo): badminton-highlight-indexer, khelsutra,
sports-data-collector, rally-annotator, sports-obsreport: sports-obsreport has zero CI ? no
.github directory at all despite shipping versioned releases
559     - [P2] khelsutra-guru workspace (cross-repo): badminton-highlight-indexer, khelsutra,
sports-data-collector, rally-annotator, sports-obsreport: Lint/type gates exist only in the
engine ? sports-data-collector CI is pytest-only, plus GH-Actions version drift across repos
560     - [P2] khelsutra-guru workspace (cross-repo): badminton-highlight-indexer, khelsutra,
sports-data-collector, rally-annotator, sports-obsreport: sports-data-collector working tree is
dirty: uncommitted .gitignore WIP + a full stale nested clone of itself
561     - [P0] badminton-highlight-indexer (Khelsutra): #340/#330: /api/process still re-bills
Gemini on re-submit of an already-processed video ? no idempotency short-circuit (live P0 cost
bug)
562     - [P1] badminton-highlight-indexer (Khelsutra): PR #390 (D1 main.py split, step 1) is
CONFLICTING and stale ? master diverged 9 commits on main.py since it was cut
563     - [P1] badminton-highlight-indexer (Khelsutra): #332: NSFW pre-flight gate still absent
? non-owner alpha users can route arbitrary video to Gemini (ToS/abuse + paid-garbage risk)
564     - [P2] badminton-highlight-indexer (Khelsutra): #327: path-jail still blocks gs://-style
ingest in hosted mode ? validate_input_path runs before scheme resolution
565     - [P2] badminton-highlight-indexer (Khelsutra): #388 premise is half-stale: the three
job-worker functions already live in backend/api/job_worker.py (since PR #234, 06-18) ?
remaining scope is shim removal + test patch-target migration
566     - [P2] badminton-highlight-indexer (Khelsutra): #384: coverage floor 70 and offline
guard lanes still duplicated verbatim between ci.yml and nightly.yml
567     - [P2] badminton-highlight-indexer (Khelsutra): #301: upload retention sweep tool exists
but nothing schedules it ? unbounded upload growth on any hosted deployment
568     - [P2] badminton-highlight-indexer (Khelsutra): #349 (+#331 overlap): WASB streaming
decode still forced single-threaded ? L4 GPU ~80% idle; #331 additionally needs a refresh
(droplet baseline retired, its lever 3 already shipped)
569     - [P2] badminton-highlight-indexer (Khelsutra): #328: per-video runlog/trace endpoint
still missing ? obsreport exists server-side but is not user-accessible
570     - [P1] Khelsutra/badminton-highlight-indexer: No timeout on any serving-path ffmpeg
subprocess ? one hung ffmpeg permanently wedges the single-worker job queue
571     - [P1] Khelsutra/badminton-highlight-indexer: MD5 segmentation reuse still unimplemented
? every re-submit of a byte-identical video re-bills Gemini and re-pays the full CPU pipeline
572     - [P1] Khelsutra/badminton-highlight-indexer: WASB streaming decode still single-
threaded (num_workers forced to 0), no NVDEC anywhere ? L4 GPU ~80% idle during detection
573     - [P1] Khelsutra/badminton-highlight-indexer: Upload retention sweep still manual ?
7-day privacy retention window unenforced without an operator (#301 open)
574     - [P2] Khelsutra/badminton-highlight-indexer: Remote-input scratch copies (rally-
input-*/rally-worker-*) are never reclaimed ? by-design temp leak outside all cleanup tooling
575
576
577     ## Unverified P3 pool
578
579     - badminton-highlight-indexer: regression.py silently passes when no baseline exists
(audit B8, still open) (critical-fix, `backend/eval/regression.py`)
```

5. assistant (2026-07-02T04:46:02.800Z)

```
Now let me read the repo docs and inspect the current state.
```

6. user (2026-07-02T04:46:03.737Z)

```
1       # Sports Data Collector & Curator
2
3       A clean, extensible Python codebase and CLI toolchain designed to discover, ingest,
validate, and curate sports video datasets. The repository stores annotations and video metadata
in an SQLite database, supports sport-specific tables and schemas, manages video storage outside
of Git, and enforces strict commercial license compliance.
4
5       > **Pipeline role:** this repo is the *data producer* feeding the **badminton-highlight-
indexer**, which trains/evaluates an offline rally segmenter on this curated, commercially-
```

licensed data. See docs/PIPELINE_ROLE.md for why `export`/`fetch` exist and how the two repos fit together.

Features

- **SQLite Backend**: Centralized `sports_metadata.db` with a generic `videos` table and sport-specific annotation tables (`badminton_rallies`, `tennis_points`, `cricket_deliveries`, `pickleball_rallies`, `tabletennis_rallies`).
- **Commercial License Flagging**: Each ingested source license is checked against a configured whitelist (`MIT`, `Apache-2.0`, `BSD`, `CC0`, `CC-BY`, `Public-Domain`, `Proprietary`) and the resulting `is_commercial` flag is stored alongside the video. Non-commercial sources (like `CC-BY-NC-SA`) are flagged as non-commercial but still ingested ? nothing is discarded automatically. The whitelist is read from `config/config.yaml` consistently across the parser, validator, and CLI.
- **Overlap & Duration Validation**: Ensures chronological event bounds (e.g. rally start/end times) do not overlap and that durations are positive.
- **Stroke-level CSV Parser**: Ingests ShuttleSet/CoachAI-style stroke CSV logs and aggregates them into rally durations using configurable start/end time buffers.
- **Local Ingestion Mode**: Manually registers local videos and associated CSV/JSON annotation files directly into the database as curated entries.
- **Interactive Curation CLI & Visual Play Harness**: Let's you query DB status, list staging entries, review sample timestamps, and approve/reject them. It includes a built-in play function that opens the video in your default browser at the exact rally segment (using HTML5 media fragments `#t=start,end` for local files or YouTube start time markers `&t=start` for remote videos) for manual visual verification.

Installation

Ensure you have Python 3.9+ installed.

1. Clone or download the repository.
2. Install the package and dependencies:
   ```bash
   pip install -e .            # core: curate / import-local / export / status
   pip install -e ".[fetch]"   # adds yt-dlp for video acquisition (fetch / crawler)
   pip install -e ".[dev]"     # test tooling (pytest, pytest-cov)
   ```

External requirements (not pip-installable)

The `fetch` command and the ShuttleSet crawler shell out to external binaries. They degrade gracefully (a clear error / skipped download) when missing, but are **required** for video acquisition:

- **FFmpeg / ffprobe** ? used to probe the real resolution/fps of downloaded files.
- **A JS runtime (e.g. [deno](https://deno.com))** ? needed by yt-dlp to solve YouTube's challenge for high-resolution formats. `fetch` auto-detects a deno install at `~/.deno/bin`.

Core curation/import/export/status commands work without any of these.

Usage

All commands should be executed from the root of the project. Make sure `PYTHONPATH` is set to the current directory:

```bash
# On Windows PowerShell:
$env:PYTHONPATH="."
```

```
```

1. View Database Status
To check the current statistics of staging vs. curated items:
```bash
python -m src.cli.curate status
```

2. Ingest External Datasets to Staging
To parse external raw stroke-level datasets (e.g. CoachAI format) and register them in staging:
```bash
python -m src.cli.ingest --sport badminton --parser coachai --file-path data/staging_coachai_match.csv --license CC-BY-NC-SA --video-id staging_match_non_comm
```

3. Launch the Interactive Curation Tool & Play Harness
To review staging items, inspect their timestamps, play individual rallies in the browser to visually verify their start and end bounds, and approve or reject them:
```bash
python -m src.cli.curate curate
```

4. Manually Import Local Videos & Annotations
To register a local video file alongside its JSON/CSV annotation file (bypasses staging):
```bash
python -m src.cli.curate import-local --sport badminton --video-path "data/videos/my_custom_match.mp4" --annotations-path "data/mock_local_badminton_anno.json" --match-name "My Custom Local Match" --license Proprietary
```
`import-local` works for every configured sport ? `badminton`, `tennis`, `cricket`, `pickleball`, and `tabletennis` ? from either a JSON or a CSV annotations file. For the canonical rally CSV schema and the rater-facing instructions used to produce these files (e.g. via an annotation vendor), see [`docs/annotation/`](docs/annotation/).

5. Fetch / Upgrade Videos to Best Resolution
Videos are **always downloaded at the best resolution available** (via yt-dlp `bv*+ba/b -S res,...`, merged to MP4); the actual width/height/fps is ffprobed and stored ? never assumed. The `fetch` command (re)downloads videos for entries already in the DB and updates `local_path`/`resolution`/`fps` **in place**, preserving all rally annotations and curation status (no clean slate needed):
```bash
# Upgrade every sub-720p video to best resolution, using a logged-in browser's cookies:
python -m src.cli.curate fetch --cookies-from-browser firefox

# A single video, forcing a re-download even if a good local copy exists:
python -m src.cli.curate fetch --video-id shuttleset_1 --force --cookies-from-browser firefox
```
Flags: `--sport`, `--video-id`, `--status {staging,curated,all}` (default `all`), `--min-height` (skip videos already at/above it unless `--force`; default `720`), `--max-height` (cap resolution, e.g. `1080`; default best available), `--force`, `--include-noncommercial`, `--cookies-from-browser`/`--cookies`, `--player-client`. A global resolution cap can also be set via `video_storage.max_height` in `config/config.yaml`. Re-runs are cheap and resumable; a download that falls below `--min-height` is treated as a **failure** (discarded, DB left unchanged) so a flaky low-res result never overwrites a good entry.

> **Authenticated cookies are strongly recommended.** Anonymous YouTube downloads are throttled and non-deterministic (often falling back to 144p). Pass `--cookies-from-browser firefox` (or `chrome`/`edge`/`brave`, fully closed) ? or `--cookies cookies.txt` ? to use a logged-in (ideally Premium) session for fast, reliable best-resolution downloads. Chrome's app-bound cookie encryption may block direct reads on Windows; Firefox reads cleanly even while open.
>

```
94      > **JS runtime:** yt-dlp needs one (e.g. [deno](https://deno.com)) to solve YouTube's
challenges for high-res formats; `fetch` auto-detects a deno install at `~/.deno/bin`. The
default `--player-client` avoids `web`/`web_safari`, which YouTube forces into SABR streaming
(drops to 144p).
95
96      ### 6. Export Annotations as Indexer-Ready Eval/Training Sets
97      To turn curated annotations into ground-truth files the badminton-highlight-indexer can
score and train against, export per-video `start,end` CSVs plus a `manifest.json`:
98      ```bash
99      # Curated + commercially-safe videos across all supported sports (default):
100     python -m src.cli.curate export --out-dir data/eval_export
101
102     # One sport, also including staging entries:
103     python -m src.cli.curate export --sport badminton --include-staging
104     ```
105     The exporter writes `<out-dir>/<sport>/<video_id>.csv` (decimal-second `start,end` rows)
and a `manifest.json` pairing each CSV with its video `local_path`/`video_url`, fps, resolution,
and license. The indexer keys videos by **file MD5**, not by our `video_id`, so the downstream
eval/training run must process the exact video file named in the manifest. Flags: `--sport`
(restrict to one sport), `--include-staging` (default: curated only), `--include-noncommercial`
(default: commercial-safe only).
106
107     ---
108
109     ## Configuration
110
111     Adjust settings in `config/config.yaml`:
112     - **`database_path`**: Path to the SQLite database.
113     - **`video_storage`**: Local folder path and automatic download size limit.
114     - **`commercial_licenses`**: Whitelist of licenses allowed for commercial use.
115     - **`supported_sports`**: Allowed sports categories.
116
117     ---
118
119     ## Documentation
120
121     - [docs/PIPELINE_ROLE.md](docs/PIPELINE_ROLE.md) ? how this repo feeds the **badminton-
highlight-indexer** (`export` / `fetch`).
122     - [docs/DECISIONS.md](docs/DECISIONS.md) ? collector-local decision records
(**ADR-001**: keep explicit per-sport code instead of a registry, with an "adding a new sport"
checklist).
123     - [docs/annotation/](docs/annotation/) ? the vendor annotation package used to produce
golden rally labels:
124       - [ROORA_BRIEF.md](docs/annotation/ROORA_BRIEF.md) ? vendor brief + the canonical
rally CSV schema.
125       - [ANNOTATION_GUIDE.md](docs/annotation/ANNOTATION_GUIDE.md) ? the illustrated rater
guide (singles & doubles).
126       - [INTAKE_MANIFEST_TEMPLATE.csv](docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv) ? per-
video intake sheet.
127       - [NDA_TEMPLATE.md](docs/annotation/NDA_TEMPLATE.md) ? mutual NDA template (have
counsel review).
128
129     ---
130
131     ## Running Tests
132
133     We use `pytest` for unit and integration testing:
134     ```bash
135     python -m pytest tests/
136     python -m pytest tests/ --cov=src --cov-report=term-missing   # with coverage (~87%)
137     ```
138
```

7. user (2026-07-02T04:46:04.813Z)

```
No files found
```

8. user (2026-07-02T04:46:05.684Z)

```
On branch master
Your branch is up to date with 'origin/master'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore

no changes added to commit (use "git add" and/or "git commit -a")
adcd206 fix(db): set SQLite busy_timeout so concurrent writers wait, not error (P9) (#53)
6f67d65 feat(import): callable import_annotations + a maturity gate (P9) (#52)
c281f82 feat(export): add source file MD5 to the eval/training manifest (P9) (#51)
a152cf5 fix(cvat): warn on rows silently skipped by load_canonical_csv (#50)
75bb0fa docs: archive CODE_HEALTH_PLAN ? project DONE (#49)
origin  https://github.com/Khelsutra/sports-data-collector.git (fetch)
origin  https://github.com/Khelsutra/sports-data-collector.git (push)
```

9. user (2026-07-02T04:46:17.670Z)

```
Already up to date.
adcd206 fix(db): set SQLite busy_timeout so concurrent writers wait, not error (P9) (#53)
```

10. user (2026-07-02T04:46:26.031Z)

```
Preparing worktree (new branch 'fix/curate-verify-registry-rot')
branch 'fix/curate-verify-registry-rot' set up to track 'origin/master'.
HEAD is now at adcd206 fix(db): set SQLite busy_timeout so concurrent writers wait, not error
(P9) (#53)
```

11. user (2026-07-02T04:46:33.835Z)

```
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-curate-verify/config:
config.yaml

C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-curate-verify/data:
mock_local_badminton_anno.json
roora_badminton_canonical.csv
roora_badminton_issues.txt
sports_metadata.db
staging_coachai_match.csv
validation

C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-curate-verify/src/cli:
curate.py
ingest.py
normalize_annotations.py

C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-curate-verify/src/core:
batch_preflight.py
config.py
db.py
downloader.py
manifests.py
parsing.py
proxy.py
schemas
```

```
validator.py

C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-curate-verify/tests:
test_batch_preflight.py
test_cleanup.py
test_cli.py
test_config.py
test_crawler_tempfile.py
test_cvat_parser.py
test_cvat_robustness.py
test_db.py
test_db_pragmas.py
test_downloader.py
test_import_guard.py
test_license_consistency.py
test_licensing_gate.py
test_local_import.py
test_manifests.py
test_normalize_annotations.py
test_parser.py
test_parsing.py
test_proxy.py
test_reporting.py
test_reporting_faq.py
test_reporting_noraise.py
test_schemas.py
test_validator.py
data/mock_local_badminton_anno.json
data/roora_badminton_canonical.csv
data/roora_badminton_issues.txt
data/sports_metadata.db
data/staging_coachai_match.csv
data/validation/pilot_badminton_01_report.json
data/validation/pilot_badminton_01_report.md
data/validation/pilot_badminton_01_review.csv
```

12. user (2026-07-02T04:46:44.847Z)

```
columns: ['video_id', 'sport', 'video_url', 'local_path', 'license_type', 'license_url',
'is_commercial', 'status', 'match_name', 'fps', 'resolution', 'annotation_license']
{'video_id': 'adarsh_avi_singles', 'sport': 'badminton', 'video_url': None, 'local_path':
'C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden Labelled\\Adarsh and Avi.MP4',
'license_type': 'Proprietary', 'license_url': None, 'is_commercial': 1, 'status': 'curated',
'match_name': 'Adarsh and Avi (singles)', 'fps': 59.94, 'resolution': '1080p',
'annotation_license': None}
{'video_id': 'gbaaddy', 'sport': 'badminton', 'video_url': None, 'local_path':
'C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden Labelled\\gBaaddy20May2026.MP4',
'license_type': 'Proprietary', 'license_url': None, 'is_commercial': 1, 'status': 'curated',
'match_name': 'gBaaddy 20 May 2026 (multi-court)', 'fps': 59.94, 'resolution': '1080p',
'annotation_license': None}
{'video_id': 'kushagra_singles', 'sport': 'badminton', 'video_url': None, 'local_path':
'C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden
Labelled\\KushagraYashAviFirstVideo.MP4', 'license_type': 'Proprietary', 'license_url': None,
'is_commercial': 1, 'status': 'curated', 'match_name': 'Kushagra Yash Avi (singles, multi-
court)', 'fps': 59.94, 'resolution': '4K', 'annotation_license': None}
{'video_id': 'largetest_doubles', 'sport': 'badminton', 'video_url': None, 'local_path':
'C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden Labelled\\LargeTestVideo.MP4',
'license_type': 'Proprietary', 'license_url': None, 'is_commercial': 1, 'status': 'curated',
'match_name': 'LargeTestVideo (doubles)', 'fps': 29.97, 'resolution': '5.3K',
'annotation_license': None}
{'video_id': 'mahadevpura_singles', 'sport': 'badminton', 'video_url': None, 'local_path':
'C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden
Labelled\\MahadevpuraSingles.MP4', 'license_type': 'Proprietary', 'license_url': None,
'is_commercial': 1, 'status': 'curated', 'match_name': 'Mahadevpura Singles (borderline multi-
```

court)', 'fps': 29.97, 'resolution': '1080p', 'annotation_license': None}
{'video_id': 'my_custom_local_match', 'sport': 'badminton', 'video_url': None, 'local_path':
'data/videos/my_custom_match.mp4', 'license_type': 'Proprietary', 'license_url': None,
'is_commercial': 1, 'status': 'curated', 'match_name': 'My Custom Local Match', 'fps': 30.0,
'resolution': '1080p', 'annotation_license': None}
{'video_id': 'shuttleset_1', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=O669aZhH0LI', 'local_path': './data/videos\\shuttleset_1.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Kento MOMOTA CHOU Tien Chen Fuzhou Open 2019 Finals', 'fps': 25.0, 'resolution':
'1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_10', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=li1sbr6S34g', 'local_path': './data/videos\\shuttleset_10.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'CHOU Tien Chen NG Ka Long Angus Sudirman Cup 2019 Group Stage', 'fps': 25.0,
'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_11', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=yD6WKVqsAKc', 'local_path': './data/videos\\shuttleset_11.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'CHOU Tien Chen Jonatan CHRISTIE Indonesia Open 2019 Quarter-finals', 'fps': 25.0,
'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_12', 'sport': 'badminton', 'video_url':
'https://youtube.com/watch?v=example', 'local_path': None, 'license_type': 'Unknown',
'license_url': None, 'is_commercial': 0, 'status': 'staging', 'match_name': 'NG Ka Long Angus
SHI Yu Qi Thailand Masters 2020 SemiFinals', 'fps': 30.0, 'resolution': '1080p',
'annotation_license': 'MIT'}
{'video_id': 'shuttleset_13', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=boQC4J4E1ZQ', 'local_path': './data/videos\\shuttleset_13.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Kento MOMOTA Viktor AXELSEN Malaysia Masters 2020 Finals', 'fps': 25.0,
'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_14', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=5W6txLGZ1Rs', 'local_path': './data/videos\\shuttleset_14.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Anders ANTONSEN Jonatan CHRISTIE Indonesia Masters 2020 QuarterFinals', 'fps':
25.0, 'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_15', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=yu9oyMXRGHY', 'local_path': './data/videos\\shuttleset_15.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Anthony Sinisuka GINTING Anders ANTONSEN Indonesia Masters 2020 Final', 'fps':
25.0, 'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_16', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=5kS_a7vS5xI', 'local_path': './data/videos\\shuttleset_16.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Anthony Sinisuka GINTING Viktor AXELSEN Indonesia Masters 2020 SemiFinals',
'fps': 25.0, 'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_17', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=G9r400zkkz8', 'local_path': './data/videos\\shuttleset_17.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'NG Ka Long Angus Jonatan CHRISTIE Malaysia Masters 2020 QuarterFinals', 'fps':
25.0, 'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_18', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=8lHAsyRhYYQ', 'local_path': './data/videos\\shuttleset_18.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Viktor AXELSEN SHI Yu Qi All England Open 2020 QuarterFinals', 'fps': 25.0,
'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_19', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=YmW83aQFADg', 'local_path': './data/videos\\shuttleset_19.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Viktor AXELSEN CHEN Long Malaysia Masters 2020 QuarterFinals', 'fps': 25.0,
'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_2', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=-aOI9_JxoWc', 'local_path': './data/videos\\shuttleset_2.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'CHEN Long CHOU Tien Chen World Tour Finals Group Stage', 'fps': 25.0,
'resolution': '1080p', 'annotation_license': 'MIT'}

{'video_id': 'shuttleset_20', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=4e3JJ4rvT3Q', 'local_path': './data/videos\\shuttleset_20.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Viktor AXELSEN NG Ka Long Angus Malaysia Masters 2020 SemiFinals', 'fps': 25.0,
'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_21', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=gloiZ_gTJaE', 'local_path': './data/videos\\shuttleset_21.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'curated',
'match_name': 'An Se Young Ratchanok Intanon YONEX Thailand Open 2021 QuarterFinals', 'fps':
30.0, 'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_22', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=8u_UHCnYSkk', 'local_path': './data/videos\\shuttleset_22.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Mia Blichfeldt Busanan Ongbamrungphan YONEX Thailand Open 2021 QuarterFinals',
'fps': 30.0, 'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_23', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=rSK9Qx8LapE', 'local_path': './data/videos\\shuttleset_23.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Ng Ka Long Angus Lee Cheuk Yiu YONEX Thailand Open 2021 QuarterFinals', 'fps':
30.0, 'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_24', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=NlDrJyQUTSI', 'local_path': './data/videos\\shuttleset_24.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Anthony Sinisuka Ginting Rasmus Gemke YONEX Thailand Open 2021 QuarterFinals',
'fps': 30.0, 'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_25', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=FZPrpoGdyHI', 'local_path': './data/videos\\shuttleset_25.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Carolina Marin Supanida Katethong YONEX Thailand Open 2021 QuarterFinals', 'fps':
30.0, 'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_3', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=eugfCRwSBJo', 'local_path': './data/videos\\shuttleset_3.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Kento MOMOTA CHOU Tien Chen KOREA OPEN 2019 Final', 'fps': 25.0, 'resolution':
'1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_4', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=y6QbtrTV-K0', 'local_path': './data/videos\\shuttleset_4.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'CHEN Long CHOU Tien Chen Denmark Open 2019 QuarterFinal', 'fps': 25.0,
'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_6', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=SYyvDrUgClc', 'local_path': './data/videos\\shuttleset_6.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'Kento MOMOTA CHOU Tien Chen Denmark Open 2018 Finals', 'fps': 25.0, 'resolution':
'1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_8', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=32j2Tg64Zbg', 'local_path': './data/videos\\shuttleset_8.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'CHOU Tien Chen Anders ANTONSEN Fuzhou Open 2019 Semi-finals', 'fps': 25.0,
'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'shuttleset_9', 'sport': 'badminton', 'video_url':
'https://www.youtube.com/watch?v=8E98Gpk-fOM', 'local_path': './data/videos\\shuttleset_9.mp4',
'license_type': 'Unknown', 'license_url': None, 'is_commercial': 0, 'status': 'staging',
'match_name': 'CHOU Tien Chen Jonatan CHRISTIE Sudirman Cup 2019 Quarter-finals', 'fps': 25.0,
'resolution': '1080p', 'annotation_license': 'MIT'}
{'video_id': 'staging_match_non_comm', 'sport': 'badminton', 'video_url':
'https://youtube.com/watch?v=example', 'local_path': None, 'license_type': 'CC-BY-NC-SA',
'license_url': None, 'is_commercial': 0, 'status': 'staging', 'match_name':
'staging_coachai_match', 'fps': 30.0, 'resolution': '1080p', 'annotation_license': None}
{'video_id': 'testlarge_short', 'sport': 'badminton', 'video_url': None, 'local_path':
'C:\\Users\\avidu\\Projects\\Annotation Setup\\Collect\\Golden Labelled\\TestLargeVideo.MP4',
'license_type': 'Proprietary', 'license_url': None, 'is_commercial': 1, 'status': 'curated',
'match_name': 'TestLargeVideo (short)', 'fps': 29.97, 'resolution': '5.3K',
'annotation_license': None}

13. assistant (2026-07-02T04:46:56.813Z)

All 6 dangling rows and the 2 placeholder rows confirmed. Now verify where the MP4 files actually exist on disk.

14. user (2026-07-02T04:46:58.529Z)

```
Exit code 2
Archive
Golden Labelled
Trajectories
golden_labels_badminton.csv
rally_review.csv
---
Adarsh and Avi.rallies.csv
KushagraYashAviFirstVideo.rallies.csv
LargeTestVideo.rallies.csv
MahadevpuraSingles.rallies.csv
TestLargeVideo.rallies.csv
gBaaddy20May2026.rallies.csv
---
ls: cannot access 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup/Collect/': No such
file or directory
```

15. user (2026-07-02T04:46:59.397Z)

```
ls: cannot access '/f/': No such file or directory
---
$Recycle.Bin
$SysReset
Documents and Settings
DumpStack.log.tmp
F3_Upgrade
```

16. user (2026-07-02T04:47:07.743Z)

```
Name Root                                       Used
Free
---- ----                                       ----
----
C    C:\                                534028824576 266702819328
G    G:\                                547363966976 253367676928
Temp C:\Users\avidu\AppData\Local\Temp\ 534028824576 266702819328
```

17. user (2026-07-02T04:47:07.882Z)

```
123:| original-6 labels | `F:\[Khelsutra]?\Training\Golden Labelled\*.rallies.csv` | ~12 KB | ?
| train + eval |
```

18. user (2026-07-02T04:47:14.094Z)

```
123:| original-6 labels | `F:\[Khelsutra]?\Training\Golden Labelled\*.rallies.csv` | ~12 KB | ?
| train + eval |
204:> `[Done] Video N/` (Drive) or `[Khelsutra] ?` (F:) fail with "matched no objects." The
owner-box
```

19. user (2026-07-02T04:47:14.821Z)

```
100       --video-uri gcs://khelsutra-rally-corpus/videos/<clip>.mp4 \
101       --out-uri   gcs://khelsutra-rally-corpus --segmenter wasb_hybrid --config config.json
102
103     # TRAIN from the bucket (reads labels/ [+ videos/], extracts trajectories, builds
manifest on-box,
104     # runs training.gen0.harness, pushes weights/<version>/):
```

```
105     RALLY_DETECTOR_IMPL=native python -m backend.worker train \
106       --corpus-uri gcs://khelsutra-rally-corpus --out-uri gcs://khelsutra-rally-corpus \
107       --version <v> --trajectory-source detector --segmenter wasb_hybrid --config
config.json
108     ```
109
110     A bare/relative path resolves under a configured `input_root`/`input_backend`
111     (`backend/storage/ref.py::resolve_input_uri`), so workflows can be written bucket-
relative.
112
113     ---
114
115     ## 5. The gap ? what's still local-only (why the box is still a blocker)
116
117     All of this is **tiny except the originals** ? the metadata that blocks cloud eval is <
25 MB total:
118
119     | Data | Where it is now | Size | In bucket? | Needed for |
120     |---|---|---|---|---|
121     | 20 WASB trajectory CSVs | `C:\?\Annotation Setup\Collect\Trajectories\` | **17.5 MB**
| ? none | eval/litmus (cached). *(No `cached` train source exists ? train synths or detector-
extracts.)* |
122     | golden manifest | `output/human_lovo_manifest.json` (C:-absolute paths) | 4.5 KB | ? |
eval corpus definition |
123     | original-6 labels | `F:\[Khelsutra]?\Training\Golden Labelled\*.rallies.csv` | ~12 KB
| ? | train + eval |
124     | 7 new-clip labels | repo `output/*.rallies.csv` (gitignored) **+** F: archive (backed
up 2026-06-22) | ~12 KB | ? partial/inconsistent | train + eval |
125     | source videos / proxies | F: archive + Google Drive golden folder | **100s of GB** | ?
3 only | infer + train (real WASB extraction) |
126
127     > The corpus *metadata* (trajectories + labels + manifest, ~25 MB) is what actually
blocks GPU-free
128     > cloud eval and the served Gate-A litmus ? and it's trivially small to migrate. The
**videos** are the
129     > only large migration, and they already exist on **Google Drive** (the golden-corpus
folder), so they
130     > can go **Drive ? bucket from a cloud box**, keeping the local machine out of the path
entirely.
131
132     ---
133
134     ## 6. Migration plan (close the gap)
135
136     **Phase 1 ? corpus metadata (?25 MB, do first, removes the eval blocker).** Stage all
trajectories,
137     all labels, and a **bucket-relative eval manifest** into `gs://khelsutra-rally-corpus`.
Pending the §7
138     naming decision. Sketch:
139
140     ```bash
141     # trajectories (decision-free; absent today)
142     gcloud storage cp "C:\?\Collect\Trajectories\*.csv" gs://khelsutra-rally-
corpus/trajectories/
143     # labels (after the §7.1 naming decision) -> labels/<canonical-name>.rallies.csv
144     # eval manifest with gs:// (or bucket-relative) paths -> a stable manifest object (§7.3)
145     ```
146
147     **Phase 2 ? videos ? `videos/<name>.mp4`** (bucket name = the eval `name`, consistent
with
148     `trajectories/` + `labels/`). Sources (resolved 2026-06-22 ? see
`scratch/copy_videos_to_gcs.ps1`):
149
150     | Source | Clips | Size |
151     |---|---|---|
```

```
152     | Drive golden folder `?/Golden Label Videos Request - June 15/[Done] Video N/`
(**proxies**) | `mahadevpura_1/2`, `GX010128`, `GX020094`, `GX030094`, `Badminton_BXH_2`,
`Boxhill_Doubles` (7) | ~13.5 GB |
153     | F: archive (**full MP4s** ? proxy-first before upload) | `GX010137`, `GX010141`, +
original-6 (`adarsh_avi_singles`, `kushagra_singles`, `largetest_doubles`, `gbaaddy`,
`testlarge_short`, `mahadevpura_singles`) (8) | ~50 GB |
154
155     **Two ways to run it:**
156     - **Decoupled (preferred ? no local box):** `rclone` on a cloud VM with a Google-Drive
remote ?
157       `rclone copy gdrive:"<?/[Done] Video N>/<proxy>.mp4" :gcs:khelsutra-rally-
corpus/videos/<name>.mp4`
158       (one-time owner step: `rclone config` ? `gdrive` remote OAuth). Or `gdown <file-id>`
on the box for
159       the shared folder. This is the path that actually removes the local box.
160     - **Interim (today, owner box ? streams Drive/F: ? local ? GCS):** `pwsh
scratch/copy_videos_to_gcs.ps1
161       -Execute` (dry-run without `-Execute`; `-Only <name>` for one; skip-existing). Uses
local bandwidth.
162
163     **Priority:** the 7 Drive **proxies** first (already 1080p, sufficient for detection).
The 8 large F:
164     full MP4s should be **proxied to 1080p first** (cheaper storage + faster cloud decode)
before upload.
165     ? Two pre-existing non-canonical video objects (`videos/GX030094_proxy.mp4`,
`videos/mahadevpura-1.MP4`)
166     are superseded once `videos/GX030094.mp4` / `videos/mahadevpura_1.mp4` land ? verify-
dupe-then-delete
167     (same as the label cleanup). Track per-clip in §8.
168
169     **Phase 3 ? make eval/litmus read GCS (real code, NOT a config flip).** The eval drivers
170     (`served_gate_a.evaluate_video`, `calibrate_wasb.load_golden_gts`,
`TrackNetRunner.parse_trajectory_csv`)
171     open `spec["trajectory"]`/`spec["labels"]` with plain `open()` and gate on
`os.path.isfile()` ? they
172     have **no storage-layer fetch**, so a `gs://` manifest cannot be consumed today. Phase-3
must thread
173     `storage.fetch`/`resolve_input_uri` through those paths first. Then: re-probe
`fps`/`frame_width` from
174     the bucket videos (don't trust the copied values ? a wrong fps silently mis-aligns every
label), and
175     **re-measure the served Gate-A litmus on the GCS 15-clip set as a [[launch-report-
convention]] event**
176     (dual-set RAW+golden no-regression, not a silent eval-set swap ? see PR #236).
177
178     ---
179
```

20. assistant (2026-07-02T04:47:31.096Z)

F: is not mounted (only C:, G:, Temp). Let me check the exact documented F: paths in the copy script, and do a thorough search for the six MP4s on the reachable drives (C: workspace and G: Drive).

21. user (2026-07-02T04:47:32.522Z)

```
20        # --- Drive golden proxies (preferred) ---
21        'mahadevpura_2'       = "$DRIVE\[Done] Video 3 - Badminton
Multicourt\mahadevpura-2_proxy.mp4"
22        'GX020094'            = "$DRIVE\[Done] Video 4 - Badminton\GX020094_proxy.mp4"
23        'GX030094'            = "$DRIVE\[Done] Video 6 - Badminton\GX030094_proxy.mp4"
24        'Badminton_BXH_2'     = "$DRIVE\[Done] Video 10 - Badminton Multicourt\Badminton BXH
2_proxy.mp4"
25        'Boxhill_Doubles'     = "$DRIVE\[Done] Video 12 - Badminton Multicourt\Boxhill
Doubles_proxy.mp4"
```

```
26        'GX010128'            = "$DRIVE\[Done] Video 13 - Badminton\GX010128.MP4"
27        'mahadevpura_1'       = "$DRIVE\[Done] Video 14 - Badminton\mahadevpura-1.MP4"
28        # --- F: full MP4s (large; consider proxy-first before upload) ---
29        'GX010137'            = "$FROOT\Badminton - Boxhill - 17 June\GX010137.MP4"          #
*6.4GB
30        'GX010141'            = "$FROOT\Badminton - Boxhill - 17 June\GX010141.MP4"          #
*5.5GB
31        'adarsh_avi_singles'  = "$FROOT\Training\Golden Labelled\Adarsh and Avi.MP4"
# *10.9GB
32        'kushagra_singles'    = "$FROOT\Training\Golden
Labelled\KushagraYashAviFirstVideo.MP4" # *7.7GB
33        'largetest_doubles'   = "$FROOT\Training\Golden Labelled\LargeTestVideo.MP4"
# *11GB
34        'gbaaddy'             = "$FROOT\Training\Golden Labelled\gBaaddy20May2026.MP4"
# *6.7GB
35        'testlarge_short'     = "$FROOT\Training\Golden Labelled\TestLargeVideo.MP4"
# *1.4GB
36        'mahadevpura_singles' = "$FROOT\Training\Golden Labelled\MahadevpuraSingles.MP4"
# 187MB
37      }
38
39      # which dests already exist in the bucket (skip those)
40      $have = @{}
41      gcloud storage ls "$BUCKET/" 2>$null | ForEach-Object {
$have[[IO.Path]::GetFileName($_)] = $true }
42
43      $copied = 0; $skipped = 0; $missing = 0
44      foreach ($name in $src.Keys) {
45        if ($Only -and $name -ne $Only) { continue }
46        $s = $src[$name]; $dst = "$BUCKET/$name.mp4"; $destName = "$name.mp4"
47        $exists = Test-Path -LiteralPath $s
48        $sizeMB = if ($exists) { [math]::Round((Get-Item -LiteralPath $s).Length/1MB) } else {
0 }
49        if (-not $exists) { "MISSING SRC  {0,-20} {1}" -f $name, $s; $missing++; continue }
```

22. user (2026-07-02T04:47:46.454Z)

```
G:\My Drive\Sharing\Badminton\Boxhill\June 1 2026\Adarsh and Avi.MP4 | 11,445,052,772 bytes
G:\My Drive\Sharing\Golden Annotations\MahadevpuraSingles.MP4 | 196,182,380 bytes
search done
```

23. user (2026-07-02T04:48:55.302Z)

```
<system-reminder>[Truncated: PARTIAL view ? showing lines 1-1167 of 1532 total (27874 tokens,
cap 25000). Call Read with offset=1168 limit=1167 for the next page, or Grep to find a specific
section. Do NOT answer from this page alone if the answer may be further in the file.]</system-
reminder>

1       import os
2       import sys
3       import time
4       import json
5       import csv
6       import argparse
7       from dataclasses import dataclass, field
8       from typing import List, Dict, Any, Optional, Tuple, Union
9       from tabulate import tabulate
10
11      from src.core.config import load_config
12      from src.core.parsing import safe_int as _safe_int, safe_float_required as _safe_float
13      from src.core.db import SportsDatabase
14      from src.core.manifests import EvalSetManifest
15      from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
16      from src.core.schemas.badminton import BadmintonRally
17      from src.core.schemas.tennis import TennisPoint
```

```
    from src.core.schemas.cricket import CricketDelivery
    from src.core.schemas.pickleball import PickleballRally
    from src.core.schemas.tabletennis import TableTennisRally
    from src.core.validator import DatasetValidator
    from src.core.downloader import download_best_video, probe_resolution_fps,
height_to_label

    #: A parsed annotation row of any sport ? the union of all segment schema types.
    #: Used so _parse_local_annotations / _guard_import return concrete types, not Any.
    Annotation = Union[BadmintonRally, TennisPoint, CricketDelivery,
                       PickleballRally, TableTennisRally]


    class CurationError(Exception):
        """A user-correctable pre-flight failure (bad input, duplicate ids, clobber guard).

        Raised by library-style methods (e.g. ``_guard_import``) instead of calling
        ``sys.exit(1)`` directly, so the logic is reusable from tests or a future API.
        The CLI ``main()`` translates it into ``sys.exit(1)``; a library caller can
        catch and handle it. See docs/CODE_HEALTH_PLAN.md §7 Tier D (A2).
        """


    @dataclass
    class ImportResult:
        """The structured outcome of ``import_annotations`` ? so a service/UI caller gets
data back
        instead of stdout + sys.exit. ``status`` is the CurationStatus the video was
recorded with (the
        maturity gate: ``staging`` = needs review, ``curated`` = confirmed)."""
        video_id: str
        count: int
        status: str
        match_name: str
        is_commercial: bool
        license_type: str
        warnings: List[str] = field(default_factory=list)
        video_meta: Optional[VideoMetadata] = None
        annotations: List[Annotation] = field(default_factory=list)


    def _truthy(value: Any) -> bool:
        """Coerce a CSV/JSON cell to a boolean, treating "0"/"false"/"no"/"" as False.

        B4: ``bool("0")`` is True (non-empty string), so the JSON path's bare ``bool(...)``
        disagreed with the CSV path's explicit truthiness list on the same logical value.
        Both paths now route through this single helper.
        """
        if isinstance(value, bool):
            return value
        if value is None:
            return False
        return str(value).strip().lower() in ("true", "1", "yes")


    class SportsDataCLI:
        def __init__(self, config_path: str = "config/config.yaml"):
            self.config_path = config_path
            self.config = self._load_config()

            # Resolve DB path
            db_path = self.config.get("database_path", "data/sports_metadata.db")
            self.db = SportsDatabase(db_path)
            self.validator = DatasetValidator(config_path)

```

```
80          def _load_config(self) -> Dict[str, Any]:
81              if not os.path.exists(self.config_path):
82                  print(f"Warning: Configuration file not found at {self.config_path}. Using
defaults.")
83              return load_config(self.config_path)
84
85          def status(self):
86              """Displays status summary of all videos in the database."""
87              # Query total count by status via the public DB read API.
88              rows = self.db.get_status_counts()  # list of (sport, status, cnt)
89
90              if not rows:
91                  print("\nDatabase is currently empty. Run crawlers or import local data.\n")
92                  return
93
94              data = [[sport, status, cnt] for sport, status, cnt in rows]
95              print("\n=== Dataset Ingestion Status ===")
96              print(tabulate(data, headers=["Sport", "Status", "Count"], tablefmt="grid"))
97              print()
98
99          @staticmethod
100         def _annotation_key(sport_str: str, a: Annotation) -> Union[int, Tuple[int, int],
None]:
101             """The per-sport primary-key value of one annotation (for duplicate
detection)."""
102             if sport_str in ("badminton", "pickleball", "tabletennis"):
103                 return getattr(a, "rally_number", None)
104             if sport_str == "tennis":
105                 return getattr(a, "point_number", None)
106             if sport_str == "cricket":
107                 return (getattr(a, "over", None), getattr(a, "ball", None))
108             return None
109
110         def _guard_import(self, sport_str: str, video_id: str, annotations:
List[Annotation],
111                           force: bool = False) -> None:
112             """Pre-insert safety checks (run BEFORE any DB write, so a rejection is
atomic)."""
113             # (1) Duplicate primary-key guard. int(float()) truncation (e.g. 7 and 7.5 both
114             # become 7) would otherwise raise IntegrityError mid-insert, leaving a committed
115             # video row with rolled-back annotations. Catch it before touching the DB.
116             keys = [self._annotation_key(sport_str, a) for a in annotations]
117             keys = [k for k in keys if k is not None]
118             seen, dupes = set(), set()
119             for k in keys:
120                 if k in seen:
121                     dupes.add(k)
122                 seen.add(k)
123             if dupes:
124                 raise CurationError(
125                     f"Duplicate rally/point identifier(s) {sorted(dupes)} in the annotations
"
126                     f"(often a fractional id like 7.5 truncating to 7). "
127                     f"Re-run with --normalize, or normalize first: "
128                     f"python -m src.cli.normalize_annotations --in <csv> --out clean.csv")
129
130             # (2) Golden-label protection ? refuse to silently clobber an existing video's
131             # annotations (filename-derived video_ids collide easily).
132             if self.db.get_video(video_id):
133                 try:
134                     n = len(self.db.get_segment_intervals(video_id, SportType(sport_str)))
135                 except Exception:
136                     n = 0
137                 if n > 0 and not force:
138                     raise CurationError(
```

```
                        f"video_id '{video_id}' already has {n} annotated segment(s); "
                        f"importing would REPLACE them. "
                        f"Pass --force to overwrite, or use a distinct --video-id to keep both.")
                if n > 0:
                    print(f"Warning: --force given ? replacing {n} existing segment(s) for video_id '{video_id}'.")

        def import_annotations(
            self,
            *,
            sport: str,
            video_path: Optional[str],
            annotations_path: str,
            video_id: Optional[str] = None,
            match_name: Optional[str] = None,
            license: str = "Proprietary",
            fps: Optional[float] = None,
            resolution: Optional[str] = None,
            normalize: bool = False,
            force: bool = False,
            status: CurationStatus = CurationStatus.CURATED,
        ) -> ImportResult:
            """Ingest one video's rally labels into the DB and RETURN a structured result ? the callable
            core behind the CLI ``import-local`` (and a future UI/service contribute path). Raises
            ``CurationError`` on any user-correctable failure (unsupported sport, missing/bad annotations,
            validation, duplicate/clobber guard); it prints nothing and never calls ``sys.exit``, so a
            service can catch and surface it.

            ``status`` is the MATURITY GATE: pass ``CurationStatus.STAGING`` for a contributed label that
            NEEDS REVIEW (the safe default for programmatic/UI imports ? never auto-promote to golden),
            ``CURATED`` once confirmed. The CLI passes CURATED (a manual owner import is reviewer-vetted).
            """
            warnings: List[str] = []
            sport_str = (sport or "").lower()
            if sport_str not in self.config.get("supported_sports", ["badminton"]):
                raise CurationError(
                    f"Sport '{sport_str}' is not supported. Supported: {self.config.get('supported_sports')}")

            if video_path and not os.path.exists(video_path):
                warnings.append(f"Local video path '{video_path}' does not exist on disk, but continuing registration.")
            if not annotations_path or not os.path.exists(annotations_path):
                raise CurationError(f"Annotations file '{annotations_path}' not found.")

            match_name = match_name or os.path.basename(video_path or annotations_path).split('.')[0]
            vid = video_id or match_name.lower().replace(" ", "_")

            try:
                license_type = LicenseType(license)
            except ValueError:
                warnings.append(f"Unknown license '{license}'. Defaulting to Unknown (non-commercial).")
                license_type = LicenseType.UNKNOWN
            is_commercial = self.validator.check_license_commercial(license_type)

```

```
            # Optional: normalize a raw CSV annotator export before parsing (drops unusable
rows,
            # renumbers fractional/duplicate rally ids) so the guards below can't fire.
            parse_path = annotations_path
            if normalize and parse_path.lower().endswith(".csv"):
                from src.cli.normalize_annotations import normalize_file
                normalized_path = parse_path + ".normalized.csv"
                rep = normalize_file(parse_path, normalized_path)
                warnings.append("normalize-annotations:\n" + rep.summary())
                parse_path = normalized_path

            try:
                annotations = self._parse_local_annotations(sport_str, vid, parse_path)
            except Exception as e:
                raise CurationError(f"Error parsing annotations: {e}") from e

            is_valid, errors = True, []
            if sport_str == "badminton":
                is_valid, errors = self.validator.validate_badminton_rallies(annotations)
            elif sport_str == "tennis":
                is_valid, errors = self.validator.validate_tennis_points(annotations)
            elif sport_str == "cricket":
                is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
            elif sport_str == "pickleball":
                is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
            elif sport_str == "tabletennis":
                is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)
            if not is_valid:
                raise CurationError("Annotations failed validation:\n" + "\n".join(f"  -
{e}" for e in errors))

            # Pre-insert safety (raises CurationError) ? atomic: no half-written DB on
rejection.
            self._guard_import(sport_str, vid, annotations, force=force)

            video_meta = VideoMetadata(
                video_id=vid, sport=SportType(sport_str), video_url=None,
local_path=video_path,
                license_type=license_type, license_url=None, is_commercial=is_commercial,
                status=status, match_name=match_name, fps=fps, resolution=resolution,
            )
            self.db.insert_video(video_meta)
            count = 0
            if sport_str == "badminton":
                count = self.db.insert_badminton_rallies(annotations)
            elif sport_str == "tennis":
                count = self.db.insert_tennis_points(annotations)
            elif sport_str == "cricket":
                count = self.db.insert_cricket_deliveries(annotations)
            elif sport_str == "pickleball":
                count = self.db.insert_pickleball_rallies(annotations)
            elif sport_str == "tabletennis":
                count = self.db.insert_tabletennis_rallies(annotations)

            return ImportResult(
                video_id=vid, count=count, status=status.value, match_name=match_name,
                is_commercial=is_commercial, license_type=license_type.value,
                warnings=warnings, video_meta=video_meta, annotations=annotations,
            )

        def import_local(self, args):
            """CLI: manually register a local video + annotations (a curated owner import).
Thin wrapper
            over ``import_annotations`` ? prints the warnings/summary, emits the
observability report, and
```

```
            translates a ``CurationError`` into ``sys.exit(1)``."""
            try:
                res = self.import_annotations(
                    sport=args.sport, video_path=args.video_path,
annotations_path=args.annotations_path,
                    video_id=args.video_id, match_name=args.match_name,
license=args.license,
                    fps=args.fps, resolution=args.resolution,
                    normalize=getattr(args, "normalize", False), force=getattr(args,
"force", False),
                    status=CurationStatus.CURATED,  # a manual owner import is already
reviewer-vetted
                )
            except CurationError as e:
                print(f"Error: {e}")
                sys.exit(1)

            for w in res.warnings:
                print(f"Warning: {w}")
            print(f"\nSuccessfully imported local video '{res.match_name}':")
            print(f"  - Video ID: {res.video_id}")
            print(f"  - Sport: {args.sport.lower()}")
            print(f"  - Commercial Use Allowed: {res.is_commercial} (License:
{res.license_type})")
            print(f"  - Registered {res.count} annotated segments.\n")

            # Per-video observability report (next to the video). Never raises.
            from src.reporting import emit_import_report
            src_fmt = os.path.splitext(args.annotations_path)[1].lstrip(".").lower() or
"unknown"
            report_paths = emit_import_report(
                video_meta_obj=res.video_meta, annotations=res.annotations, is_valid=True,
                errors=[], count=res.count, sport=args.sport.lower(), source_format=src_fmt,
                fallback_dir=os.path.dirname(os.path.abspath(args.annotations_path)),
            )
            if report_paths:
                print(f"  - Observability report: {report_paths.get('technical_md')}")
                print(f"  - Customer summary    : {report_paths.get('customer_md')}\n")

        def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Annotation]:
            """Parses simple JSON or CSV annotations files."""
            parsed_items = []
            ext = os.path.splitext(filepath)[1].lower()

            if ext == '.json':
                with open(filepath, 'r', encoding='utf-8') as f:
                    data = json.load(f)

                if not isinstance(data, list):
                    raise ValueError("JSON file must contain a list of objects.")

                # Uses the same safe casts as the CSV path: a missing/blank required
                # time raises a clear ValueError (caught by the caller) instead of a
                # bare KeyError, and optional ints degrade to sensible defaults.
                for idx, obj in enumerate(data):
                    if sport == "badminton":
                        parsed_items.append(BadmintonRally(
                            video_id=video_id,
                            rally_number=_safe_int(obj.get("rally_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            score_before=obj.get("score_before"),
                            score_after=obj.get("score_after"),
                            shots_count=_safe_int(obj.get("shots_count"), None),
```

```
                            ending_reason=obj.get("ending_reason"),
                            last_shot_outcome=obj.get("last_shot_outcome")
                        ))
                    elif sport == "tennis":
                        parsed_items.append(TennisPoint(
                            video_id=video_id,
                            point_number=_safe_int(obj.get("point_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            set_score=obj.get("set_score"),
                            game_score=obj.get("game_score"),
                            server=obj.get("server"),
                            ending_type=obj.get("ending_type")
                        ))
                    elif sport == "cricket":
                        parsed_items.append(CricketDelivery(
                            video_id=video_id,
                            over=_safe_int(obj.get("over"), 0),
                            ball=_safe_int(obj.get("ball"), 0),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            batsman=obj.get("batsman"),
                            bowler=obj.get("bowler"),
                            runs=_safe_int(obj.get("runs"), None),
                            # B4: route through the same truthiness helper as the CSV path
                            # below ? bool("0") is True (non-empty string), so the JSON and
                            # CSV paths previously disagreed on the same logical value.
                            wicket=_truthy(obj.get("wicket")),
                            extras=_safe_int(obj.get("extras"), 0)
                        ))
                    elif sport == "pickleball":
                        parsed_items.append(PickleballRally(
                            video_id=video_id,
                            rally_number=_safe_int(obj.get("rally_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            score_before=obj.get("score_before"),
                            score_after=obj.get("score_after"),
                            shots_count=_safe_int(obj.get("shots_count"), None),
                            ending_reason=obj.get("ending_reason"),
                            last_shot_outcome=obj.get("last_shot_outcome")
                        ))
                    elif sport == "tabletennis":
                        parsed_items.append(TableTennisRally(
                            video_id=video_id,
                            rally_number=_safe_int(obj.get("rally_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            score_before=obj.get("score_before"),
                            score_after=obj.get("score_after"),
                            shots_count=_safe_int(obj.get("shots_count"), None),
                            ending_reason=obj.get("ending_reason"),
                            last_shot_outcome=obj.get("last_shot_outcome")
                        ))
            elif ext == '.csv':
                with open(filepath, 'r', encoding='utf-8') as f:
                    reader = csv.DictReader(f)
                    # Normalize headers
                    reader.fieldnames = [n.lower().strip() for n in reader.fieldnames] if
reader.fieldnames else []

                    for idx, row in enumerate(reader):
                        if sport == "badminton":
                            parsed_items.append(BadmintonRally(
                                video_id=video_id,
```

```
                                rally_number=_safe_int(row.get("rally_number"), idx + 1),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                score_before=row.get("score_before"),
                                score_after=row.get("score_after"),
                                shots_count=_safe_int(row.get("shots_count"), None),
                                ending_reason=row.get("ending_reason"),
                                last_shot_outcome=row.get("last_shot_outcome")
                            ))
                        elif sport == "tennis":
                            parsed_items.append(TennisPoint(
                                video_id=video_id,
                                point_number=_safe_int(row.get("point_number"), idx + 1),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                set_score=row.get("set_score"),
                                game_score=row.get("game_score"),
                                server=row.get("server"),
                                ending_type=row.get("ending_type")
                            ))
                        elif sport == "cricket":
                            parsed_items.append(CricketDelivery(
                                video_id=video_id,
                                over=_safe_int(row.get("over"), 0),
                                ball=_safe_int(row.get("ball"), 0),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                batsman=row.get("batsman"),
                                bowler=row.get("bowler"),
                                runs=_safe_int(row.get("runs"), None),
                                wicket=_truthy(row.get("wicket")),
                                extras=_safe_int(row.get("extras"), 0)
                            ))
                        elif sport == "pickleball":
                            parsed_items.append(PickleballRally(
                                video_id=video_id,
                                rally_number=_safe_int(row.get("rally_number"), idx + 1),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                score_before=row.get("score_before"),
                                score_after=row.get("score_after"),
                                shots_count=_safe_int(row.get("shots_count"), None),
                                ending_reason=row.get("ending_reason"),
                                last_shot_outcome=row.get("last_shot_outcome")
                            ))
                        elif sport == "tabletennis":
                            parsed_items.append(TableTennisRally(
                                video_id=video_id,
                                rally_number=_safe_int(row.get("rally_number"), idx + 1),
                                start_time=_safe_float(row.get("start_time")),
                                end_time=_safe_float(row.get("end_time")),
                                score_before=row.get("score_before"),
                                score_after=row.get("score_after"),
                                shots_count=_safe_int(row.get("shots_count"), None),
                                ending_reason=row.get("ending_reason"),
                                last_shot_outcome=row.get("last_shot_outcome")
                            ))
            else:
                raise ValueError("Unsupported annotations file extension. Use .json or
.csv.")

            return parsed_items

        def _fetch_one_with_retry(self, url, dest_path, args, max_height, max_size_mb):
            """Download one video at best resolution, retrying transient failures.
```

```
436
437             A below-min-height result almost always means YouTube rate-limited the
438             request and served a low fallback (not that the video lacks HD), so we
439             retry with a growing backoff. Returns (DownloadResult, None) on success,
440             or (None, failure_reason) once retries are exhausted. A below-min file is
441             discarded each attempt so it never lands in the DB.
442             """
443             attempts = max(1, args.retries + 1)
444             reason = "unknown error"
445             for i in range(attempts):
446                 result = download_best_video(
447                     url, dest_path,
448                     max_height=max_height, max_size_mb=max_size_mb,
449                     cookies_from_browser=args.cookies_from_browser,
450                     cookies_file=args.cookies,
451                     player_client=args.player_client,
452                 )
453                 if result.success and (result.height is None or result.height >=
args.min_height):
454                     return result, None
455
456                 if not result.success:
457                     reason = result.error or "unknown error"
458                 else:
459                     reason = (f"got {result.resolution_label} (< min-height
{args.min_height}p) ? "
460                               f"YouTube served a low format (rate-limited?)")
461                     try:
462                         os.remove(result.path)
463                     except OSError:
464                         pass
465
466                 if i < attempts - 1:
467                     backoff = args.retry_backoff * (i + 1)
468                     print(f"  attempt {i+1}/{attempts} failed: {reason}\n  retrying in
{backoff:.0f}s...")
469                     time.sleep(backoff)
470             return None, reason
471
472         def fetch(self, args):
473             """(Re)download videos at best available resolution, updating the DB in place.
474
475             This is the in-place upgrade path: rows, rally annotations, and curation
476             status are all preserved ? only the video file plus its `local_path`,
477             `resolution`, and `fps` are refreshed. By default it skips videos that
478             already have a local file at or above `--min-height`, so re-runs are cheap
479             and resumable; `--force` re-fetches everything.
480             """
481             storage = self.config.get("video_storage", {})
482             local_dir = storage.get("local_dir", "./data/videos")
483             max_download_size_mb = storage.get("max_download_size_mb")
484             max_height = args.max_height if args.max_height is not None else
storage.get("max_height")
485             os.makedirs(local_dir, exist_ok=True)
486
487             commercial_only = not args.include_noncommercial
488             if args.status == "all":
489                 statuses = [CurationStatus.CURATED, CurationStatus.STAGING]
490             else:
491                 statuses = [CurationStatus(args.status)]
492
493             supported = self.config.get("supported_sports", ["badminton"])
494             sport_strs = [args.sport] if args.sport else supported
495
496             # Collect target videos.
```

```
            videos = []
            for sport_str in sport_strs:
                try:
                    sport = SportType(sport_str.lower())
                except ValueError:
                    print(f"Warning: skipping unknown sport '{sport_str}'.")
                    continue
                for status in statuses:
                    videos.extend(self.db.get_videos_by_status(status, sport))
            if args.video_id:
                videos = [v for v in videos if v.video_id == args.video_id]

            if not videos:
                print("\nNo matching videos to fetch.\n")
                return

            results = []  # (video_id, action, detail)
            for video in videos:
                if commercial_only and not video.is_commercial:
                    results.append((video.video_id, "skip", "non-commercial"))
                    continue
                if not video.video_url or "watch?v=example" in video.video_url:
                    # Either no URL, or the parser's placeholder (the ShuttleSet catalog
                    # had no URL for this match) ? nothing to download.
                    results.append((video.video_id, "skip", "no real source URL"))
                    continue

                dest_path = os.path.join(local_dir, f"{video.video_id}.mp4")

                # Skip if we already have a good-enough local file (unless --force).
                if not args.force and os.path.exists(dest_path):
                    _, height, _ = probe_resolution_fps(dest_path)
                    if height is not None and height >= args.min_height:
                        results.append((video.video_id, "have", f"{height_to_label(height)}
already"))
                        continue

                print(f"Fetching {video.video_id} ({video.match_name})...")
                result, fail_reason = self._fetch_one_with_retry(
                    video.video_url, dest_path, args, max_height, max_download_size_mb)
                if fail_reason is not None:
                    results.append((video.video_id, "FAIL", fail_reason))
                    if args.sleep:
                        time.sleep(args.sleep)
                    continue

                # Update only the file-derived fields; preserve everything else (UPSERT
                # mutates the row in place, leaving rallies intact).
                video.local_path = result.path
                if result.resolution_label:
                    video.resolution = result.resolution_label
                if result.fps:
                    video.fps = result.fps
                self.db.insert_video(video)

                detail = (f"{result.resolution_label or '?'}"
                          f"{f' {result.fps:g}fps' if result.fps else ''},
{result.size_mb:.0f} MB")
                results.append((video.video_id, "fetched", detail))
                if args.sleep:
                    time.sleep(args.sleep)

            print("\n=== Fetch summary ===")
            print(tabulate([[vid, action, detail] for vid, action, detail in results],
                           headers=["Video ID", "Action", "Detail"], tablefmt="grid"))
```

```
560             fetched = sum(1 for _, a, _ in results if a == "fetched")
561             failed = [r for r in results if r[1] == "FAIL"]
562             print(f"\nFetched/updated : {fetched}   Already-good : {sum(1 for _,a,_ in
results if a=='have')}   "
563                   f"Skipped : {sum(1 for _,a,_ in results if a=='skip')}   Failed :
{len(failed)}")
564             if failed:
565                 print("Failures (left unchanged in DB):")
566                 for vid, _, detail in failed:
567                     print(f"  - {vid}: {detail}")
568             print()
569
570         # Code/data licenses that can NEVER apply to video footage ? if one of these is
571         # stamped on a remote/broadcast video it is an annotation-vs-video license
conflation.
572         _CODE_DATA_LICENSES = {LicenseType.MIT, LicenseType.APACHE_2, LicenseType.BSD}
573
574         def reclassify_licenses(self, args):
575             """Correct annotation-vs-video license conflation in existing rows.
576
577             A third-party video (one with a remote `video_url` ? i.e. broadcast footage like
578             ShuttleSet, whether or not it has since been downloaded to `local_path`) whose
579             `license_type` is a code/data license (MIT/Apache-2.0/BSD) is a conflation: that
580             license covers the ANNOTATIONS, not the footage. This moves the license to
581             `annotation_license`, sets the video `license_type` to Unknown, and recomputes
582             `is_commercial=False` so such videos stop being exported as commercially clean.
583
584             Dry-run by default; pass --apply to persist.
585             """
586             changed = []
587             seen = set()
588             for status in CurationStatus:
589                 for video in self.db.get_videos_by_status(status):
590                     if video.video_id in seen:
591                         continue
592                     seen.add(video.video_id)
593                     # The broadcast signal is a third-party video_url; a downloaded copy
594                     # (local_path set) does not change that the FOOTAGE rights are not ours.
595                     is_third_party = bool(video.video_url)
596                     if is_third_party and video.license_type in self._CODE_DATA_LICENSES:
597                         changed.append(video)
598
599             if not changed:
600                 print("No conflated rows found ? every remote video already has a video-
footage license.")
601                 return
602
603             print(f"{'APPLYING' if args.apply else 'DRY-RUN'} ? {len(changed)} conflated
row(s):")
604             for v in changed:
605                 print(f"  {v.video_id:40s} {v.license_type.value} (video) ->
annotation_license; "
606                       f"video license -> Unknown; is_commercial {v.is_commercial} -> False")
607                 if args.apply:
608                     v.annotation_license = v.annotation_license or v.license_type
609                     v.license_type = LicenseType.UNKNOWN
610                     v.is_commercial =
self.validator.check_license_commercial(v.license_type)
611                     self.db.insert_video(v)
612
613             if args.apply:
614                 print("\nApplied. Re-run `curate export` to regenerate the manifest without
these videos.")
615             else:
616                 print("\nDry-run only. Re-run with --apply to persist, then `curate
```

```
export`.")
617
618         def export_eval_set(self, args):
619             """Export curated annotations as indexer-ready eval/training sets.
620
621             Emits, per qualifying video, a `<out_dir>/<sport>/<video_id>.csv` in the
622             indexer's `start,end` ground-truth format (decimal seconds), plus a single
623             `<out_dir>/manifest.json` that pairs each CSV with its video file. The
624             manifest is the contract the downstream eval/training driver reads: the
625             indexer keys videos by *file MD5*, not by our `video_id`, so it must run
626             against the exact `local_path`/`video_url` recorded here.
627
628             By default only CURATED + commercially-safe videos are exported, mirroring
629             the license gate already enforced elsewhere in the toolchain.
630             """
631             from src.core.proxy import md5_file  # local import (mirrors
prep_annotation_proxy)
632
633             out_dir = args.out_dir
634             commercial_only = not args.include_noncommercial
635             statuses = [CurationStatus.CURATED]
636             if args.include_staging:
637                 statuses.append(CurationStatus.STAGING)
638
639             supported = self.config.get("supported_sports", ["badminton"])
640             sport_strs = [args.sport] if args.sport else supported
641
642             manifest = []
643             skipped_noncomm = 0
644             skipped_empty = 0
645
646             for sport_str in sport_strs:
647                 try:
648                     sport = SportType(sport_str.lower())
649                 except ValueError:
650                     print(f"Warning: skipping unknown sport '{sport_str}'.")
651                     continue
652
653                 videos = []
654                 for status in statuses:
655                     videos.extend(self.db.get_videos_by_status(status, sport))
656
657                 for video in videos:
658                     if commercial_only and not video.is_commercial:
659                         skipped_noncomm += 1
660                         continue
661
662                     intervals = self.db.get_segment_intervals(video.video_id, sport)
663                     if not intervals:
664                         skipped_empty += 1
665                         continue
666
667                     sport_dir = os.path.join(out_dir, sport.value)
668                     os.makedirs(sport_dir, exist_ok=True)
669                     csv_path = os.path.join(sport_dir, f"{video.video_id}.csv")
670                     with open(csv_path, "w", newline="", encoding="utf-8") as f:
671                         writer = csv.writer(f)
672                         writer.writerow(["start", "end"])
673                         for start, end in intervals:
674                             writer.writerow([start, end])
675
676                     # Full-file MD5 = the indexer's join key (it keys videos by file MD5,
not our video_id).
677                     # Computed when the local file is present; None (with a warning) for a
URL-only/offline
```

```
                    # entry, so the downstream run knows the deterministic join isn't
available for it.
                    md5 = None
                    if video.local_path and os.path.exists(video.local_path):
                        md5 = md5_file(video.local_path)
                    elif video.local_path:
                        print(f"Warning: '{video.video_id}' local file missing
({video.local_path}); "
                              f"exporting without an md5 (the indexer join needs the exact
file).")

                    manifest.append({
                        "video_id": video.video_id,
                        "sport": sport.value,
                        "csv": os.path.relpath(csv_path, out_dir).replace(os.sep, "/"),
                        "md5": md5,
                        "local_path": video.local_path,
                        "video_url": video.video_url,
                        "match_name": video.match_name,
                        "fps": video.fps,
                        "resolution": video.resolution,
                        "license_type": video.license_type.value,
                        "is_commercial": video.is_commercial,
                        "status": video.status.value,
                        "num_segments": len(intervals),
                    })

            os.makedirs(out_dir, exist_ok=True)
            manifest_doc = {
                "commercial_only": commercial_only,
                "statuses": [s.value for s in statuses],
                "total_videos": len(manifest),
                "total_segments": sum(m["num_segments"] for m in manifest),
                "eval_sets": manifest,
            }
            # Validate the manifest against the typed contract before writing, so a
            # field rename / type drift / miscount fails loudly at export time instead
            # of silently breaking the downstream indexer's training/eval run.
            EvalSetManifest.model_validate(manifest_doc).validate_counts()
            manifest_path = os.path.join(out_dir, "manifest.json")
            with open(manifest_path, "w", encoding="utf-8") as f:
                json.dump(manifest_doc, f, indent=2)

            print(f"\n=== Exported eval/training set to '{out_dir}' ===")
            if manifest:
                summary = [[m["sport"], m["video_id"], m["num_segments"],
                            "YES" if m["is_commercial"] else "NO",
                            os.path.basename(m["local_path"]) if m["local_path"] else "(no
local file)"]
                           for m in manifest]
                print(tabulate(summary, headers=["Sport", "Video ID", "Segments", "Comm?",
"Video file"], tablefmt="grid"))
            print(f"\nVideos exported : {len(manifest)}")
            print(f"Total segments  : {manifest_doc['total_segments']}")
            if skipped_noncomm:
                print(f"Skipped (non-commercial) : {skipped_noncomm}  (pass --include-
noncommercial to include)")
            if skipped_empty:
                print(f"Skipped (no annotations) : {skipped_empty}")
            print(f"Manifest        : {manifest_path}\n")

        def prep_annotation_proxy(self, args):
            """Re-encode a video into a frame-identical annotation proxy (ADR-002).

            The proxy is what the annotation vendor receives: downscaled and
```

```
            scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,
            but with the source's EXACT frame count / fps / timeline, so frame N
            on the proxy is frame N on the original and returned labels apply to
            the original unchanged. Parity is verified after the encode; a proxy
            that fails verification is deleted, never shared. Provenance (source +
            proxy MD5s) is appended to the proxy manifest ? downstream processing
            (import-local, export, the indexer) keeps running on the ORIGINAL file.
            """
            from src.core import proxy as proxylib

            src = args.video_path
            if not os.path.exists(src):
                print(f"Error: video '{src}' not found.")
                sys.exit(1)

            stem = os.path.basename(src).split('.')[0]
            # Mirror import-local's video_id derivation so the manifest row and a
            # later DB registration of the same source agree on the key.
            video_id = args.video_id or stem.lower().replace(" ", "_")
            out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
            # Same-file guard must survive case-insensitive filesystems (Windows):
            # ffmpeg -y on a case-variant of the source would truncate the original.
            same_file = (os.path.normcase(os.path.realpath(out))
                         == os.path.normcase(os.path.realpath(src)))
            if not same_file and os.path.exists(out):
                try:
                    same_file = os.path.samefile(out, src)
                except OSError:
                    pass
            if same_file:
                print("Error: --out must differ from --video-path (it would overwrite "
                      "the original recording).")
                sys.exit(1)
            if os.path.exists(out) and not args.force:
                print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
                sys.exit(1)

            encoder = getattr(args, "encoder", proxylib.DEFAULT_ENCODER)
            if encoder not in proxylib.ENCODERS:
                print(f"Error: unknown --encoder '{encoder}' (expected one of
{proxylib.ENCODERS}).")
                sys.exit(1)
            if not proxylib.encoder_available(encoder):
                print(f"Error: encoder '{encoder}' is not available in this ffmpeg build "
                      f"(no h264_nvenc / no GPU?). Use --encoder libx264 (CPU).")
                sys.exit(1)

            try:
                src_info = proxylib.probe_video(src)
            except proxylib.ProxyError as e:
                print(f"Error: {e}")
                sys.exit(1)
            if not src_info.avg_fps:  # None or 0 (e.g. a cover-art/attached-pic stream)
                print(f"Error: could not determine fps of '{src}' ? the frame<->time "
                      f"contract needs it.")
                sys.exit(1)
            if proxylib.is_vfr(src_info) and not args.allow_vfr:
                print(f"Error: '{src}' looks variable-frame-rate "
                      f"(r={src_info.r_fps} vs avg={src_info.avg_fps}).")
                print("  The vendor pipeline recomputes times as frame/fps, which is only "
                      "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
                      "proceed anyway (timestamps stay identical to the original, but "
                      "frame/fps seconds are approximate for BOTH files).")
                sys.exit(1)

```

```
            # Hash the source up front: the provenance row needs it anyway, and it
            # lets the manifest collision guard run BEFORE the expensive encode.
            print("Hashing source (provenance)...")
            source_md5 = proxylib.md5_file(src)
            existing = proxylib.read_manifest_rows(args.manifest)
            clashes = [r for r in existing
                       if r.get("video_id") == video_id and r.get("source_md5") !=
source_md5]
            if clashes and not args.force:
                print(f"Error: manifest '{args.manifest}' already maps video_id "
                      f"'{video_id}' to a DIFFERENT source (md5
{clashes[-1].get('source_md5')}).")
                print("  Filename-derived video_ids collide easily ? pass a distinct "
                      "--video-id to keep both, or --force to append anyway (last row
wins).")
                sys.exit(1)
            if any(r.get("video_id") == video_id and r.get("source_md5") == source_md5
                   for r in existing):
                print(f"Note: manifest already has row(s) for '{video_id}' with this "
                      f"source; the new row supersedes them (last row wins).")

            os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
            cmd = proxylib.build_proxy_command(src, out, height=args.height,
                                               crf=args.crf, gop=args.gop,
                                               preset=args.preset, encoder=encoder)
            print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
                  f"fps {src_info.avg_fps:g} preserved, encoder={encoder})...")
            # From here on a failure must never leave a proxy file behind: a partial
            # or unverified file under the to-be-shared name is exactly the artifact
            # this flow exists to prevent (and -y may already have truncated a
            # previous good proxy on --force).
            try:
                proxylib.run_ffmpeg(cmd)
                proxy_info = proxylib.probe_video(out)
            except proxylib.ProxyError as e:
                print(f"Error: {e}")
                self._delete_failed_proxy(out)
                sys.exit(1)
            except BaseException:
                # KeyboardInterrupt mid-encode included ? clean up, then re-raise.
                self._delete_failed_proxy(out)
                raise

            errors = proxylib.verify_parity(src_info, proxy_info)
            if errors:
                print("Error: proxy FAILED frame<->time parity verification:")
                for err in errors:
                    print(f"  - {err}")
                self._delete_failed_proxy(out)
                sys.exit(1)

            print("Hashing proxy (provenance)...")
            proxy_md5 = proxylib.md5_file(out)

            from datetime import datetime, timezone
            row = {
                "video_id": video_id,
                "source_path": os.path.abspath(src),
                "source_md5": source_md5,
                "proxy_path": os.path.abspath(out),
                "proxy_md5": proxy_md5,
                "fps": f"{src_info.avg_fps:g}",
                "frame_count": src_info.nb_frames,
                "duration_s": f"{src_info.duration:.3f}" if src_info.duration is not None
else "",
```

```
                "source_resolution": src_info.resolution,
                "proxy_resolution": proxy_info.resolution,
                "encoder_settings": (
                    (f"h264_nvenc preset={proxylib.NVENC_PRESET} rc=vbr cq={args.crf}"
                     if encoder == "nvenc"
                     else f"libx264 crf={args.crf} preset={args.preset}")
                    + f" g={args.gop} height<={args.height} fps_mode=passthrough
audio=aac160k"),
                "created_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
            }
            try:
                proxylib.append_manifest_row(args.manifest, row)
            except OSError as e:
                # E.g. the CSV is open in Excel. The proxy itself is verified and
                # fine ? surface the row so the provenance isn't lost, keep the file.
                print(f"Error: could not append to manifest '{args.manifest}': {e}")
                print("  The proxy IS verified and kept. Close the manifest file and "
                      "append this row manually (or re-run with --force):")
                for k in proxylib.MANIFEST_FIELDS:
                    print(f"    {k}: {row.get(k, '')}")
                sys.exit(1)

            src_mb = os.path.getsize(src) / (1024 * 1024)
            out_mb = os.path.getsize(out) / (1024 * 1024)
            print("\n=== Annotation proxy ready ===")
            print(f"  Video ID    : {video_id}")
            print(f"  Source      : {src} ({src_info.resolution}, {src_mb:.0f} MB, md5
{source_md5})")
            print(f"  Proxy       : {out} ({proxy_info.resolution}, {out_mb:.0f} MB, md5
{proxy_md5})")
            print(f"  Parity      : VERIFIED ? {src_info.nb_frames} frames @
{src_info.avg_fps:g} fps on both")
            print(f"  Manifest    : {args.manifest}")
            print("\nNext:")
            print("  1. Share the PROXY with the vendor (CVAT at frame step 1; first/last "
                  "box of every rally on the exact contact/dead frame).")
            print("  2. On delivery, run convert-cvat with --video-path pointing at the "
                  "ORIGINAL (or pass --fps from the manifest row).")
            print("  3. import-local / export keep pointing at the ORIGINAL file ? the "
                  "indexer keys videos by its MD5.\n")

        def prep_annotation_batch(self, args):
            """Mirror a tree of source videos into annotation proxies (ADR-002).

            Input layout: ``<input-dir>/<match-folder>/<well-named video>`` (videos
            sitting directly in ``<input-dir>`` are accepted too). The output
            mirrors the SAME folder layout under ``--out-dir`` but holds ONLY the
            proxies ? i.e. exactly the tree that gets shared with the vendor.

            A **preflight gate** runs first (see ``src/core/batch_preflight.py``):
            it probes/fingerprints every video and BLOCKS the batch on duplicates
            (the same video in two folders) unless ``--allow-duplicates``, while
            surfacing empty subdirs, VFR sources, and multi-video folders as
            warnings. ``--dry-run`` prints that report and stops (no encode).

            Every video then runs through the full single-video flow (probe, VFR
            gate, parity verification, provenance manifest, all guards); one
            video's failure is reported and does not stop the rest. Already-proxied
            videos are skipped without --force, so re-runs are cheap and resumable.
            """
            from src.core import batch_preflight as pf

            root = args.input_dir
            if not os.path.isdir(root):
                print(f"Error: --input-dir '{root}' is not a directory.")
```

```
                sys.exit(1)

            # --- Preflight gate (probe + fingerprint the whole tree, once) --------
            report = pf.run_preflight(root)
            print(pf.render_preflight(report))

            if report.n_videos == 0:
                print(f"\nError: no video files found under '{root}' "
                      f"(recognized: {', '.join(sorted(pf.VIDEO_EXTS))}).")
                sys.exit(1)

            if report.has_blockers and not getattr(args, "allow_duplicates", False):
                print("\nGATE: BLOCKED ? duplicate source videos detected (see above). "
                      "Remove the copy, or pass --allow-duplicates if intentional.")
                sys.exit(1)
            if report.has_blockers:
                print("\nNote: --allow-duplicates given ? proceeding despite duplicate
sources.")

            if getattr(args, "dry_run", False):
                print("\n--dry-run: preflight only, no proxies written.")
                return

            # Encode targets come straight from the preflight (already probed).
            targets = [(e.rel_dir, e.path) for e in report.entries]
            results = []
            for idx, (rel, src) in enumerate(targets, start=1):
                stem = os.path.basename(src).split('.')[0]
                out = os.path.join(args.out_dir, rel, f"{stem}_proxy.mp4")
                label = os.path.join(rel, os.path.basename(src)) if rel else
os.path.basename(src)
                if os.path.exists(out) and not args.force:
                    results.append((rel or ".", os.path.basename(src), "have",
                                    "proxy exists (--force to redo)"))
                    continue
                one = argparse.Namespace(
                    video_path=src, out=out, video_id=None,
                    height=args.height, crf=args.crf, gop=args.gop, preset=args.preset,
                    encoder=getattr(args, "encoder", "libx264"),
                    manifest=args.manifest, allow_vfr=args.allow_vfr, force=args.force,
                )
                print(f"\n--- [{idx}/{len(targets)}] {label} ---")
                try:
                    self.prep_annotation_proxy(one)
                    results.append((rel or ".", os.path.basename(src), "ok",
                                    os.path.relpath(out, args.out_dir)))
                except SystemExit:
                    # The single-video flow already printed the error and cleaned up.
                    results.append((rel or ".", os.path.basename(src), "FAIL",
                                    "see error above"))

            print("\n=== Batch proxy summary ===")
            print(tabulate([[r, v, s, d] for r, v, s, d in results],
                           headers=["Folder", "Video", "Status", "Detail"],
tablefmt="grid"))
            ok = sum(1 for _, _, s, _ in results if s == "ok")
            have = sum(1 for _, _, s, _ in results if s == "have")
            failed = [r for r in results if r[2] == "FAIL"]
            print(f"\nProxied : {ok}   Already-have : {have}   Failed : {len(failed)}")
            print(f"Manifest: {args.manifest}")
            print(f"Share the '{args.out_dir}' tree with the vendor ? it contains ONLY "
                  f"proxies; every other stage runs on the originals.\n")
            if failed:
                sys.exit(1)

```

```
        @staticmethod
        def _delete_failed_proxy(path: str):
            """Remove a failed/unverified proxy ? it must never reach the vendor."""
            if not os.path.exists(path):
                return
            try:
                os.remove(path)
                print(f"Deleted the failed proxy '{path}' (an unverified proxy must "
                      f"never be shared ? it poisons golden labels).")
            except OSError:
                print(f"WARNING: could not delete the failed proxy '{path}' ? do NOT "
                      f"share it with the vendor.")

        def convert_cvat(self, args):
            """Convert a vendor CVAT rally export into our canonical per-rally CSV.

            Absorbs the CVAT bounding-box delivery format (rally fields carried as box
            attributes) on the ingestion side: each rally's time is recomputed from
            its frame range at the video's true fps, and content defects (overlaps,
            zero-duration, off-vocabulary endings, coarse sampling) are written to an
            issues report rather than silently "fixed". The emitted CSV imports via
            `import-local`; any blocker issue must be resolved first, since the
            importer's validator rejects overlapping / zero-duration rallies.
            """
            from src.parsers.cvat.rally_cvat import (
                convert, write_canonical_csv, render_issue_report,
            )
            try:
                result = convert(args.input, fps=args.fps,
                                 video_path=args.video_path, step=args.step)
            except (FileNotFoundError, ValueError) as e:
                print(f"Error: {e}")
                sys.exit(1)

            if not result.records:
                print(f"Error: the CVAT export parsed to ZERO rallies ({args.input}). "
                      f"Likely the wrong job was exported or the rally_number attribute was
"
                      f"renamed (boxes without a matched rally_number are skipped).")
                sys.exit(1)

            write_canonical_csv(result.records, args.out)

            # Observability report next to the canonical CSV. Never raises.
            from src.reporting import emit_delivery_report
            src_name = os.path.splitext(os.path.basename(args.input))[0]
            dur = (result.meta.get("stop_frame") / result.fps) if
(result.meta.get("stop_frame") and result.fps) else None
            emit_delivery_report(
                out_dir=os.path.dirname(os.path.abspath(args.out)) or ".", source=src_name,
                records=result.records, issues=result.issues, fps=result.fps,
video_path=args.video_path,
                video_duration_s=dur, source_format="cvat", step=result.step,
            )

            report = render_issue_report(result, os.path.basename(args.input))
            if args.issues:
                with open(args.issues, "w", encoding="utf-8") as f:
                    f.write(report)
            print(report)
            print(f"Canonical CSV written: {args.out}  ({len(result.records)} rallies)")
            if result.n_blockers:
                print(f"\n  {result.n_blockers} BLOCKER issue(s) ? import-local will reject
"
                      f"this file until they are resolved (by the vendor or manual
```

```
review).")
            else:
                print("\n  No blockers. Next:")
                print(f"    python -m src.cli.curate import-local --sport <sport> "
                      f"--video-path <video> --annotations-path {args.out} --fps
{result.fps:g}")

        def validate_delivery(self, args):
            """Run the full validation suite on a rally delivery and emit a report.

            Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
            CSV (--csv). Writes, into --out-dir:
              <id>_report.md    human report (scoreboard + issues + rallies to review)
              <id>_report.json  machine summary (for CI / a bulk-ingest loop)
              <id>_review.csv   the annotatable manual-verification sheet
            Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
            """
            import json
            from src.parsers.cvat.rally_cvat import (
                convert, load_canonical_csv, detect_issues,
                build_summary, render_validation_md, write_review_csv,
            )
            vid = args.video_id or "delivery"
            fps = None
            step = None
            stop_frame = None
            if args.input:
                try:
                    result = convert(args.input, fps=args.fps, video_path=args.video_path)
                except (FileNotFoundError, ValueError) as e:
                    print(f"Error: {e}")
                    sys.exit(1)
                records, issues, fps = result.records, result.issues, result.fps
                step = result.step
                stop_frame = result.meta.get("stop_frame")
                if not records:
                    print(f"Error: the CVAT export parsed to ZERO rallies ({args.input}). "
                          f"The wrong job was exported or the rally_number attribute was
renamed. "
                          f"GATE: BLOCKED.")
                    sys.exit(2)
            elif args.csv:
                try:
                    records = load_canonical_csv(args.csv)
                except FileNotFoundError as e:
                    print(f"Error: {e}")
                    sys.exit(1)
                if not records:
                    print(f"Error: no usable rally rows in {args.csv}")
                    sys.exit(1)
                issues = detect_issues(records)
            else:
                print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
                sys.exit(1)

            os.makedirs(args.out_dir, exist_ok=True)
            summary = build_summary(records, issues, source=vid, fps=fps)
            md = render_validation_md(records, issues, source=vid)
            json_path = os.path.join(args.out_dir, f"{vid}_report.json")
            md_path = os.path.join(args.out_dir, f"{vid}_report.md")
            review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
            with open(json_path, "w", encoding="utf-8") as f:
                json.dump(summary, f, indent=2)
            with open(md_path, "w", encoding="utf-8") as f:
                f.write(md)
```

```
            write_review_csv(records, issues, review_path)

            # Observability report (envelope JSON + technical md + customer summary),
            # colocated with the existing artifacts. Never raises.
            from src.reporting import emit_delivery_report
            duration_s = (stop_frame / fps) if (stop_frame and fps) else None
            report_paths = emit_delivery_report(
                out_dir=args.out_dir, source=vid, records=records, issues=issues, fps=fps,
                video_path=args.video_path, video_duration_s=duration_s,
                source_format=("cvat" if args.input else "csv"), step=step,
            )

            print(md)
            print(f"\nWrote:\n  {md_path}\n  {json_path}\n  {review_path}")
            if report_paths:
                print(f"  {report_paths.get('technical_md')}\n
{report_paths.get('customer_md')}\n  {report_paths.get('json')}")
            if summary["n_blockers"]:
                print(f"\n  GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be
resolved.")
                sys.exit(2)
            print("\n  GATE: PASS ? review the flagged rallies, then `import-local`.")

        def curate(self):
            """Interactive loop to review and curate staging video annotations."""
            staging_videos = self.db.get_videos_by_status(CurationStatus.STAGING)
            if not staging_videos:
                print("\nNo staging video annotations to review.\n")
                return

            print(f"\nFound {len(staging_videos)} matches in STAGING state for curation:")
            table_data = []
            for idx, video in enumerate(staging_videos):
                table_data.append([
                    idx + 1,
                    video.video_id,
                    video.match_name,
                    video.sport.value,
                    video.license_type.value,
                    "YES" if video.is_commercial else "NO"
                ])

            print(tabulate(table_data, headers=["#", "Video ID", "Match Name", "Sport",
"License", "Commercial Safe?"], tablefmt="grid"))
            print("\nEnter a number (1-{}) to review details, or 'q' to
quit:".format(len(staging_videos)))

            choice = input("> ").strip()
            if choice.lower() == 'q':
                return

            try:
                idx = int(choice) - 1
                if idx < 0 or idx >= len(staging_videos):
                    print("Invalid choice.")
                    return
                self._curate_video_interactive(staging_videos[idx])
            except ValueError:
                print("Invalid input.")

        def _curate_video_interactive(self, video: VideoMetadata):
            """Displays details of a specific staging video and prompts for curation
action."""

            # Load and count events
```

24. user (2026-07-02T04:49:01.196Z)

```
1168            rallies = []
1169            pts = []
1170            delivs = []
1171            if video.sport == SportType.BADMINTON:
1172                rallies = self.db.get_badminton_rallies(video.video_id)
1173            elif video.sport == SportType.PICKLEBALL:
1174                rallies = self.db.get_pickleball_rallies(video.video_id)
1175            elif video.sport == SportType.TABLE_TENNIS:
1176                rallies = self.db.get_tabletennis_rallies(video.video_id)
1177            elif video.sport == SportType.TENNIS:
1178                pts = self.db.get_tennis_points(video.video_id)
1179            elif video.sport == SportType.CRICKET:
1180                delivs = self.db.get_cricket_deliveries(video.video_id)
1181
1182            while True:
1183                print("\n================ REVIEWING VIDEO ================")
1184                print(f"Match Name   : {video.match_name}")
1185                print(f"Video ID     : {video.video_id}")
1186                print(f"Sport        : {video.sport.value}")
1187                print(f"Video URL    : {video.video_url or 'N/A'}")
1188                print(f"Local Path   : {video.local_path or 'N/A'}")
1189                print(f"License      : {video.license_type.value} ({video.license_url or 'no
URL'})")
1190                print(f"Commercial OK: {video.is_commercial}")
1191
1192                if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
1193                    print(f"Total Rallies: {len(rallies)}")
1194                    if rallies:
1195                        print("Sample rallies:")
1196                        sample_data = [[r.rally_number, f"{r.start_time}s - {r.end_time}s",
r.score_after, r.shots_count] for r in rallies[:5]]
1197                        print(tabulate(sample_data, headers=["Rally #", "Duration", "Score",
"Shots"], tablefmt="simple"))
1198                elif video.sport == SportType.TENNIS:
1199                    print(f"Total Points : {len(pts)}")
1200                    if pts:
1201                        print("Sample points:")
1202                        sample_data = [[p.point_number, f"{p.start_time}s - {p.end_time}s",
p.game_score] for p in pts[:5]]
1203                        print(tabulate(sample_data, headers=["Point #", "Duration",
"Score"], tablefmt="simple"))
1204                elif video.sport == SportType.CRICKET:
1205                    print(f"Total Balls  : {len(delivs)}")
1206                    if delivs:
1207                        print("Sample deliveries:")
1208                        sample_data = [[f"{d.over}.{d.ball}", f"{d.start_time}s -
{d.end_time}s", d.runs, "Wicket" if d.wicket else ""] for d in delivs[:5]]
1209                        print(tabulate(sample_data, headers=["Over.Ball", "Duration",
"Runs", "Wicket"], tablefmt="simple"))
1210
1211                print("\nChoose Curation Action:")
1212                print("  [a] Approve (promote to CURATED)")
1213                print("  [r] Reject (delete from database)")
1214                print("  [p] Play a rally/point segment in browser")
1215                print("  [k] Keep in Staging (do nothing and go back)")
1216
1217                action = input("> ").strip().lower()
1218                if action == 'a':
1219                    self.db.update_status(video.video_id, CurationStatus.CURATED)
1220                    print(f"Successfully promoted '{video.match_name}' to CURATED.\n")
1221                    break
1222                elif action == 'r':
```

```
                    self.db.delete_video(video.video_id)
                    print(f"Successfully rejected and deleted '{video.match_name}'.\n")
                    break
                elif action == 'k':
                    print("Curation state unchanged.\n")
                    break
                elif action == 'p':
                    self._play_segment_interactive(video, rallies, pts, delivs)
                else:
                    print("Invalid action selected.")

        def _play_segment_interactive(self, video: VideoMetadata, rallies:
List[BadmintonRally], pts: List[TennisPoint], delivs: List[CricketDelivery]):
            import webbrowser

            # Get start/end based on user input
            start_time = None
            end_time = None
            label = "segment"

            if video.sport in (SportType.BADMINTON, SportType.PICKLEBALL,
SportType.TABLE_TENNIS):
                if not rallies:
                    print("No rallies to play.")
                    return
                idx_str = input(f"Enter Rally number to play (1-{len(rallies)}): ").strip()
                try:
                    rally_num = int(idx_str)
                    rally = next((r for r in rallies if r.rally_number == rally_num), None)
                    if rally:
                        start_time = rally.start_time
                        end_time = rally.end_time
                        label = f"Rally {rally_num}"
                except ValueError:
                    pass
            elif video.sport == SportType.TENNIS:
                if not pts:
                    print("No points to play.")
                    return
                idx_str = input(f"Enter Point number to play (1-{len(pts)}): ").strip()
                try:
                    pt_num = int(idx_str)
                    pt = next((p for p in pts if p.point_number == pt_num), None)
                    if pt:
                        start_time = pt.start_time
                        end_time = pt.end_time
                        label = f"Point {pt_num}"
                except ValueError:
                    pass
            elif video.sport == SportType.CRICKET:
                if not delivs:
                    print("No deliveries to play.")
                    return
                over_str = input("Enter Over number: ").strip()
                ball_str = input("Enter Ball number: ").strip()
                try:
                    o_num = int(over_str)
                    b_num = int(ball_str)
                    d = next((x for x in delivs if x.over == o_num and x.ball == b_num),
None)
                    if d:
                        start_time = d.start_time
                        end_time = d.end_time
                        label = f"Over {o_num} Ball {b_num}"
                except ValueError:
```

```
                    pass

            if start_time is None or end_time is None:
                print("Segment not found or invalid input.")
                return

            print(f"Playing {label} (start: {start_time}s, end: {end_time}s)...")

            # Check video location
            play_url = None
            if video.local_path and os.path.exists(video.local_path):
                abs_path = os.path.abspath(video.local_path)
                # Create file:// URL with media fragment (browsers support #t=start,end
fragment)
                play_url = f"file:///{abs_path.replace(os.sep,
'/')}#t={start_time},{end_time}"
            elif video.video_url:
                # YouTube URL: seek to start time
                if "youtube.com" in video.video_url or "youtu.be" in video.video_url:
                    t_sec = int(start_time)
                    sep = "&" if "?" in video.video_url else "?"
                    play_url = f"{video.video_url}{sep}t={t_sec}s"
                else:
                    play_url = video.video_url

            if play_url:
                print(f"Opening browser: {play_url}")
                webbrowser.open(play_url)
            else:
                print("Error: No local video file or remote URL available to play.")


    def main():
        parser = argparse.ArgumentParser(description="Sports Data Curator CLI")
        subparsers = parser.add_subparsers(dest="command")

        # Status command
        subparsers.add_parser("status", help="Display summary status of sports metadata
database")

        # Curate command
        subparsers.add_parser("curate", help="Launch interactive curation tool for staging
entries")

        # Import-local command
        import_parser = subparsers.add_parser("import-local", help="Manually import a local
video and annotations")
        import_parser.add_argument("--sport", required=True, help="Sport type (e.g.
badminton, tennis, cricket)")
        import_parser.add_argument("--video-path", required=True, help="Path to local video
file")
        import_parser.add_argument("--annotations-path", required=True, help="Path to local
JSON/CSV annotations file")
        import_parser.add_argument("--match-name", help="Name of the match (defaults to
video filename)")
        import_parser.add_argument("--video-id", help="Custom alphanumeric video ID")
        import_parser.add_argument("--license", default="Proprietary", help="License type
(e.g. MIT, CC-BY-NC, Proprietary)")
        import_parser.add_argument("--fps", type=float, default=30.0, help="Frames per
second of the video")
        import_parser.add_argument("--resolution", default="1080p", help="Resolution of the
video (e.g., 1080p, 720p)")
        import_parser.add_argument("--force", action="store_true",
                                   help="Overwrite an existing video's annotations (default:
refuse, to protect golden labels)")
```

```
1337        import_parser.add_argument("--normalize", action="store_true",
1338                                   help="Run normalize-annotations on a CSV input first
(drops unusable rows, "
1339                                        "renumbers fractional/duplicate rally ids 1..N by
start time)")
1340
1341        # Fetch command ? (re)download videos at best resolution, update DB in place
1342        fetch_parser = subparsers.add_parser(
1343            "fetch", help="(Re)download videos at best available resolution, updating the DB
in place")
1344        fetch_parser.add_argument("--sport", help="Restrict to a single sport (default: all
supported_sports)")
1345        fetch_parser.add_argument("--video-id", help="Fetch a single video by ID")
1346        fetch_parser.add_argument("--status", choices=["staging", "curated", "all"],
default="all",
1347                                  help="Which curation statuses to fetch (default: all)")
1348        fetch_parser.add_argument("--min-height", type=int, default=720,
1349                                  help="Skip videos already at/above this height unless
--force (default: 720)")
1350        fetch_parser.add_argument("--max-height", type=int, default=None,
1351                                  help="Cap download resolution (e.g. 1080); default: best
available")
1352        fetch_parser.add_argument("--force", action="store_true",
1353                                  help="Re-download even if a good-enough local file already
exists")
1354        fetch_parser.add_argument("--include-noncommercial", action="store_true",
1355                                  help="Include videos whose license is not commercially
safe (default: commercial only)")
1356        fetch_parser.add_argument("--cookies-from-browser", dest="cookies_from_browser",
metavar="BROWSER",
1357                                  help="Use a logged-in browser's YouTube cookies (e.g.
chrome, edge, firefox, brave). "
1358                                       "A Premium login gives reliable best-resolution
downloads. "
1359                                       "Chromium browsers must be CLOSED on Windows (cookie
DB lock).")
1360        fetch_parser.add_argument("--cookies", metavar="FILE",
1361                                  help="Path to a Netscape-format cookies.txt (alternative
to --cookies-from-browser)")
1362        fetch_parser.add_argument("--player-client", dest="player_client",
default="default", metavar="CLIENT",
1363                                  help="YouTube player client yt-dlp uses (default:
'default'). Avoid web/web_safari "
1364                                       "(SABR-forced -> drops to 144p). 'tv'/'mweb' also
serve 1080p if needed.")
1365        fetch_parser.add_argument("--sleep", type=float, default=4.0, metavar="SECONDS",
1366                                  help="Pause between videos to avoid YouTube rate-limiting
(default: 4)")
1367        fetch_parser.add_argument("--retries", type=int, default=2, metavar="N",
1368                                  help="Retries per video when a download fails or returns a
low format (default: 2)")
1369        fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
1370                                  help="Base backoff between retries; grows linearly per
attempt (default: 20)")
1371
1372        # Prep-annotation-proxy command ? vendor-handoff re-encode (ADR-002)
1373        prep_parser = subparsers.add_parser(
1374            "prep-annotation-proxy",
1375            help="Re-encode a video into a light, frame-identical annotation proxy for "
1376                 "vendor handoff (ADR-002): downscaled + scrub-friendly, same frame "
1377                 "count/fps/timeline, parity-verified, provenance recorded")
1378        prep_parser.add_argument("--video-path", required=True,
1379                                 help="The ORIGINAL source video (stays the system-of-record
file)")
```

```
1380        prep_parser.add_argument("--out",
1381                                 help="Proxy output path (default:
data/videos/proxies/<stem>_proxy.mp4)")
1382        prep_parser.add_argument("--video-id",
1383                                 help="Manifest key (default: derived from the filename,
like import-local)")
1384        prep_parser.add_argument("--height", type=int, default=1080,
1385                                 help="Max proxy height; never upscales (default: 1080 ? "
1386                                      "720 risks losing the table-tennis ball)")
1387        prep_parser.add_argument("--crf", type=int, default=19,
1388                                 help="x264 quality, lower=better (default: 19)")
1389        prep_parser.add_argument("--gop", type=int, default=15,
1390                                 help="Max keyframe interval in frames; small = snappy "
1391                                      "frame-stepping (default: 15)")
1392        prep_parser.add_argument("--preset", default="medium",
1393                                 help="libx264 speed/size preset (default: medium; ignored
for nvenc)")
1394        prep_parser.add_argument("--encoder", choices=["libx264", "nvenc"],
default="libx264",
1395                                 help="Video encoder: libx264 (CPU, default) or nvenc "
1396                                      "(NVIDIA GPU ? much faster on 4K, slightly larger
files)")
1397        prep_parser.add_argument("--manifest", default=os.path.join("data",
"annotation_proxy_manifest.csv"),
1398                                 help="Provenance CSV to append to (default:
data/annotation_proxy_manifest.csv; "
1399                                      "commit it ? it is the proxy<->original mapping of
record)")
1400        prep_parser.add_argument("--allow-vfr", action="store_true",
1401                                 help="Proceed on a variable-frame-rate source (frame/fps "
1402                                      "seconds are approximate; default: refuse)")
1403        prep_parser.add_argument("--force", action="store_true",
1404                                 help="Overwrite an existing proxy file / append a manifest
"
1405                                      "row for a video_id already mapped to a different
source")
1406
1407        # Prep-annotation-batch command ? proxy a whole directory tree (ADR-002)
1408        batch_parser = subparsers.add_parser(
1409            "prep-annotation-batch",
1410            help="Proxy every video under a directory tree, mirroring its folder layout "
1411                 "into --out-dir (the proxies-only tree that goes to the vendor)")
1412        batch_parser.add_argument("--input-dir", required=True,
1413                                  help="Root directory: each subdirectory holds well-named "
1414                                       "source video(s); root-level videos also accepted")
1415        batch_parser.add_argument("--out-dir", required=True,
1416                                  help="Mirrored output root ? receives ONLY the proxies")
1417        batch_parser.add_argument("--height", type=int, default=1080,
1418                                  help="Max proxy height; never upscales (default: 1080)")
1419        batch_parser.add_argument("--crf", type=int, default=19,
1420                                  help="x264 quality, lower=better (default: 19)")
1421        batch_parser.add_argument("--gop", type=int, default=15,
1422                                  help="Max keyframe interval in frames (default: 15)")
1423        batch_parser.add_argument("--preset", default="medium",
1424                                  help="libx264 speed/size preset (default: medium; ignored
for nvenc)")
1425        batch_parser.add_argument("--encoder", choices=["libx264", "nvenc"],
default="libx264",
1426                                  help="Video encoder: libx264 (CPU, default) or nvenc "
1427                                       "(NVIDIA GPU ? much faster on 4K, slightly larger
files)")
1428        batch_parser.add_argument("--manifest", default=os.path.join("data",
"annotation_proxy_manifest.csv"),
1429                                  help="Provenance CSV appended per video (default: "
1430                                       "data/annotation_proxy_manifest.csv)")
```

```
1431        batch_parser.add_argument("--allow-vfr", action="store_true",
1432                                  help="Proceed on variable-frame-rate sources (default:
refuse)")
1433        batch_parser.add_argument("--allow-duplicates", action="store_true",
1434                                  help="Proceed even if the same video appears in >1 folder
"
1435                                       "(default: BLOCK; use for an intentional A/A test)")
1436        batch_parser.add_argument("--dry-run", action="store_true",
1437                                  help="Run the preflight scan and print the report, then
stop "
1438                                       "(no proxies written)")
1439        batch_parser.add_argument("--force", action="store_true",
1440                                  help="Redo existing proxies / re-key colliding manifest
video_ids")
1441
1442        # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
1443        cvat_parser = subparsers.add_parser(
1444            "convert-cvat",
1445            help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
1446        cvat_parser.add_argument("--input", required=True,
1447                                 help="Path to the CVAT XML export (image- or track-mode)")
1448        cvat_parser.add_argument("--out", required=True,
1449                                 help="Path to write the canonical per-rally CSV (import-
local input)")
1450        cvat_parser.add_argument("--video-path",
1451                                 help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
1452        cvat_parser.add_argument("--fps", type=float,
1453                                 help="Override fps instead of probing the video")
1454        cvat_parser.add_argument("--step", type=int,
1455                                 help="Frame sampling step (default: read from CVAT meta,
else inferred)")
1456        cvat_parser.add_argument("--issues",
1457                                 help="Path to also write the human-readable issues report")
1458
1459        # Validate-delivery command ? run the full check suite + emit a structured report
1460        vd_parser = subparsers.add_parser(
1461            "validate-delivery",
1462            help="Validate a rally delivery (CVAT export or canonical CSV) and emit report +
review sheet")
1463        vd_parser.add_argument("--input", help="CVAT XML export (use with --video-
path/--fps)")
1464        vd_parser.add_argument("--csv", help="An already-canonical per-rally CSV to
validate")
1465        vd_parser.add_argument("--video-path",
1466                               help="Source video; fps via ffprobe (when using --input)")
1467        vd_parser.add_argument("--fps", type=float, help="Override fps (when using
--input)")
1468        vd_parser.add_argument("--video-id", help="Names the report files (<id>_report.md,
etc.)")
1469        vd_parser.add_argument("--out-dir", default="data/validation",
1470                               help="Where to write the report + review sheet")
1471
1472        # Export command
1473        export_parser = subparsers.add_parser(
1474            "export", help="Export curated annotations as indexer-ready eval/training CSVs +
manifest")
1475        export_parser.add_argument("--out-dir", default="data/eval_export",
1476                                   help="Output directory for per-video CSVs and
manifest.json")
1477        export_parser.add_argument("--sport",
1478                                   help="Restrict to a single sport (default: all
supported_sports)")
1479        export_parser.add_argument("--include-staging", action="store_true",
```

```
1480                                    help="Also export videos still in staging (default:
curated only)")
1481        export_parser.add_argument("--include-noncommercial", action="store_true",
1482                                    help="Include videos whose license is not commercially
safe (default: commercial only)")
1483
1484        # Reclassify-licenses command (one-shot corrective maintenance)
1485        reclass_parser = subparsers.add_parser(
1486            "reclassify-licenses",
1487            help="Fix annotation-vs-video license conflation (e.g. MIT stamped on broadcast
video)")
1488        reclass_parser.add_argument("--apply", action="store_true",
1489                                    help="Persist changes (default: dry-run preview only)")
1490
1491        args = parser.parse_args()
1492        cli = SportsDataCLI()
1493
1494        try:
1495            _dispatch(cli, args, parser)
1496        except CurationError as e:
1497            # A library-style pre-flight failure (A2): print the actionable message and
1498            # exit 1, preserving the prior CLI behaviour now that _guard_import raises.
1499            print(f"Error: {e}")
1500            sys.exit(1)
1501
1502
1503    def _dispatch(cli: "SportsDataCLI", args, parser) -> None:
1504        """Route the parsed command to its handler. Separated so the CurationError
1505        catch in main() wraps every command uniformly."""
1506        if args.command == "status":
1507            cli.status()
1508        elif args.command == "reclassify-licenses":
1509            cli.reclassify_licenses(args)
1510        elif args.command == "curate":
1511            cli.curate()
1512        elif args.command == "import-local":
1513            cli.import_local(args)
1514        elif args.command == "prep-annotation-proxy":
1515            cli.prep_annotation_proxy(args)
1516        elif args.command == "prep-annotation-batch":
1517            cli.prep_annotation_batch(args)
1518        elif args.command == "convert-cvat":
1519            cli.convert_cvat(args)
1520        elif args.command == "validate-delivery":
1521            cli.validate_delivery(args)
1522        elif args.command == "export":
1523            cli.export_eval_set(args)
1524        elif args.command == "fetch":
1525            cli.fetch(args)
1526        else:
1527            # Default: print help
1528            parser.print_help()
1529
1530    if __name__ == "__main__":
1531        main()
1532
```

25. assistant (2026-07-02T04:49:07.771Z)

Now let me read the DB layer and schemas.

26. user (2026-07-02T04:49:10.195Z)

```
1       import sqlite3
2       import os
```

```
3       import logging
4       from contextlib import contextmanager
5       from enum import Enum
6       from typing import List, Dict, Any, Optional, Type, TypeVar
7       from src.core.schemas.base import VideoMetadata, SportType, CurationStatus, LicenseType
8       from src.core.schemas.badminton import BadmintonRally
9       from src.core.schemas.tennis import TennisPoint
10      from src.core.schemas.cricket import CricketDelivery
11      from src.core.schemas.pickleball import PickleballRally
12      from src.core.schemas.tabletennis import TableTennisRally
13
14      logger = logging.getLogger(__name__)
15
16      # Maps each sport to its annotation table and the column(s) that order segments
17      # chronologically. All child tables expose (start_time, end_time), so a
18      # single generic reader can serve every sport ? adding a new sport is one entry
19      # here plus its CREATE TABLE, with no new query code.
20      SEGMENT_TABLES = {
21          SportType.BADMINTON: ("badminton_rallies", "rally_number"),
22          SportType.TENNIS: ("tennis_points", "point_number"),
23          SportType.CRICKET: ("cricket_deliveries", "over_num, ball_num"),
24          SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
25          SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
26      }
27
28
29      class SportsDatabase:
30          def __init__(self, db_path: str):
31              self.db_path = db_path
32              # Ensure target directory exists
33              os.makedirs(os.path.dirname(os.path.abspath(db_path)), exist_ok=True)
34              self._init_db()
35
36          @contextmanager
37          def _get_connection(self):
38              """Yield a SQLite connection and always close it.
39
40              Used as ``with self._get_connection() as conn:``. A plain
41              ``sqlite3.connect(...)`` used as a context manager commits/rolls back the
42              transaction but does NOT close the connection, which leaks handles and
43              raises ResourceWarnings. This wrapper guarantees the connection is closed
44              on exit; writers still commit explicitly inside the block.
45              """
46              conn = sqlite3.connect(self.db_path)
47              conn.row_factory = sqlite3.Row
48              # SQLite disables foreign keys per-connection by default; enable so the
49              # ON DELETE CASCADE declarations on the child tables actually fire.
50              conn.execute("PRAGMA foreign_keys = ON")
51              # Wait (up to 5s) for a competing writer's lock instead of failing immediately
with
52              # "database is locked" ? makes concurrent access robust (a sibling CLI, or a
future
53              # web-driven import_annotations writer) rather than racy. Writers still commit
explicitly.
54              conn.execute("PRAGMA busy_timeout = 5000")
55              try:
56                  yield conn
57              finally:
58                  conn.close()
59
60          def _init_db(self):
61              with self._get_connection() as conn:
62                  cursor = conn.cursor()
63
64                  # 1. Create common videos table
```

```
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS videos (
                    video_id TEXT PRIMARY KEY,
                    sport TEXT NOT NULL,
                    video_url TEXT,
                    local_path TEXT,
                    license_type TEXT NOT NULL,
                    license_url TEXT,
                    annotation_license TEXT,
                    is_commercial INTEGER NOT NULL DEFAULT 0,
                    status TEXT NOT NULL DEFAULT 'staging',
                    match_name TEXT NOT NULL,
                    fps REAL,
                    resolution TEXT
                )
                """)

                # 2. Create badminton_rallies table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS badminton_rallies (
                    video_id TEXT NOT NULL,
                    rally_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    score_before TEXT,
                    score_after TEXT,
                    shots_count INTEGER,
                    ending_reason TEXT,
                    last_shot_outcome TEXT,
                    PRIMARY KEY (video_id, rally_number),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # 3. Create tennis_points table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS tennis_points (
                    video_id TEXT NOT NULL,
                    point_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    set_score TEXT,
                    game_score TEXT,
                    server TEXT,
                    ending_type TEXT,
                    PRIMARY KEY (video_id, point_number),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # 4. Create cricket_deliveries table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS cricket_deliveries (
                    video_id TEXT NOT NULL,
                    over_num INTEGER NOT NULL,
                    ball_num INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    batsman TEXT,
                    bowler TEXT,
                    runs INTEGER,
                    wicket INTEGER DEFAULT 0,
                    extras INTEGER DEFAULT 0,
                    PRIMARY KEY (video_id, over_num, ball_num),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
```

```
                )
                """)

                # 5. Create pickleball_rallies table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS pickleball_rallies (
                    video_id TEXT NOT NULL,
                    rally_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    score_before TEXT,
                    score_after TEXT,
                    shots_count INTEGER,
                    ending_reason TEXT,
                    last_shot_outcome TEXT,
                    PRIMARY KEY (video_id, rally_number),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                # 6. Create tabletennis_rallies table
                cursor.execute("""
                CREATE TABLE IF NOT EXISTS tabletennis_rallies (
                    video_id TEXT NOT NULL,
                    rally_number INTEGER NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    score_before TEXT,
                    score_after TEXT,
                    shots_count INTEGER,
                    ending_reason TEXT,
                    last_shot_outcome TEXT,
                    PRIMARY KEY (video_id, rally_number),
                    FOREIGN KEY (video_id) REFERENCES videos(video_id) ON DELETE CASCADE
                )
                """)

                conn.commit()
                self._migrate(cursor)
                conn.commit()

        #: The current schema version. Bump when adding a migration step below.
        SCHEMA_VERSION = 1

        def _migrate(self, cursor) -> None:
            """Run the versioned migration ladder via PRAGMA user_version.

            Each step is gated on the DB's current version and bumps it after applying.
            The column-sniffing inside each step stays (so a DB that predates versioning
            upgrades transparently on first open), but is now scoped and ordered. Future
            migrations add a new step + bump SCHEMA_VERSION ? no more ad-hoc ALTERs.
            See docs/CODE_HEALTH_PLAN.md §7 Tier C.
            """
            current = cursor.execute("PRAGMA user_version").fetchone()[0]
            if current >= self.SCHEMA_VERSION:
                return  # up to date

            # --- v1: add last_shot_outcome to rally tables + split video/annotation license
            if current < 1:
                # add last_shot_outcome to rally tables that predate the column. CREATE
TABLE
                # IF NOT EXISTS won't alter an existing table, so do it explicitly and
                # idempotently (skip if the column is already there).
                for tbl in ("badminton_rallies", "pickleball_rallies",
"tabletennis_rallies"):
```

```
                    existing = [r[1] for r in cursor.execute(f"PRAGMA
table_info({tbl})").fetchall()]
                    if "last_shot_outcome" not in existing:
                        cursor.execute(f"ALTER TABLE {tbl} ADD COLUMN last_shot_outcome
TEXT")

                # split the conflated single license into video vs annotation license.
                # annotation_license records the dataset's annotation rights (e.g.
ShuttleSet
                # MIT) for provenance; license_type stays the VIDEO's rights, which gates
                # commercial use.
                video_cols = [r[1] for r in cursor.execute("PRAGMA
table_info(videos)").fetchall()]
                if "annotation_license" not in video_cols:
                    cursor.execute("ALTER TABLE videos ADD COLUMN annotation_license TEXT")

                cursor.execute(f"PRAGMA user_version = {self.SCHEMA_VERSION}")

        def insert_video(self, video: VideoMetadata) -> bool:
            """Insert a video entry, or update it in place if it already exists.

            Uses an UPSERT rather than INSERT OR REPLACE: REPLACE would DELETE the
            existing row, which (with foreign keys enabled) would cascade-delete the
            video's child annotations. UPSERT mutates the row in place and leaves
            children untouched.
            """
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                INSERT INTO videos (
                    video_id, sport, video_url, local_path, license_type,
                    license_url, annotation_license, is_commercial, status, match_name, fps,
resolution
                ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                ON CONFLICT(video_id) DO UPDATE SET
                    sport=excluded.sport,
                    video_url=excluded.video_url,
                    local_path=excluded.local_path,
                    license_type=excluded.license_type,
                    license_url=excluded.license_url,
                    annotation_license=excluded.annotation_license,
                    is_commercial=excluded.is_commercial,
                    status=excluded.status,
                    match_name=excluded.match_name,
                    fps=excluded.fps,
                    resolution=excluded.resolution
                """, (
                    video.video_id,
                    video.sport.value,
                    video.video_url,
                    video.local_path,
                    video.license_type.value,
                    video.license_url,
                    video.annotation_license.value if video.annotation_license else None,
                    1 if video.is_commercial else 0,
                    video.status.value,
                    video.match_name,
                    video.fps,
                    video.resolution
                ))
                conn.commit()
                return True

        def insert_badminton_rallies(self, rallies: List[BadmintonRally]) -> int:
            if not rallies:
```

```
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                # First clear existing rallies for this video_id to avoid key conflicts on
updates
                video_id = rallies[0].video_id
                cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))

                for rally in rallies:
                    cursor.execute("""
                    INSERT INTO badminton_rallies (
                        video_id, rally_number, start_time, end_time,
                        score_before, score_after, shots_count, ending_reason,
last_shot_outcome
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        rally.video_id,
                        rally.rally_number,
                        rally.start_time,
                        rally.end_time,
                        rally.score_before,
                        rally.score_after,
                        rally.shots_count,
                        rally.ending_reason,
                        rally.last_shot_outcome
                    ))
                conn.commit()
                return len(rallies)

        def insert_tennis_points(self, points: List[TennisPoint]) -> int:
            if not points:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = points[0].video_id
                cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))

                for pt in points:
                    cursor.execute("""
                    INSERT INTO tennis_points (
                        video_id, point_number, start_time, end_time,
                        set_score, game_score, server, ending_type
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        pt.video_id,
                        pt.point_number,
                        pt.start_time,
                        pt.end_time,
                        pt.set_score,
                        pt.game_score,
                        pt.server,
                        pt.ending_type
                    ))
                conn.commit()
                return len(points)

        def insert_cricket_deliveries(self, deliveries: List[CricketDelivery]) -> int:
            if not deliveries:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = deliveries[0].video_id
                cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
(video_id,))
```

```
                for deliv in deliveries:
                    cursor.execute("""
                    INSERT INTO cricket_deliveries (
                        video_id, over_num, ball_num, start_time, end_time,
                        batsman, bowler, runs, wicket, extras
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        deliv.video_id,
                        deliv.over,
                        deliv.ball,
                        deliv.start_time,
                        deliv.end_time,
                        deliv.batsman,
                        deliv.bowler,
                        deliv.runs,
                        1 if deliv.wicket else 0,
                        deliv.extras
                    ))
                conn.commit()
                return len(deliveries)

        def insert_pickleball_rallies(self, rallies: List[PickleballRally]) -> int:
            if not rallies:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = rallies[0].video_id
                cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
(video_id,))

                for rally in rallies:
                    cursor.execute("""
                    INSERT INTO pickleball_rallies (
                        video_id, rally_number, start_time, end_time,
                        score_before, score_after, shots_count, ending_reason,
last_shot_outcome
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        rally.video_id,
                        rally.rally_number,
                        rally.start_time,
                        rally.end_time,
                        rally.score_before,
                        rally.score_after,
                        rally.shots_count,
                        rally.ending_reason,
                        rally.last_shot_outcome
                    ))
                conn.commit()
                return len(rallies)

        def insert_tabletennis_rallies(self, rallies: List[TableTennisRally]) -> int:
            if not rallies:
                return 0
            with self._get_connection() as conn:
                cursor = conn.cursor()
                video_id = rallies[0].video_id
                cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
(video_id,))

                for rally in rallies:
                    cursor.execute("""
                    INSERT INTO tabletennis_rallies (
                        video_id, rally_number, start_time, end_time,
```

```
                        score_before, score_after, shots_count, ending_reason,
last_shot_outcome
                    ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                    """, (
                        rally.video_id,
                        rally.rally_number,
                        rally.start_time,
                        rally.end_time,
                        rally.score_before,
                        rally.score_after,
                        rally.shots_count,
                        rally.ending_reason,
                        rally.last_shot_outcome
                    ))
                conn.commit()
                return len(rallies)

        @staticmethod
        def _row_to_video(row) -> VideoMetadata:
            """Rehydrate a DB row into VideoMetadata.

            B5: enum construction (SportType/LicenseType/CurationStatus) used to raise a
            bare ValueError on any value not in the enum ? a typo or a value from an older
            schema made the whole query loop explode with a cryptic message and no clue
            WHICH row was bad. Now: license values unknown to the enum map to UNKNOWN and
            are logged loudly (OQ2 default: map + log, don't lose data); sport/status have
            no safe default so they still raise but with the offending video_id in context.
            """
            keys = row.keys()
            ann = row['annotation_license'] if 'annotation_license' in keys else None
            video_id = row['video_id']

            def _safe_license(value, field: str) -> LicenseType:
                """Parse a license enum value; map unknown -> UNKNOWN with a warning."""
                if not value:
                    return LicenseType.UNKNOWN
                try:
                    return LicenseType(value)
                except ValueError:
                    logger.warning(
                        "video %r has unrecognized %s=%r; mapping to UNKNOWN",
                        video_id, field, value)
                    return LicenseType.UNKNOWN

            try:
                sport = SportType(row['sport'])
            except ValueError as e:
                # No safe default for sport ? surface which row is bad before re-raising.
                raise ValueError(
                    f"video {video_id!r} has unrecognized sport={row['sport']!r}; "
                    f"cannot rehydrate (fix the DB row or extend SportType)") from e

            try:
                status = CurationStatus(row['status'])
            except ValueError as e:
                raise ValueError(
                    f"video {video_id!r} has unrecognized status={row['status']!r}; "
                    f"cannot rehydrate (fix the DB row)") from e

            return VideoMetadata(
                video_id=video_id,
                sport=sport,
                video_url=row['video_url'],
                local_path=row['local_path'],
                license_type=_safe_license(row['license_type'], 'license_type'),
```

```
                license_url=row['license_url'],
                annotation_license=_safe_license(ann, 'annotation_license') if ann else
None,
                is_commercial=bool(row['is_commercial']),
                status=status,
                match_name=row['match_name'],
                fps=row['fps'],
                resolution=row['resolution']
            )

        def get_video(self, video_id: str) -> Optional[VideoMetadata]:
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("SELECT * FROM videos WHERE video_id = ?", (video_id,))
                row = cursor.fetchone()
                if row:
                    return self._row_to_video(row)
            return None

        def get_videos_by_status(self, status: CurationStatus, sport: Optional[SportType] =
None) -> List[VideoMetadata]:
            query = "SELECT * FROM videos WHERE status = ?"
            params = [status.value]
            if sport:
                query += " AND sport = ?"
                params.append(sport.value)

            videos = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute(query, params)
                for row in cursor.fetchall():
                    videos.append(self._row_to_video(row))
            return videos

        def get_status_counts(self) -> List[tuple]:
            """Return (sport, status, count) rows for the status summary.

            Public read API so callers (e.g. the CLI `status` command) don't have to
            reach into the private `_get_connection` context manager.
            """
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute(
                    "SELECT sport, status, COUNT(*) as cnt FROM videos GROUP BY status,
sport"
                )
                return [(r["sport"], r["status"], r["cnt"]) for r in cursor.fetchall()]

        def get_badminton_rallies(self, video_id: str) -> List[BadmintonRally]:
            rallies = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                SELECT * FROM badminton_rallies
                WHERE video_id = ?
                ORDER BY rally_number
                """, (video_id,))
                for row in cursor.fetchall():
                    rallies.append(BadmintonRally(
                        video_id=row['video_id'],
                        rally_number=row['rally_number'],
                        start_time=row['start_time'],
                        end_time=row['end_time'],
                        score_before=row['score_before'],
```

```
                    score_after=row['score_after'],
                    shots_count=row['shots_count'],
                    ending_reason=row['ending_reason'],
                    last_shot_outcome=row['last_shot_outcome']
                ))
        return rallies

    def get_tennis_points(self, video_id: str) -> List[TennisPoint]:
        points = []
        with self._get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute("""
            SELECT * FROM tennis_points
            WHERE video_id = ?
            ORDER BY point_number
            """, (video_id,))
            for row in cursor.fetchall():
                points.append(TennisPoint(
                    video_id=row['video_id'],
                    point_number=row['point_number'],
                    start_time=row['start_time'],
                    end_time=row['end_time'],
                    set_score=row['set_score'],
                    game_score=row['game_score'],
                    server=row['server'],
                    ending_type=row['ending_type']
                ))
        return points

    def get_cricket_deliveries(self, video_id: str) -> List[CricketDelivery]:
        deliveries = []
        with self._get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute("""
            SELECT * FROM cricket_deliveries
            WHERE video_id = ?
            ORDER BY over_num, ball_num
            """, (video_id,))
            for row in cursor.fetchall():
                deliveries.append(CricketDelivery(
                    video_id=row['video_id'],
                    over=row['over_num'],
                    ball=row['ball_num'],
                    start_time=row['start_time'],
                    end_time=row['end_time'],
                    batsman=row['batsman'],
                    bowler=row['bowler'],
                    runs=row['runs'],
                    wicket=bool(row['wicket']),
                    extras=row['extras']
                ))
        return deliveries

    def get_pickleball_rallies(self, video_id: str) -> List[PickleballRally]:
        rallies = []
        with self._get_connection() as conn:
            cursor = conn.cursor()
            cursor.execute("""
            SELECT * FROM pickleball_rallies
            WHERE video_id = ?
            ORDER BY rally_number
            """, (video_id,))
            for row in cursor.fetchall():
                rallies.append(PickleballRally(
                    video_id=row['video_id'],
```

```
                        rally_number=row['rally_number'],
                        start_time=row['start_time'],
                        end_time=row['end_time'],
                        score_before=row['score_before'],
                        score_after=row['score_after'],
                        shots_count=row['shots_count'],
                        ending_reason=row['ending_reason'],
                        last_shot_outcome=row['last_shot_outcome']
                    ))
            return rallies

        def get_tabletennis_rallies(self, video_id: str) -> List[TableTennisRally]:
            rallies = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("""
                SELECT * FROM tabletennis_rallies
                WHERE video_id = ?
                ORDER BY rally_number
                """, (video_id,))
                for row in cursor.fetchall():
                    rallies.append(TableTennisRally(
                        video_id=row['video_id'],
                        rally_number=row['rally_number'],
                        start_time=row['start_time'],
                        end_time=row['end_time'],
                        score_before=row['score_before'],
                        score_after=row['score_after'],
                        shots_count=row['shots_count'],
                        ending_reason=row['ending_reason'],
                        last_shot_outcome=row['last_shot_outcome']
                    ))
            return rallies

        def get_segment_intervals(self, video_id: str, sport: SportType) -> List[tuple]:
            """Return chronologically-ordered (start_time, end_time) tuples for a video,
            regardless of sport. This is the sport-agnostic view used to export
            indexer-ready evaluation/training labels.

            Sports that have no annotation table yet (e.g. tabletennis, pickleball)
            yield an empty list rather than raising.
            """
            mapping = SEGMENT_TABLES.get(sport)
            if mapping is None:
                return []
            table, order_by = mapping  # both come from the trusted map above, never user
input
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute(
                    f"SELECT start_time, end_time FROM {table} "
                    f"WHERE video_id = ? ORDER BY {order_by}",
                    (video_id,),
                )
                return [(row["start_time"], row["end_time"]) for row in cursor.fetchall()]

        def update_status(self, video_id: str, status: CurationStatus) -> bool:
            with self._get_connection() as conn:
                cursor = conn.cursor()
                cursor.execute("UPDATE videos SET status = ? WHERE video_id = ?",
(status.value, video_id))
                conn.commit()
                return cursor.rowcount > 0

        def delete_video(self, video_id: str) -> bool:
```

```
630             """Deletes a video and cascading child entries."""
631             with self._get_connection() as conn:
632                 cursor = conn.cursor()
633                 # Execute cascade deletes manually just in case SQLite foreign key support
is not active
634                 cursor.execute("DELETE FROM badminton_rallies WHERE video_id = ?",
(video_id,))
635                 cursor.execute("DELETE FROM tennis_points WHERE video_id = ?", (video_id,))
636                 cursor.execute("DELETE FROM cricket_deliveries WHERE video_id = ?",
(video_id,))
637                 cursor.execute("DELETE FROM pickleball_rallies WHERE video_id = ?",
(video_id,))
638                 cursor.execute("DELETE FROM tabletennis_rallies WHERE video_id = ?",
(video_id,))
639                 cursor.execute("DELETE FROM videos WHERE video_id = ?", (video_id,))
640                 conn.commit()
641                 return cursor.rowcount > 0
642
```

27. user (2026-07-02T04:49:18.258Z)

```
1       from pydantic import BaseModel, Field, model_validator
2       from typing import Optional
3       from enum import Enum
4
5       class SportType(str, Enum):
6           BADMINTON = "badminton"
7           TENNIS = "tennis"
8           TABLE_TENNIS = "tabletennis"
9           PICKLEBALL = "pickleball"
10          CRICKET = "cricket"
11
12      class LicenseType(str, Enum):
13          MIT = "MIT"
14          APACHE_2 = "Apache-2.0"
15          BSD = "BSD"
16          CC0 = "CC0"
17          CC_BY = "CC-BY"
18          CC_BY_NC = "CC-BY-NC"
19          CC_BY_NC_SA = "CC-BY-NC-SA"
20          PUBLIC_DOMAIN = "Public-Domain"
21          PROPRIETARY = "Proprietary"
22          UNKNOWN = "Unknown"
23
24      class CurationStatus(str, Enum):
25          STAGING = "staging"
26          CURATED = "curated"
27          REJECTED = "rejected"
28
29      #: Licenses considered safe for COMMERCIAL use. The single source of truth: B1's
30      #: is_commercial validator uses this directly, and validator.DatasetValidator's
31      #: default whitelist is derived from it ([lt.value for lt in COMMERCIAL_LICENSES],
32      #: via T9). A user can still override via config.yaml's commercial_licenses list.
33      COMMERCIAL_LICENSES = frozenset({
34          LicenseType.MIT, LicenseType.APACHE_2, LicenseType.BSD, LicenseType.CC0,
35          LicenseType.CC_BY, LicenseType.PUBLIC_DOMAIN, LicenseType.PROPRIETARY,
36      })
37
38
39      def is_license_commercial(license_type: LicenseType) -> bool:
40          """True iff `license_type` is in the commercial whitelist (COMMERCIAL_LICENSES)."""
41          return license_type in COMMERCIAL_LICENSES
42
43
44      class VideoMetadata(BaseModel):
```

```
45          video_id: str = Field(..., description="Unique alphanumeric identifier for the
video")
46          sport: SportType
47          video_url: Optional[str] = Field(None, description="Remote URL of the video (if
available)")
48          local_path: Optional[str] = Field(None, description="Local path to video file (if
downloaded or custom)")
49          license_type: LicenseType = Field(
50              ...,
51              description="License of the VIDEO FOOTAGE. This ? not the annotation license ? "
52              "is what gates commercial use (is_commercial). Third-party / broadcast video is
"
53              "'Unknown' unless rights are actually held.",
54          )
55          license_url: Optional[str] = None
56          annotation_license: Optional[LicenseType] = Field(
57              None,
58              description="License of the ANNOTATIONS only (e.g. ShuttleSet's MIT). Recorded
for "
59              "provenance; it does NOT grant any rights to the video footage and is never used
"
60              "for commercial gating.",
61          )
62          is_commercial: bool = Field(False, description="Flag indicating if the VIDEO license
allows commercial use")
63          status: CurationStatus = CurationStatus.STAGING
64          match_name: str = Field(..., description="Name of the match/event")
65          fps: Optional[float] = Field(None, description="Frames per second")
66          resolution: Optional[str] = Field(None, description="Resolution string (e.g. 1080p,
720p)")
67
68          @model_validator(mode="after")
69          def _check_is_commercial_consistency(self) -> "VideoMetadata":
70              """B1/D7: STRICT consistency-check ? is_commercial MUST agree with license_type.
71
72              ``is_commercial`` is an independent bool with no link to ``license_type``, so a
73              caller (or a legacy DB row) could set ``is_commercial=True`` while
74              ``license_type=CC-BY-NC``. The export gate then ships non-commercial footage as
75              "commercially clean" into the indexer's training pool ? a license-compliance
76              landmine for a product whose commercial-clean licensing is non-negotiable.
77
78              This validator makes the invariant un-enforceable-violable: it recomputes the
79              expected flag from ``license_type`` via the whitelist and raises ValueError on
80              any mismatch. Owner-ratified strict (D7, 2026-06-24).
81              """
82              expected = is_license_commercial(self.license_type)
83              if self.is_commercial != expected:
84                  raise ValueError(
85                      f"is_commercial={self.is_commercial} is inconsistent with license_type="
86                      f"{self.license_type.value!r} (expected {expected} per the commercial "
87                      f"whitelist). Set is_commercial={expected}, or change license_type."
88                  )
89              return self
90
```

28. user (2026-07-02T04:49:19.712Z)

```
1       [build-system]
2       requires = ["setuptools>=61.0.0"]
3       build-backend = "setuptools.build_meta"
4
5       [project]
6       name = "sports-data-collector"
7       version = "0.1.0"
8       description = "A collector and curator for sports video datasets with sport-specific
```

```
schemas and license checks."
9       readme = "README.md"
10      requires-python = ">=3.9"
11      dependencies = [
12          "pydantic>=2.0.0",
13          "pyyaml>=6.0",
14          "tabulate>=0.9.0",
15          # Hardens CVAT XML/zip parsing against entity-expansion / billion-laughs
16          # attacks (see docs/CODE_HEALTH_PLAN.md §7 Tier C). BSD-2-Clause ?
17          # commercially-clean per the project's licensing rule.
18          "defusedxml>=0.7.0",
19      ]
20
21      [project.optional-dependencies]
22      # Video acquisition (`fetch` / crawler). Also requires the external binaries
23      # ffprobe (FFmpeg) and, for YouTube high-res, a JS runtime such as deno ?
24      # neither is pip-installable; see README "External requirements".
25      fetch = [
26          "yt-dlp>=2024.1.0"
27      ]
28      dev = [
29          "pytest>=7.0.0",
30          "pytest-cov>=4.0.0"
31      ]
32      # Per-video observability reports (SOFT dependency: ingestion degrades to a no-op
33      # if absent). Shared, permissively-licensed report toolkit. Pin a tag once the
34      # private repo is published, e.g.:
35      #   obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
36      # For local development install it editable from a sibling checkout:
37      #   pip install -e ../sports-obsreport --no-build-isolation
38      report = [
39          "obsreport>=0.1.3",
40      ]
41
```

29. user (2026-07-02T04:49:22.217Z)

```
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-curate-verify\README.md:65:python -m
src.cli.ingest --sport badminton --parser coachai --file-path data/staging_coachai_match.csv
--license CC-BY-NC-SA --video-id staging_match_non_comm
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-curate-verify\src\cli\curate.py:518:
if not video.video_url or "watch?v=example" in video.video_url:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-curate-verify\src\cli\ingest.py:23:
parser.add_argument("--video-url", default="https://youtube.com/watch?v=example", help="URL to
the video")
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\scratchpad\wt-curate-
verify\src\parsers\badminton\coachai.py:71:        video_url = kwargs.get("video_url") or
"https://youtube.com/watch?v=example"
```

30. assistant (2026-07-02T04:50:31.440Z)

Now let me look at test patterns, config, and check for lint configs.

31. user (2026-07-02T04:50:32.743Z)

```
1       """End-to-end coverage for the SportsDataCLI commands.
2
3       Each test runs against a throwaway config + SQLite DB in tmp_path, so the real
4       data/ directory is never touched. Network (yt-dlp) and the browser are mocked.
5       """
```

```
      import csv as _csv
      import json
      from types import SimpleNamespace
      from unittest import mock

      import pytest
      import yaml

      from src.cli.curate import SportsDataCLI
      from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
      from src.core.schemas.badminton import BadmintonRally


      @pytest.fixture
      def cli(tmp_path):
          cfg = {
              "database_path": str(tmp_path / "test.db"),
              "video_storage": {
                  "local_dir": str(tmp_path / "videos"),
                  "max_download_size_mb": 100,
                  "auto_download": False,
                  "max_height": None,
              },
              "commercial_licenses": ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-
Domain", "Proprietary"],
              "supported_sports": ["badminton", "tennis", "tabletennis", "pickleball",
"cricket"],
          }
          cfg_path = tmp_path / "config.yaml"
          cfg_path.write_text(yaml.safe_dump(cfg), encoding="utf-8")
          return SportsDataCLI(config_path=str(cfg_path))


      def _import_args(**kw):
          base = dict(
              sport=None, video_path="nonexistent.mp4", annotations_path=None,
              match_name=None, video_id=None, license="Proprietary", fps=30.0,
resolution="1080p",
          )
          base.update(kw)
          return SimpleNamespace(**base)


      def _write_csv(path, header, rows):
          with open(path, "w", newline="", encoding="utf-8") as f:
              w = _csv.writer(f)
              w.writerow(header)
              for r in rows:
                  w.writerow(r)


      # ---------------------------------------------------------------------- #
      # import-local: every sport, both JSON and CSV (covers dispatch + parse)       #
      # ---------------------------------------------------------------------- #

      def test_import_local_badminton_json(cli, tmp_path):
          p = tmp_path / "bad.json"
          p.write_text(json.dumps([
              {"rally_number": 1, "start_time": 10.0, "end_time": 20.0, "shots_count": 5,
"ending_reason": "winner"},
              {"rally_number": 2, "start_time": 25.0, "end_time": 30.0, "shots_count": 1,
"ending_reason": "service_fault"},
          ]), encoding="utf-8")
          cli.import_local(_import_args(sport="badminton", annotations_path=str(p),
video_id="bad1", match_name="Bad"))
```

```
        assert len(cli.db.get_badminton_rallies("bad1")) == 2


    def test_import_local_pickleball_csv(cli, tmp_path):
        p = tmp_path / "pb.csv"
        _write_csv(p, ["rally_number", "start_time", "end_time", "shots_count",
"ending_reason"],
                   [[1, 10.0, 20.0, 6, "winner"], [2, 25.0, 27.0, 1, "service_fault"]])
        cli.import_local(_import_args(sport="pickleball", annotations_path=str(p),
video_id="pb1", match_name="PB"))
        assert len(cli.db.get_pickleball_rallies("pb1")) == 2


    def test_import_local_tabletennis_json(cli, tmp_path):
        p = tmp_path / "tt.json"
        p.write_text(json.dumps([
            {"rally_number": 1, "start_time": 5.0, "end_time": 9.0, "shots_count": 7,
"ending_reason": "winner"},
        ]), encoding="utf-8")
        cli.import_local(_import_args(sport="tabletennis", annotations_path=str(p),
video_id="tt1", match_name="TT"))
        assert len(cli.db.get_tabletennis_rallies("tt1")) == 1


    def test_import_local_tennis_csv(cli, tmp_path):
        p = tmp_path / "tn.csv"
        _write_csv(p, ["point_number", "start_time", "end_time", "server", "ending_type"],
                   [[1, 10.0, 20.0, "player1", "winner"]])
        cli.import_local(_import_args(sport="tennis", annotations_path=str(p),
video_id="tn1", match_name="TN"))
        assert len(cli.db.get_tennis_points("tn1")) == 1


    def test_import_local_cricket_json(cli, tmp_path):
        p = tmp_path / "ck.json"
        p.write_text(json.dumps([
            {"over": 0, "ball": 1, "start_time": 5.0, "end_time": 15.0, "runs": 4, "wicket":
False},
        ]), encoding="utf-8")
        cli.import_local(_import_args(sport="cricket", annotations_path=str(p),
video_id="ck1", match_name="CK"))
        assert len(cli.db.get_cricket_deliveries("ck1")) == 1


    # ---------------------------------------------------------------------- #
    # import-local: error / edge branches                                    #
    # ---------------------------------------------------------------------- #

    def test_import_local_unsupported_sport_exits(cli, tmp_path):
        p = tmp_path / "x.json"
        p.write_text("[]", encoding="utf-8")
        with pytest.raises(SystemExit):
            cli.import_local(_import_args(sport="football", annotations_path=str(p)))


    def test_import_local_missing_annotations_exits(cli):
        with pytest.raises(SystemExit):
            cli.import_local(_import_args(sport="badminton",
annotations_path="does_not_exist.json"))


    def test_import_local_overlap_fails_validation(cli, tmp_path):
        p = tmp_path / "ov.csv"
        _write_csv(p, ["rally_number", "start_time", "end_time"],
```

32. user (2026-07-02T04:50:36.139Z)

Sports Data Collector Configuration

Path to the SQLite database

```
database_path: "./data/sports_metadata.db"
```

Storage configuration for downloaded videos

```
video_storage:
  # Folder where downloaded or local videos will be stored
  local_dir: "./data/videos"
  # Maximum allowed size for automatic downloads (in Megabytes)
  max_download_size_mb: 12000
  # Whether to attempt video downloads automatically during ingestion
  auto_download: true
  # Optional cap on download resolution height (e.g. 1080 to avoid pulling 4K).
  # Leave unset/null to always fetch the best resolution available.
  max_height: null
```

Whitelist of licenses considered safe for commercial use

```
commercial_licenses:
  - "MIT"
  - "Apache-2.0"
  - "BSD"
  - "CC0"
  - "CC-BY"
  - "Public-Domain"
  - "Proprietary"  # Used for custom user-added local videos
```

Ingestion limits

```
ingestion_limits:
  max_total_download_size_mb: 60000
  max_total_video_count: 10
```

Whitelist of all sports supported in the system

```
supported_sports:
  - "badminton"
  - "tennis"
  - "tabletennis"
  - "pickleball"
  - "cricket"
```

Validation / parsing thresholds. Each has a rationale comment; override here only with a deliberate reason. See docs/CODE_HEALTH_PLAN.md §7 Tier C.

```
validation:
  coachai:
    # Seconds subtracted from the first stroke (serve) to get the rally start.
    # 1.0s covers the serve wind-up without eating the previous rally's tail.
    start_buffer: 1.0
    # Seconds added to the last stroke to get the rally end. 2.0s covers the
    # shuttle settling / umpire call after the last shot.
    end_buffer: 2.0
  rally_cvat:
    # Rallies with fewer than this many shots/second are flagged "shot_density"
    # (medium) ? the span may be inflated by a stray box. 0.3/s ? a shot every
    # ~3.3s, well below any real rally tempo.
    min_shots_per_second: 0.3
    # A gap between consecutive rallies longer than this is flagged
    # "possible_missed_rally" (medium). 15.0s catches an omitted rally that left
```

```
    # a visible hole without false-firing on normal between-rally breaks.
    max_gap_s: 15.0
    # The ±boundary bar (seconds) rally first/last boxes must meet. 0.3s is the
    # golden-quality tolerance from Brief §4; a coarser sampling grid makes it
    # structurally unmeetable (the boundary_grid_snap blocker).
    boundary_tolerance_s: 0.3
ci.yml
name: CI

on:
  push:
    branches: [master]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -e ".[dev]"
      - name: Run tests
        run: python -m pytest -q
.
..
.git
.gitattributes
.github
.gitignore
README.md
cleanup.py
config
data
C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-curate-verify/tests/test_db.py
```

33. assistant (2026-07-02T04:50:42.358Z)

CI is pytest-only; tests use tmp_path + throwaway config. Now let me verify the two G: candidate files against DB metadata and the golden label CSVs.

34. user (2026-07-02T04:50:47.014Z)


```
{
    "programs": [

    ],
    "stream_groups": [

    ],
    "streams": [
        {
            "width": 1920,
            "height": 1080,
            "r_frame_rate": "30000/1001",
            "avg_frame_rate": "30000/1001",
            "nb_frames": "13418"
        }
    ],
    "format": {
```

```
        "duration": "447.713933",
        "size": "196182380"
    }
}
```

35. user (2026-07-02T04:50:47.309Z)

```
29,359.664,366.661,winner,badminton
30,374.162,381.659,winner,badminton
31,394.661,399.412,winner,badminton
===
94,1370.997,1378.001,unforced_error,badminton
95,1385.257,1391.510,forced_error,badminton
96,1398.257,1405.757,unforced_error,badminton
===
rally_number,start_time,end_time,ending_reason,sport
1,37.434,72.194,winner,badminton
```

36. user (2026-07-02T04:51:04.842Z)

```
{
    "programs": [

    ],
    "stream_groups": [

    ],
    "streams": [
        {
            "width": 1920,
            "height": 1080,
            "r_frame_rate": "60000/1001",
            "avg_frame_rate": "60000/1001",
            "nb_frames": "84480"
        }
    ],
    "format": {
        "duration": "1409.408000",
        "size": "11445052772"
    }
}
```

37. user (2026-07-02T04:51:52.680Z)

Normalize line endings: store text as LF in the repo, check out native.

```
* text=auto
```

Treat data/media as binary ? never line-ending-convert or text-diff these.

```
*.db binary
*.sqlite binary
*.sqlite3 binary
*.mp4 binary
*.png binary
*.jpg binary
*.jpeg binary
staging_match_non_comm rallies: 2
shuttleset_12 rallies: 79
```

38. user (2026-07-02T04:52:19.918Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-curate-verify/src/cli/curate.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

39. user (2026-07-02T04:52:43.201Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-curate-verify/src/cli/curate.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

40. user (2026-07-02T04:52:52.654Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-curate-verify/src/cli/curate.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

41. user (2026-07-02T04:52:59.483Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-curate-verify/src/cli/curate.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

42. assistant (2026-07-02T04:53:04.160Z)

Now the tests. Following the `test_cli.py` fixture pattern, in a new `tests/test_verify.py`.

43. assistant (2026-07-02T04:53:29.987Z)

```
"""Tests for `curate verify` ? the registry-vs-disk local_path audit.

Every test runs against a throwaway config + SQLite DB in tmp_path (same pattern
as test_cli.py), so the real data/ directory is never touched.
"""
from types import SimpleNamespace

import pytest
import yaml

from src.cli.curate import CurationError, SportsDataCLI
from src.core.schemas.base import CurationStatus, LicenseType, SportType, VideoMetadata


@pytest.fixture
def cli(tmp_path):
    cfg = {
        "database_path": str(tmp_path / "test.db"),
        "video_storage": {
            "local_dir": str(tmp_path / "videos"),
            "max_download_size_mb": 100,
            "auto_download": False,
            "max_height": None,
        },
        "commercial_licenses": ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-Domain",
"Proprietary"],
        "supported_sports": ["badminton", "tennis", "tabletennis", "pickleball", "cricket"],
    }
    cfg_path = tmp_path / "config.yaml"
    cfg_path.write_text(yaml.safe_dump(cfg), encoding="utf-8")
    return SportsDataCLI(config_path=str(cfg_path))


def _video(video_id, *, local_path=None, video_url=None, sport=SportType.BADMINTON,
           status=CurationStatus.CURATED, license_type=LicenseType.PROPRIETARY):
    return VideoMetadata(
        video_id=video_id, sport=sport, video_url=video_url, local_path=local_path,
        license_type=license_type, license_url=None,
        is_commercial=(license_type == LicenseType.PROPRIETARY),
        status=status, match_name=video_id,
```

```
    )


def _verify_args(**kw):
    base = dict(sport=None, status="all", strict=False)
    base.update(kw)
    return SimpleNamespace(**base)


# ---------------------------------------------------------------------- #
# verify_local_paths: classification                                     #
# ---------------------------------------------------------------------- #


def test_verify_classifies_ok_missing_and_pathless(cli, tmp_path):
    real = tmp_path / "real.mp4"
    real.write_bytes(b"\x00" * 16)
    cli.db.insert_video(_video("v_ok", local_path=str(real)))
    cli.db.insert_video(_video("v_gone", local_path=str(tmp_path / "moved_away.mp4")))
    cli.db.insert_video(_video("v_urlonly", video_url="https://www.youtube.com/watch?v=abc",
                               license_type=LicenseType.UNKNOWN))

    report = cli.verify_local_paths()

    assert [v.video_id for v in report.ok] == ["v_ok"]
    assert [v.video_id for v in report.missing] == ["v_gone"]
    assert [v.video_id for v in report.pathless] == ["v_urlonly"]
    assert report.n_videos == 3


def test_verify_covers_every_status_exactly_once(cli, tmp_path):
    gone = str(tmp_path / "gone.mp4")
    cli.db.insert_video(_video("v_staging", local_path=gone, status=CurationStatus.STAGING,
                               license_type=LicenseType.UNKNOWN))
    cli.db.insert_video(_video("v_curated", local_path=gone, status=CurationStatus.CURATED))
    cli.db.insert_video(_video("v_rejected", local_path=gone, status=CurationStatus.REJECTED,
                               license_type=LicenseType.UNKNOWN))

    report = cli.verify_local_paths()
    assert sorted(v.video_id for v in report.missing) == ["v_curated", "v_rejected",
"v_staging"]
    assert report.n_videos == 3  # one bucket entry per row, no dupes

    only_curated = cli.verify_local_paths(statuses=[CurationStatus.CURATED])
    assert [v.video_id for v in only_curated.missing] == ["v_curated"]


def test_verify_relative_path_resolves_against_cwd(cli, tmp_path, monkeypatch):
    (tmp_path / "videos").mkdir(exist_ok=True)
    (tmp_path / "videos" / "rel.mp4").write_bytes(b"\x00")
    cli.db.insert_video(_video("v_rel", local_path="./videos/rel.mp4"))

    monkeypatch.chdir(tmp_path)
    assert [v.video_id for v in cli.verify_local_paths().ok] == ["v_rel"]

    # From anywhere else the same relative path is dangling ? same semantics as
    # every other local_path consumer (export / play harness).
    monkeypatch.chdir(tmp_path / "videos")
    assert [v.video_id for v in cli.verify_local_paths().missing] == ["v_rel"]


def test_verify_sport_filter_and_unknown_sport(cli, tmp_path):
    cli.db.insert_video(_video("v_bad", local_path=str(tmp_path / "gone.mp4")))
    cli.db.insert_video(_video("v_tt", sport=SportType.TABLE_TENNIS,
```

```
                                local_path=str(tmp_path / "gone2.mp4")))

    report = cli.verify_local_paths(sport="badminton")
    assert [v.video_id for v in report.missing] == ["v_bad"]

    with pytest.raises(CurationError):
        cli.verify_local_paths(sport="football")


# ---------------------------------------------------------------------------
# CLI wrapper: output + exit codes                                          #
# ---------------------------------------------------------------------------


def test_verify_cli_reports_and_exits_zero_without_strict(cli, tmp_path, capsys):
    cli.db.insert_video(_video("v_gone", local_path=str(tmp_path / "gone.mp4")))

    cli.verify(_verify_args())  # must NOT raise SystemExit without --strict

    out = capsys.readouterr().out
    assert "MISSING : 1" in out
    assert "v_gone" in out
    assert "Dangling rows" in out


def test_verify_cli_strict_exits_nonzero_on_dangling(cli, tmp_path):
    cli.db.insert_video(_video("v_gone", local_path=str(tmp_path / "gone.mp4")))

    with pytest.raises(SystemExit) as exc:
        cli.verify(_verify_args(strict=True))
    assert exc.value.code == 1


def test_verify_cli_strict_passes_when_all_resolve(cli, tmp_path, capsys):
    real = tmp_path / "real.mp4"
    real.write_bytes(b"\x00")
    cli.db.insert_video(_video("v_ok", local_path=str(real)))

    cli.verify(_verify_args(strict=True))  # no dangling rows -> no SystemExit

    out = capsys.readouterr().out
    assert "Resolved OK : 1" in out
    assert "MISSING : 0" in out


def test_verify_cli_status_filter(cli, tmp_path, capsys):
    cli.db.insert_video(_video("v_staging_gone", local_path=str(tmp_path / "gone.mp4"),
                               status=CurationStatus.STAGING, license_type=LicenseType.UNKNOWN))

    # Only curated rows requested -> the staging dangler is out of scope.
    cli.verify(_verify_args(status="curated", strict=True))
    assert "No matching videos registered" in capsys.readouterr().out


def test_verify_cli_flags_placeholder_url_rows(cli, capsys):
    cli.db.insert_video(_video("v_placeholder", video_url="https://youtube.com/watch?v=example",
                               status=CurationStatus.STAGING, license_type=LicenseType.UNKNOWN))
    cli.db.insert_video(_video("v_urlonly", video_url="https://www.youtube.com/watch?v=real123",
                               status=CurationStatus.STAGING, license_type=LicenseType.UNKNOWN))

    cli.verify(_verify_args())

    out = capsys.readouterr().out
    assert "no local_path and no real source URL" in out   # the placeholder row
```

```
    assert "no local_path (URL-only entry)" in out          # the real-URL row
```

44. user (2026-07-02T04:53:30.063Z)

```
File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-curate-
verify/tests/test_verify.py (file state is current in your context ? no need to Read it back)
```

45. assistant (2026-07-02T04:53:38.863Z)

Now run the new tests plus the existing CLI tests.

46. user (2026-07-02T04:53:44.559Z)

```
....................................                                     [100%]
40 passed in 2.41s
```

47. assistant (2026-07-02T04:53:49.583Z)

All pass. Now document the command in the README.

48. user (2026-07-02T04:53:55.546Z)

```
<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>
```

49. user (2026-07-02T04:54:03.356Z)

```
96      ### 6. Export Annotations as Indexer-Ready Eval/Training Sets
97      To turn curated annotations into ground-truth files the badminton-highlight-indexer can
score and train against, export per-video `start,end` CSVs plus a `manifest.json`:
98      ```bash
99      # Curated + commercially-safe videos across all supported sports (default):
100     python -m src.cli.curate export --out-dir data/eval_export
101
102     # One sport, also including staging entries:
103     python -m src.cli.curate export --sport badminton --include-staging
104     ```
105     The exporter writes `<out-dir>/<sport>/<video_id>.csv` (decimal-second `start,end` rows)
and a `manifest.json` pairing each CSV with its video `local_path`/`video_url`, fps, resolution,
and license. The indexer keys videos by **file MD5**, not by our `video_id`, so the downstream
eval/training run must process the exact video file named in the manifest. Flags: `--sport`
(restrict to one sport), `--include-staging` (default: curated only), `--include-noncommercial`
(default: commercial-safe only).
106
107     ---
108
109     ## Configuration
110
111     Adjust settings in `config/config.yaml`:
112     - **`database_path`**: Path to the SQLite database.
113     - **`video_storage`**: Local folder path and automatic download size limit.
114     - **`commercial_licenses`**: Whitelist of licenses allowed for commercial use.
115     - **`supported_sports`**: Allowed sports categories.
116
117     ---
118
119     ## Documentation
120
```

50. user (2026-07-02T04:54:17.913Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-curate-verify/README.md has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

51. assistant (2026-07-02T04:54:35.714Z)

Now the DB surgery on the worktree copy. I'll use the repo's own `SportsDatabase.delete_video` for the cascade delete, plus targeted UPDATEs.

52. user (2026-07-02T04:54:46.710Z)

```
re-pointed adarsh_avi_singles -> G:\My Drive\Sharing\Badminton\Boxhill\June 1 2026\Adarsh and
Avi.MP4
re-pointed mahadevpura_singles -> G:\My Drive\Sharing\Golden Annotations\MahadevpuraSingles.MP4
shuttleset_12: placeholder video_url cleared (row + 79 rallies kept)
staging_match_non_comm: purged (demo placeholder row)
('shuttleset_12', None, None)
('adarsh_avi_singles', None, 'G:\\My Drive\\Sharing\\Badminton\\Boxhill\\June 1 2026\\Adarsh and
Avi.MP4')
('mahadevpura_singles', None, 'G:\\My Drive\\Sharing\\Golden
Annotations\\MahadevpuraSingles.MP4')
total videos: 30
shuttleset_12 rallies: 79
```

53. user (2026-07-02T04:54:59.651Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 7, in <module>
    cli.verify(SimpleNamespace(sport=None, status='all', strict=False))
    ^^^^^^^^^^
AttributeError: 'SportsDataCLI' object has no attribute 'verify'
```

54. assistant (2026-07-02T04:55:08.687Z)

The main checkout's `src` (old code) shadowed the worktree on sys.path. Fix the import order.

55. user (2026-07-02T04:55:12.995Z)

```
=== Registry local_path verification ===
+----------------------+----------+---------+-------------------------------------------------
-----------------------------------------------+
| Video ID             | Status   | File    | Local path
|
+======================+==========+=========+=================================================
===============================================+
| adarsh_avi_singles   | curated  | OK      | G:\My Drive\Sharing\Badminton\Boxhill\June 1
2026\Adarsh and Avi.MP4                          |
+----------------------+----------+---------+-------------------------------------------------
-----------------------------------------------+
| gbaaddy              | curated  | MISSING | C:\Users\avidu\Projects\Annotation
Setup\Collect\Golden Labelled\gBaaddy20May2026.MP4          |
+----------------------+----------+---------+-------------------------------------------------
-----------------------------------------------+
| kushagra_singles     | curated  | MISSING | C:\Users\avidu\Projects\Annotation
Setup\Collect\Golden Labelled\KushagraYashAviFirstVideo.MP4 |
+----------------------+----------+---------+-------------------------------------------------
-----------------------------------------------+
| largetest_doubles    | curated  | MISSING | C:\Users\avidu\Projects\Annotation
Setup\Collect\Golden Labelled\LargeTestVideo.MP4            |
+----------------------+----------+---------+-------------------------------------------------
-----------------------------------------------+
| mahadevpura_singles  | curated  | OK      | G:\My Drive\Sharing\Golden
Annotations\MahadevpuraSingles.MP4                                |
+----------------------+----------+---------+-------------------------------------------------
-----------------------------------------------+
| my_custom_local_match | curated  | MISSING | data/videos/my_custom_match.mp4
|
```

shuttleset_1	staging	OK	./data/videos\shuttleset_1.mp4
shuttleset_10	staging	OK	./data/videos\shuttleset_10.mp4
shuttleset_11	staging	OK	./data/videos\shuttleset_11.mp4
shuttleset_12	staging	-	no local_path and no real source URL
shuttleset_13	staging	OK	./data/videos\shuttleset_13.mp4
shuttleset_14	staging	OK	./data/videos\shuttleset_14.mp4
shuttleset_15	staging	OK	./data/videos\shuttleset_15.mp4
shuttleset_16	staging	OK	./data/videos\shuttleset_16.mp4
shuttleset_17	staging	OK	./data/videos\shuttleset_17.mp4
shuttleset_18	staging	OK	./data/videos\shuttleset_18.mp4
shuttleset_19	staging	OK	./data/videos\shuttleset_19.mp4
shuttleset_2	staging	OK	./data/videos\shuttleset_2.mp4
shuttleset_20	staging	OK	./data/videos\shuttleset_20.mp4
shuttleset_21	curated	OK	./data/videos\shuttleset_21.mp4
shuttleset_22	staging	OK	./data/videos\shuttleset_22.mp4
shuttleset_23	staging	OK	./data/videos\shuttleset_23.mp4

```
-------------------------------------------+
| shuttleset_24         | staging  | OK      | ./data/videos\shuttleset_24.mp4
|
+----------------------+----------+---------+-----------------------------------------------
-------------------------------------------+
| shuttleset_25         | staging  | OK      | ./data/videos\shuttleset_25.mp4
|
+----------------------+----------+---------+-----------------------------------------------
-------------------------------------------+
| shuttleset_3          | staging  | OK      | ./data/videos\shuttleset_3.mp4
|
+----------------------+----------+---------+-----------------------------------------------
-------------------------------------------+
| shuttleset_4          | staging  | OK      | ./data/videos\shuttleset_4.mp4
|
+----------------------+----------+---------+-----------------------------------------------
-------------------------------------------+
| shuttleset_6          | staging  | OK      | ./data/videos\shuttleset_6.mp4
|
+----------------------+----------+---------+-----------------------------------------------
-------------------------------------------+
| shuttleset_8          | staging  | OK      | ./data/videos\shuttleset_8.mp4
|
+----------------------+----------+---------+-----------------------------------------------
-------------------------------------------+
| shuttleset_9          | staging  | OK      | ./data/videos\shuttleset_9.mp4
|
+----------------------+----------+---------+-----------------------------------------------
-------------------------------------------+
| testlarge_short       | curated  | MISSING | C:\Users\avidu\Projects\Annotation
Setup\Collect\Golden Labelled\TestLargeVideo.MP4            |
+----------------------+----------+---------+-----------------------------------------------
-------------------------------------------+

Resolved OK : 24   MISSING : 5   No local_path : 1

Dangling rows (registered local_path does not resolve on disk):
  - my_custom_local_match: data/videos/my_custom_match.mp4
  - kushagra_singles: C:\Users\avidu\Projects\Annotation Setup\Collect\Golden
Labelled\KushagraYashAviFirstVideo.MP4
  - largetest_doubles: C:\Users\avidu\Projects\Annotation Setup\Collect\Golden
Labelled\LargeTestVideo.MP4
  - gbaaddy: C:\Users\avidu\Projects\Annotation Setup\Collect\Golden
Labelled\gBaaddy20May2026.MP4
  - testlarge_short: C:\Users\avidu\Projects\Annotation Setup\Collect\Golden
Labelled\TestLargeVideo.MP4
Re-point these rows at the real file location (or re-`fetch` the video); until then `export`
emits md5=None for them, breaking the indexer join.
```

56. assistant (2026-07-02T04:55:23.117Z)

The command works end-to-end. Now lint the changed files.

57. user (2026-07-02T04:55:26.486Z)

```
ruff 0.15.16
All checks passed!
```

58. assistant (2026-07-02T04:55:41.238Z)

Ruff clean; no mypy config exists in this repo (pyproject has no tool sections). Now commit ?
verifying branch state first per the safety protocol.

59. user (2026-07-02T04:55:43.043Z)

```
fix/curate-verify-registry-rot
adcd206 fix(db): set SQLite busy_timeout so concurrent writers wait, not error (P9) (#53)
 M README.md
 M data/sports_metadata.db
 M src/cli/curate.py
?? tests/test_verify.py
```

60. user (2026-07-02T04:55:55.937Z)

```
warning: in the working copy of 'tests/test_verify.py', LF will be replaced by CRLF the next
time Git touches it
[fix/curate-verify-registry-rot aef4621] feat(curate): add `verify` local_path audit; fix
golden-corpus registry rot
 4 files changed, 297 insertions(+)
 create mode 100644 tests/test_verify.py
aef4621 feat(curate): add `verify` local_path audit; fix golden-corpus registry rot
```

61. assistant (2026-07-02T04:56:02.107Z)

Only my commit on top of origin/master. Push and open the PR.

62. user (2026-07-02T04:56:07.071Z)

```
remote:
remote: Create a pull request for 'fix/curate-verify-registry-rot' on GitHub by visiting:
remote:      https://github.com/Khelsutra/sports-data-collector/pull/new/fix/curate-verify-
registry-rot
remote:
branch 'fix/curate-verify-registry-rot' set up to track 'origin/fix/curate-verify-registry-rot'.
To https://github.com/Khelsutra/sports-data-collector.git
 * [new branch]      fix/curate-verify-registry-rot -> fix/curate-verify-registry-rot
```

63. user (2026-07-02T04:56:44.280Z)

https://github.com/Khelsutra/sports-data-collector/pull/55

64. user (2026-07-02T04:57:13.190Z)

Structured output provided successfully